

Modern C++: From Fundamentals to Advanced Design

A Practical Guide for Students and Professionals

Dr. Danish Khan

Work in Progress — August 2026

Contents

Contents

1 Arrays	1
Introduction	
C-Style Single Dimensional Arrays	
C-Style Multidimensional Arrays	
Functions and C-Style Arrays	
From C-Style Arrays to Modern C++ Arrays	
The std::vector Container	
2 Classes & Inheritance	55
3 Virtual Functions & Abstract Classes	57
Virtual Functions	
Virtual Destructors	
Abstract Class	
Examples - A Polymorphic Design	
Concept Check	
4 Linked Lists	81
Basic Concept of Linked Lists	
Linked Lists Using Smart Pointers	
Linked Lists Using the STL	
Applications of Linked Lists	
5 Stacks & Queues	111
6 Hash Tables	113
Hash Tables	
Principles of Hash Function Design	
Principles of Compression Function Design	
Hash Collisions: Inevitability and Impact	

Collision Resolution Strategies
Hash Table Implementations
Hash Table Applications

7 Searching algorithms 163

Linear Search Algorithm
Binary Search Algorithm

8 Appendix A 177

Vectors
Linked Lists
Hash Tables

9 References 189

Preface

This book is written for students learning C++ in an academic environment and professionals transitioning to modern C++.

It emphasizes: - Clear mental models - RAII and resource safety - Modern best practices (C++11 and beyond)

1

Arrays

Introduction	2
C-Style Single Dimensional Arrays	5
C-Style Multidimensional Arrays	18
Functions and C-Style Arrays	29
From C-Style Arrays to Modern C++ Arrays	37
The <code>std::vector</code> Container	49

Imagine walking down a long hallway in an apartment building. Along one wall is a row of identical mailboxes, all neatly aligned and painted the same color. Each mailbox has a number on it: 0, 1, 2, 3, and so on.

When a resident wants to store a letter, they do not search through every box. Instead, they walk directly to the mailbox with the correct number and place the letter inside. Retrieving a letter works the same way—go straight to the numbered box and open it. This is fast because each mailbox is in a known, fixed position.

An array in C++ works in much the same way. When an array is created, the program sets aside a fixed number of memory slots, all of the same size and placed contiguously in memory. Each slot is assigned an index, starting from zero. Using this index, the program can directly access any element without inspecting the others.

Just like the mailbox hallway, the number of boxes is fixed once they are installed. You may replace what is inside a mailbox, but you cannot add a new one in the middle or extend the hallway on the go. This fixed layout is what makes arrays simple, efficient, and predictable, but also inflexible when the amount of data needs to grow.

Why Start with C-Style Arrays?

This chapter introduces C-style arrays to develop fundamental concepts such as memory layout, indexing, and pointer arithmetic. Although C-style arrays provide a transparent view of how data is stored and accessed at the memory level, modern C++ offers standard library array data structures that serve as safer and more expressive alternatives. These data structures, including `std::array` and `std::vector`, were introduced with C++11 and are discussed later in the chapter; they should be preferred in production-quality C++ code.

C++11 is a major revision of the C++ programming language standard, officially published in 2011. It introduced significant language and library features—such as uniform initialization, type inference with `auto`, lambda expressions, and improved standard library components—that made C++ safer, more expressive, and better suited for modern software development.

C++11

1.1 Introduction

Consider writing a program that records the exam scores of a class. At first, there are only a few students, so using separate variables seems manageable. But as the class grows, the program quickly becomes cluttered with dozens of nearly identical variables. Making changes—such as calculating an average or finding the highest score—becomes tedious and error-prone. What is missing is a way to treat many related values as a single, organized group.

This is where arrays become essential. Arrays allow a program to store multiple values of the same type under one name, arranged in a predictable order. Instead of han-

dling each value individually, the program can work with the collection as a whole—processing elements one by one using loops and applying algorithms uniformly. This makes tasks such as searching, sorting, and aggregating data both simpler and clearer.

Arrays are particularly useful when the amount of data is known in advance and does not change during execution. In such cases, allocating a fixed block of memory once is efficient and reliable. Because array elements are stored contiguously in memory, the program can access any element directly using its index, making operations fast and predictable.

Beyond convenience, arrays also matter when working close to the hardware. Many systems—such as embedded devices, image buffers, and performance-critical applications—require precise control over memory layout. Arrays provide this control by mapping cleanly to contiguous memory, which aligns well with how hardware and caches operate.

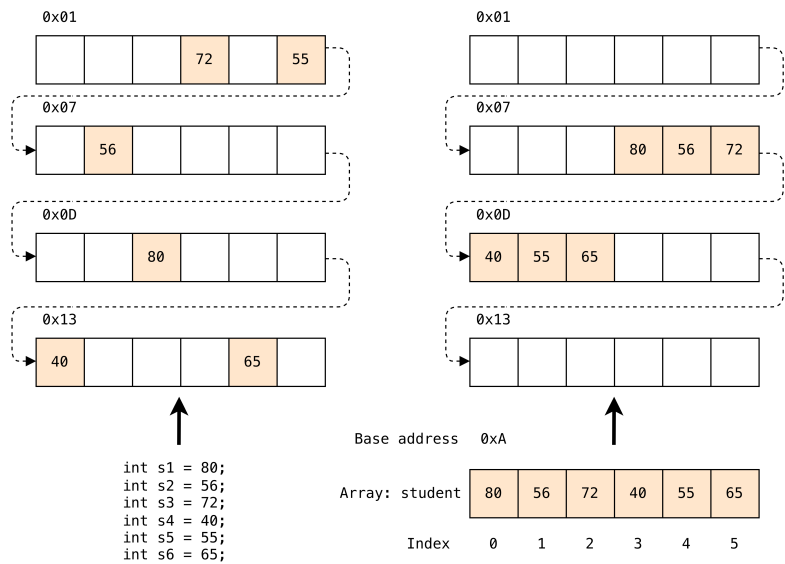


FIGURE 1.1 Variables and arrays - memory map

The left-hand side of Figure 1.1 depicts a fictitious student's grades stored as separate variables. Each variable (such as s1, s2, etc.) resides in its own memory block. For simplicity, we assume a byte-addressable memory system where each memory block is 1 byte and the integer data type also consumes 1 byte. Memory is laid out linearly, with each block assigned a hexadecimal address that increments by one byte.

In contrast, the right-hand side of the figure depicts the same values stored in contiguous memory locations and accessed using a single array name with an index that starts at 0.

i Physical Memory as a One-Dimensional Address Space

Some textbooks depict memory using a two-dimensional grid of cells to simplify visualization. While this representation may be pedagogically convenient, it is conceptually misleading. Physical memory is strictly linear, and each memory location is identified by a single, sequential address. Any two-dimensional view of memory is merely an abstraction used to help organize data conceptually and does not reflect the actual memory layout.

As discussed earlier, variable and array names are human-friendly abstractions used during program development. At runtime, these names are absent, and the generated machine code accesses memory directly via addresses. The following are the steps required when determining an array address.

- 1) An integer array which holds 6 integers or elements in general:
`int student[5];`
 The compiler records `arr` in its symbol table (name, type, size, scope).
- 2) The compiler decides where the array will live
 - Local array (inside a function) in stack
 - Global / static array in static storage (data segment)
 - Dynamic array in heap (allocator decides at runtime)
- 3) The compiler determines an array's base location at compile time. The compile time is the period when the compiler processes source code and makes decisions that are fixed before the program runs. The base location depends on whether the array is stored on the stack, in static storage, or on the heap.
- 4) At runtime, the program uses that base address directly (no name lookup)
- 5) When you write `int student[i]`, the compiler computes the element address using the following formula

$$\text{address} = \text{base} + i \times \text{sizeof}(T)$$

where *base* is the array base address (the address of the element at index 0), the index *i* is a user-given index, and *T* the size of the element's data type in Bytes.

Use Figure 1.1 and apply the above formula. The base address is `0xA`, the total number of elements are 5 and each integer size is 1 Byte (for this example only). The address of elements look like this:

The address of `student[0]` is `0xA`. The compiler allocates contiguous memory for six elements, with the address of the first element serving as the base address of the array.

The address of `student[1]` is `0xA + (1 x 1) = 0xB`.

The address of `student[2]` is `0xA + (2 x 1) = 0xC`.

Compile time

i Zero-Based Indexing in C++

C++ is a zero-based indexing language, meaning the first element of an array is accessed using index 0. A common programming error is using an index that falls outside the valid range, leading to out-of-bounds memory access.

The following section explores array addressing at the machine-code level. This material is intended for advanced readers and is not required for a basic understanding of arrays.

1.1.1 *Understanding Array Addressing Using Machine Code*

Need to write assembly code

1.2 C-Style Single Dimensional Arrays

When C++ was created, it was designed to build upon the C programming language rather than replace it. One of the core goals of C++ was compatibility with existing C code, which meant that many of C's fundamental constructs—including arrays—were carried forward unchanged. The familiar syntax for declaring arrays, such as `int arr[5];` originates directly from C.

As C++ evolved, the language introduced new abstractions and design philosophies that emphasized safety, expressiveness, and stronger type support. However, the original array mechanism remained part of the language because it provides a simple, low-level view of memory that maps directly to hardware. To distinguish this older, C-derived mechanism from newer C++ facilities, these built-in arrays are commonly referred to as C-style arrays.

The term “style” reflects not just the syntax, but the underlying programming model. C-style arrays do not track their own size, perform no bounds checking, and rely on raw memory addressing and pointer arithmetic. These characteristics are consistent with the design philosophy of C, which favors simplicity and direct control over memory.

There are two types of C-style arrays—static and dynamic—which are discussed in the following sections.

1.2.1 *Static Arrays*

Static arrays are arrays whose size is fixed and determined at compile time, allowing the compiler to allocate a contiguous block of memory with a predetermined layout before program execution begins. Once declared, the number of elements in a static array cannot be changed during runtime. Depending on their scope and storage duration, static arrays may reside in automatic (stack) storage or in static storage. When a static array is local to a function, it is placed on the stack because its compile-time

known size enables efficient allocation and deallocation as part of the function's stack frame. In all cases, however, the size of a static array remains constant.

Static arrays can be declared and initialized in multiple ways, as shown below:

a) `int array_name[5] = {10, 20, 30, 40}`

The array size is specified explicitly, and the values are initialized at declaration.

b) `int array_name[] = {10, 20, 30, 40}`

The compiler infers the array size from the number of initializer elements.

When an array is initialized using an initializer list, the elements are specified at compile time. An initializer list is a brace-enclosed, comma-separated sequence of values provided at the point of declaration.

[Initializer list](#)

Arrays can be accessed in multiple ways, as illustrated in Code Examples 1-1 through 1-4, depending on the programmer's preference.

Code Example 1-1: Accessing array elements using pointer arithmetic

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int number[] = {10, 20, 30, 40};
6     int length = sizeof(number) / sizeof(number[0]);
7
8     for (int i = 0; i < length; i++) {
9         cout << *(number + i) << endl;
10    }
11 }
```

In this example, the array name `number` decays to a pointer to its first element in most expressions. The expression `number + i` computes the address of the element at index `i` by advancing the base address by $i \times \text{sizeof}(\text{int})$ bytes, a calculation that is performed implicitly by the compiler in the background. Dereferencing this address using `*(number + i)` accesses the value of the corresponding array element. This demonstrates that array indexing `number[i]` is syntactic sugar for pointer arithmetic and dereferencing.

Array Indexing as Pointer Arithmetic

Array indexing in C++ is implemented internally using pointer arithmetic, where `arr[i]` is equivalent to `*(arr + i)`. In this process, the index `i` is not treated as a simple positional offset; instead, it is automatically scaled by the size of the array's data type (i.e., multiplied by `sizeof(data_type)`) to compute the correct memory address.

Code Example 1-2: Accessing array elements using counters

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int number[] = {10,20,30,40};
6     int length = sizeof(number) / sizeof(number[0]);
7     for (int i = 0; i < length; i++) {
8         cout << number[i] << endl;
9     }
10 }

```

This program illustrates how to determine the number of elements in a C-style array and iterate over them safely. Inside the `main()` function, an integer array named `number` is initialized with four values, with its size inferred by the compiler. The expression `sizeof(number) / sizeof(number[0])` computes the total number of elements in the array by dividing the total size of the array in bytes by the size of a single element. This value is stored in the variable `length` and used as the loop bound in a for loop. The loop starts at index 0 and iterates up to, but not including, `length`, reflecting C++'s zero-based indexing. During each iteration, the corresponding array element is accessed using `number[i]` and printed to the standard output using `cout`, ensuring that all elements are accessed within the valid range of the array.

Code Example 1-3: Accessing array elements using range-based (copy)

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int number[] = {10,20,30,40};
6     for (auto i:number) {
7         i = i * 2;
8         cout << i << endl;
9     }
10 }

```

The program defines a C-style array `number` containing four integers and then uses a range-based for loop to iterate over its elements. In the loop, `auto i : number` uses the `auto` keyword to instruct the compiler to automatically deduce the type of `i` from the elements of the array; since `number` contains integers, `i` is deduced as an `int`. On each iteration, the current element of the array is copied into the variable `i`; `i` is not an index and has no direct connection to the original array element. The statement `i = i * 2` therefore modifies only this local copy, not the values stored in the array itself. As a result, the program prints the doubled values (20, 40, 60, 80) to the output, but the original array `number` remains unchanged after the loop completes.

Code Example 1-4: Accessing array elements using range-based (reference)

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int number[] = {10,20,30,40};
6     for (auto& i:number) {
7         i = i * 2;
8         // i holds the memory address of each member of an array
9         // modifies the original
10    }
11    for (const auto& i:number) { // use for read-only
12        cout << i << endl; // displayed the modified array
13    }
14 }

```

This program defines an array `number` containing four integers and demonstrates how references work with range-based for loops. In the first loop, `for (auto& i : number)`, the variable `i` is a reference to each element of the array rather than a copy, meaning it refers directly to the original array elements in memory. As a result, the statement `i = i * 2` modifies the actual values stored in the array, doubling each element. In the second loop, `for (const auto& i : number)`, `i` is a read-only reference to each element, ensuring that the loop can access the values without accidentally modifying them. This loop simply prints the updated contents of the array (20, 40, 60, 80), confirming that the first loop permanently changed the original array.

Modifying Array Elements and Const Correctness

Array elements can be modified individually using a valid index, for example, `number[0] = 50`. However, this operation alters the original array, which may not always be desirable. Preserving the original values by copying the array into a temporary array avoids this issue, but such an approach may introduce additional memory usage and performance overhead. Declaring the array as `const` when it is used only for reading provides a safer alternative by preventing unintended modifications.

1.2.2 Dynamic Arrays

After discussing static arrays, it is natural to introduce dynamic arrays, which address a key limitation of static arrays—the requirement that the array size be known at compile time. In many practical programs, the amount of data to be stored is not known until the program is running, such as when the size depends on user input, file contents, or runtime computations. Dynamic arrays allow the array size to be determined at runtime, providing greater flexibility than static arrays.

Dynamic arrays are called dynamic because their memory allocation occurs during program execution rather than at compile time. However, this terminology can be misleading. While the size of a dynamic array is chosen at runtime, the size remains fixed once the array is allocated. The array cannot grow or shrink automatically after allocation. For this reason, dynamic arrays do not support true dynamic resizing.

A more precise way to describe dynamic arrays is to view them as run-time static arrays: their size is selected at runtime, but once allocated, they behave like static arrays with a fixed size and contiguous memory layout. Understanding this distinction helps avoid confusion between dynamic allocation and dynamic resizing, the latter of which is provided by higher-level C++ standard library containers such as `std::vector`.

From a memory management perspective, static and dynamic arrays also differ in where they are stored. Static arrays declared within a function are typically allocated on the stack, where memory is managed automatically and released when the function exits. In contrast, dynamic arrays are allocated on the heap memory, a larger memory region intended for objects whose size or lifetime cannot be determined at compile time. Heap-allocated memory persists until it is explicitly released by the programmer, making dynamic arrays more flexible but also requiring careful memory management to avoid leaks. Despite this difference in storage location, both static and dynamic arrays share the same fundamental properties: contiguous memory layout, lack of built-in size tracking, and absence of bounds checking.

Heap memory

Releasing Heap Storage for C-Style Dynamic Arrays

When using C-style dynamic arrays allocated on the heap (for example, via `new[]`), it is the programmer's responsibility to explicitly release the allocated memory using `delete[]` once it is no longer needed. Failing to do so results in memory leaks, as heap memory is not automatically reclaimed when a scope ends. In contrast, modern C++ standard library containers such as `std::vector` manage memory automatically.

Dynamic arrays may be defined using multiple syntactic forms, spanning both the pre-C++11 and post-C++11 eras.

a) Basic dynamic allocation (uninitialized elements)

```
int* arr = new int[n];
```

The statement `int* arr = new int[n];` allocates a contiguous block of memory on the heap to store `n` integers at runtime and stores the address of the first element in the pointer `arr`. Here, `int*` denotes a pointer to an integer, meaning that `arr` holds the memory address of an `int` rather than an integer value itself. The `new` operator dynamically allocates memory during program execution and returns a pointer to the allocated memory block. The value `n` represents the number of elements in the array and is typically obtained at runtime, for example from user input or a computation performed earlier in the program. Although the array size is determined during execu-

tion, the allocated elements are uninitialized, meaning no initial values are assigned to them. As a result, each element contains an indeterminate value—leftover data from previous memory usage. Reading or using these values before explicit initialization can lead to unpredictable results and undefined behavior. Furthermore, because the memory is allocated on the heap, it must be explicitly released using `delete[]` to prevent memory leaks.

b) Value-initialized dynamic array (zero-initialized)

`int* arr = new int[n]();` This form of dynamic array allocation allocates a contiguous block of memory on the heap and initializes all elements to zero at the time of allocation. By ensuring that each element starts with a well-defined value, it avoids the risks associated with uninitialized memory, such as unpredictable behavior or undefined results when elements are accessed before explicit assignment. This makes the approach safer than allocating an uninitialized array.

c) Uniform initialization (C++11 and later)

`int* arr = new int[n]{};`

The expression `new int[n]{};` is equivalent to `new int[n]();`, as both forms allocate a contiguous block of memory on the heap and value-initialize all elements, resulting in each element being set to zero. This guarantees that the array does not contain indeterminate values, making it safer to use than an uninitialized allocation. In contrast, the statement `int* arr = new int[n]();` explicitly uses parentheses to request value-initialization, whereas `new int[n]{};` uses uniform initialization syntax introduced in C++11. Both approaches produce the same behavior at runtime; however, the brace-based form is generally preferred in modern C++ because it is clearer, more consistent with uniform initialization, and helps avoid ambiguities associated with older initialization syntax.

d) Direct initialization with an initializer list (fixed size only)

`int* arr = new int[5]{10, 20, 30, 40, 50};`

This form of array initialization requires the array size to be a compile-time constant, meaning the number of elements must be known before program execution begins. The elements are initialized explicitly using a fixed initializer list, providing clear and predictable initial values. As a result, this approach cannot be used when the array size depends on a value that is only known at runtime.

e) Using auto (type deduction)

`auto arr = new int[n];`

The statement uses type deduction to infer the type of `arr` as `int*`. It dynamically allocates memory for `n` integers on the heap at runtime and stores the address of the first element in `arr`. While this form reduces verbosity, it can obscure the fact that `arr` is a pointer and that the allocated memory must be explicitly released using `delete[]`.

The following code demonstrates the use of dynamic arrays and their explicit deallocation at the end.

Code Example 1-5: Dynamic array example

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int n;
6
7     // Read array size at runtime
8     cout << "Enter number of elements: ";
9     cin >> n;
10
11    // Dynamically allocate array on the heap
12    int* arr = new int[n];
13
14    // Initialize array elements
15    for (int i = 0; i < n; ++i) {
16        arr[i] = (i + 1) * 10;
17    }
18
19    // Access and display array elements
20    for (int i = 0; i < n; ++i) {
21        cout << arr[i] << " ";
22    }
23    cout << endl;
24
25    // Release heap memory
26    delete[] arr;
27
28    return 0;
29 }

```

The program demonstrates the use of a dynamically allocated C-style array in C++. The program first reads an integer value at runtime to determine the number of elements required. It then allocates a contiguous block of memory on the heap using the `new[]` operator and stores the returned address in a pointer. The array elements are initialized and accessed using standard array indexing, which relies on pointer arithmetic internally. After the array contents are displayed, the program explicitly releases the heap-allocated memory using `delete[]`, highlighting the programmer's responsibility to manage memory when working with dynamic arrays.

Table 1.1 compares static arrays and dynamic C-style arrays by highlighting their key differences in terms of size determination, memory allocation, storage location, and lifetime management. It shows that while both array types store elements contiguously and do not support resizing, static arrays require their size to be known at compile time and are typically allocated on the stack or in static storage, whereas dynamic arrays allow the size to be determined at runtime and are allocated on the

heap, requiring explicit memory management by the programmer.

TABLE 1.1 *Comparison of Static and Dynamic C-Style Arrays*

Feature	Static Arrays	Dynamic Arrays (C-style)
Size determination	Compile time	Runtime
Ability to resize	Not supported	Not supported
Memory allocation time	Before program execution	During program execution
Memory location	Stack or static storage	Heap
Contiguous memory	Yes	Yes
Lifetime management	Automatic (scope-based)	Manual (requires deallocation)
Pointer required	No	Yes
Array-to-pointer decay	Yes	Yes
Suitable use case	Fixed-size data known in advance	Size known only at runtime
Example	<code>int a[5];</code>	<code>int* a = new int[n];</code>

1.2.3 *Memory Allocation of Static and Dynamic Arrays*

In this section, we examine how C-style arrays are allocated in memory by contrasting static arrays and dynamic arrays. Using concrete code examples, we illustrate how static arrays declared within a function are allocated on the stack, while dynamic arrays created using runtime allocation are stored on the heap. This comparison highlights the differences in memory location, lifetime, and management responsibilities, and provides a clear understanding of how array storage is handled at the memory level in C++.

Code Example 1-6: An array saved in a stack storage

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int x = 10;           // scalar variable
6     int y = 20;           // scalar variable
7     int arr[4] = {1, 2, 3, 4}; // static array
8
9     cout << "Address of x : " << &x << endl;
10    cout << "Address of y : " << &y << endl;
```

```

11
12     for (int i = 0; i < 4; i++) {
13         cout << "Address of arr[" << i << "]: "
14             << &arr[i] << endl;
15     }
16
17     return 0;
18 }

```

Output (addresses may vary)

```

Address of x   : 0x16fb8f188
Address of y   : 0x16fb8f184
Address of arr[0]: 0x16fb8f190
Address of arr[1]: 0x16fb8f194
Address of arr[2]: 0x16fb8f198
Address of arr[3]: 0x16fb8f19c

```

This program is written to illustrate how scalar variables and arrays are laid out in memory and how their addresses can be inspected.

The program begins execution in the `main()` function, where two integer variables, `x` and `y`, are declared and initialized with the values 10 and 20. These are scalar variables, meaning each variable stores a single value and occupies its own block of memory. Because they are declared inside `main()`, they are typically stored in the function's stack frame.

Next, the program declares an integer array named `arr` with a fixed size of four elements, initialized to `1, 2, 3, 4`. This is a static array, meaning its size is determined at compile time. All elements of the array are stored contiguously in memory, one immediately after another.

The program then prints the memory addresses of `x` and `y` using the address-of operator (`&`). This demonstrates that each scalar variable has a unique memory location. Printing these addresses helps visualize where individual variables reside in memory relative to each other.

After that, a for loop iterates through the array indices from 0 to 3. During each iteration, the program prints the address of the current array element using `&arr[i]`. Because arrays are stored contiguously, the output shows that the addresses of successive elements increase by a constant amount—specifically, by `sizeof(int)` bytes each time.

Finally, the program returns 0, indicating successful execution. Overall, this example demonstrates the difference between scalar variables and arrays, shows how array elements are laid out sequentially in memory, and reinforces the idea that array indexing in C++ is based on underlying pointer arithmetic.

The addresses in Figure 1.2 illustrate how local variables and a static array are laid out within the stack frame of the `main()` function. The scalar variables `x` and `y` are stored at adjacent memory locations, with their addresses differing by 4 bytes, which corresponds to the size of an `int`. The array elements `arr[0]` through `arr[3]` oc-

copy a contiguous block of memory, with each successive element located 4 bytes apart, reflecting the array's element size. The base address of the array appears at a higher memory location than the scalar variables, indicating that the compiler allocated space for **x** and **y** first and then reserved a contiguous region for the array within the same stack frame. While the exact ordering of variables within the stack is compiler-dependent, the contiguity and regular spacing of the array elements are guaranteed by the language.

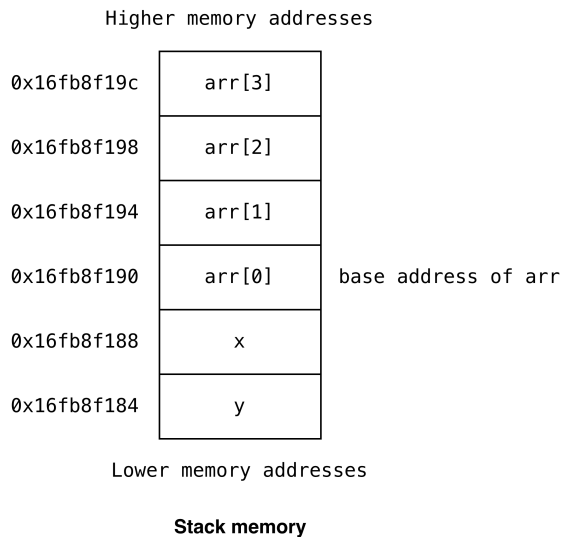


FIGURE 1.2 Memory allocation of static array

The example below follows the same structure as Code Example 1-6, with the key difference that the array is dynamically allocated rather than statically defined.

Code Example 1-7: An array contents saved in a heap storage

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int x = 10;           // scalar variable
6     int y = 20;           // scalar variable
7     int n = 4;
8
9     int* arr = new int[n]; // dynamic array (heap)

```

```

10
11 // initialize dynamic array
12 for (int i = 0; i < n; i++) {
13     arr[i] = i + 1;
14 }
15
16 cout << "Address of x : " << &x << endl;
17 cout << "Address of y : " << &y << endl;
18 cout << "Address of arr (pointer) : " << &arr << endl;
19 cout << "Value of arr (heap address) : " << arr << endl;
20
21 for (int i = 0; i < n; i++) {
22     cout << "Address of arr[" << i << "]: "
23         << &arr[i] << endl;
24 }
25
26 delete[] arr; // release heap memory
27 cout << arr << endl; // pointer still holds the address
28 arr = nullptr; // pointer points to 0
29 cout << arr << endl; // verify
30 return 0;
31 }

```

Output ((addresses may vary))

```

Address of x : 0x16ba0f1a8
Address of y : 0x16ba0f1a4
Address of arr (pointer) : 0x16ba0f198
Value of arr (heap address) : 0x104421a40
Address of arr[0]: 0x104421a40
Address of arr[1]: 0x104421a44
Address of arr[2]: 0x104421a48
Address of arr[3]: 0x104421a4c
0x104421a40
0x0

```

This program illustrates how dynamic (C-style) arrays are allocated, accessed, and deallocated in C++, and how they interact with stack and heap memory.

The variables **x**, **y**, and **n** are local variables declared inside **main()**, so they are stored in the function's stack frame. Their addresses are printed to show where they reside in stack memory. The statement **int* arr = new int[n];** dynamically allocates a contiguous block of memory on the heap large enough to store **n** integers. The variable **arr** itself is a pointer and is stored on the stack, but it holds the address of the first element of the heap-allocated array.

The for loop initializes the dynamic array elements using array indexing, which internally relies on pointer arithmetic. Printing **arr** displays the heap address of the first element, and printing the addresses of **arr[0]** through **arr[3]** shows that the elements are stored

contiguously in memory, with each address increasing by `sizeof(int)` bytes.

After the array is no longer needed, `delete[] arr;` releases the heap memory. Immediately after deletion, the pointer `arr` still contains the old heap address, even though the memory it refers to is no longer valid; this demonstrates a dangling pointer . Assigning `arr = nullptr;` explicitly clears the pointer, making it safe and indicating that it no longer points to valid memory. Printing `arr` after this assignment confirms that it now holds 0. Overall, the program demonstrates the separation of stack and heap storage, the contiguity of dynamic array elements, and the importance of proper memory management when using C-style dynamic arrays.

Dangling pointer

Figure 1.3 presents the output produced by Code Example 1-7, demonstrating the distinction between stack memory and heap memory in the context of a dynamic C-style array. The scalar variables `x` and `y` are local to `main()` and therefore reside in the stack frame, as indicated by their closely spaced addresses (`0x16ba0f1a8` and `0x16ba0f1a4`), which differ by 4 bytes, consistent with the size of an `int`. The variable `arr` itself is also a local variable, so the pointer `arr` is stored on the stack at address `0x16ba0f198`. However, the value held by this pointer (`0x104421a40`) represents the starting address of the dynamically allocated array on the heap. The addresses of `arr[0]` through `arr[3]` begin at this heap address and increase by 4 bytes each, confirming that the dynamic array elements are stored contiguously in heap memory. This output clearly demonstrates that, for dynamic arrays, only the pointer resides on the stack, while the actual array elements are allocated separately on the heap and accessed indirectly through the pointer.

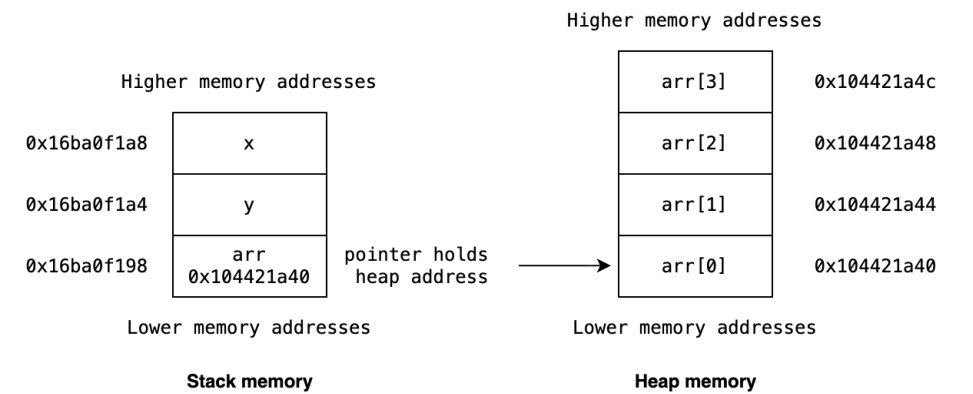


FIGURE 1.3 Memory allocation of dynamic array

i Layout Order of Local Variables in a Stack Frame

The C++ language does not specify the order in which local variables are laid

out in a stack frame. As a result, the memory addresses of variables may not reflect their order of declaration in the source code. This layout is determined by the compiler and may vary across platforms and optimization levels.

1.2.4 *Stack and Heap Storage Sizes*

Stack overflow

Stack storage is a limited region of memory used to store objects with automatic storage duration, such as local variables and function call frames. Its size is determined by the operating system and runtime environment and is typically restricted to a few megabytes on modern systems. Stack storage is managed automatically, with memory allocated when a function is entered and released when the function returns, resulting in very low allocation and deallocation overhead. Due to its limited capacity, stack storage is suitable for small, short-lived data objects but is not appropriate for large arrays or data structures, as excessive use can lead to stack overflow. On most modern systems, the operating system detects such an overflow and immediately terminates the program, often producing a runtime error such as a segmentation fault or a stack overflow exception.

On Windows systems, the default stack size is typically 1 MB, whereas on embedded systems the available stack space is often much smaller, ranging from a few kilobytes to a few hundred kilobytes. In contrast, Linux and macOS systems generally provide a larger default stack size of 8 MiB, which may be increased to 16 MiB or more depending on system policies. On Unix-like systems, the current stack size limit can be inspected using the `ulimit -s` command and adjusted as needed; for example, executing `ulimit -s 16384` sets the stack size limit to 16 MiB.

To estimate the maximum number of array elements that can be stored on the stack, consider an `int` data type that occupies 4 bytes. With a stack size of 8 MiB ($8 \times 1,048,576$ bytes), a theoretical maximum of 2,097,152 integer elements could be accommodated ($8 \times 1,048,576 \div 4$) in an array. However, this calculation represents an upper bound and is not achievable in practice.

The entire stack storage cannot be dedicated to user-defined data, as the stack is also used to maintain function call information, including return addresses, saved registers, function parameters, and additional bookkeeping data required to manage control flow. As a result, only a portion of the stack is safely available for local variables and arrays, further limiting the maximum usable array size.



Megabytes and Mebibytes: A Critical Distinction

The units MB (megabyte) and MiB (mebibyte) represent different quantities and should not be used interchangeably. MB is based on the decimal system and denotes 1,000,000 bytes (10^6 bytes), commonly used in storage device specifications and data transfer rates. In contrast, MiB is based on the binary system and denotes 1,048,576 bytes (2^{20} bytes), which aligns with how computer memory is organized and addressed. In memory-related discussions—such as stack

size, heap size, and array storage—MiB provides a more precise and technically accurate unit.

Heap storage is a larger and more flexible region of memory used for dynamic allocation during program execution. Memory allocated on the heap is obtained through explicit allocation mechanisms such as `new` and must either be released explicitly by the programmer or managed automatically through higher-level abstractions. Compared to stack storage, the heap can typically accommodate substantially larger data sizes—often ranging from hundreds of megabytes to several gigabytes—subject to available system memory and resource limits. Although heap allocation incurs greater overhead than stack allocation, it is indispensable for storing large data structures or objects whose size or lifetime cannot be determined at compile time.

The size of heap storage required by a program can be estimated by summing the sizes of all dynamically allocated objects while accounting for allocator overhead and peak simultaneous allocations. Unlike the stack, which has a fixed size limit, the heap grows dynamically and is primarily constrained by the process address space and the amount of physical and virtual memory available on the system.

1.3 C-Style Multidimensional Arrays

A multidimensional array is a data structure that generalizes the concept of a one-dimensional array by allowing elements to be accessed using more than one index. Each additional dimension introduces another level of organization, enabling the representation of more complex data relationships. In practice, this structure is often used to represent data arranged in rows and columns, such as tables or matrices, which aligns naturally with the two-dimensional array model commonly supported in C and C++.

A two-dimensional array can be thought of as a collection of arrays, where each element is accessed using a pair of indices: one specifying the row and the other specifying the column.

For example:

```
int matrix[3][4];
```

represents a static array structure with three rows and four columns.

A programmer should employ a 2D array when the data naturally requires two coordinates for meaningful representation. Common situations include:

Tabular data representation: Examples include storing examination scores organized by students and subjects, or sales data arranged by products and time periods.

Mathematical matrices: Operations such as matrix addition, multiplication, and transformations in linear algebra are naturally expressed using 2D arrays.

Grid-based problems: Many computational problems involve grids, such as board games (chess, tic-tac-toe), simulations, and cellular automata.

Image and spatial data: Images and maps are frequently represented as rows and columns of values, where each element corresponds to a pixel or spatial unit.

In general, a 2D array is appropriate when a single index is insufficient to clearly and logically describe the structure of the data.

1.3.1 *Memory Allocation*

Although a 2D array is conceptually two-dimensional, computer memory is inherently linear. In C and C++, two-dimensional arrays are stored in row-major order, meaning that all elements of the first row are stored contiguously in memory, followed by all elements of the second row, and so forth.

Consider the following declaration:

Code Example 1-8: Memory addresses allocation in multidimensional arrays

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int arr[2][3] = {
6         {1, 2, 3}, // first row
7         {4, 5, 6}  // second row
8     };
9
10    for (int i = 0; i < 2; i++) {
11        for (int j = 0; j < 3; j++) {
12            cout << "Address of arr[" << i << "][" << j << "] = "
13                << &arr[i][j] << endl;
14        }
15    }
16
17    return 0;
18 }
```

Output (addresses may vary)

```

Address of arr[0][0] = 0x16d6f3190
Address of arr[0][1] = 0x16d6f3194
Address of arr[0][2] = 0x16d6f3198
Address of arr[1][0] = 0x16d6f319c
Address of arr[1][1] = 0x16d6f31a0
Address of arr[1][2] = 0x16d6f31a4
```

The memory addresses of the elements in the two-dimensional array increase sequen-

tially, with each successive address differing by `sizeof(int)` bytes, which is typically four bytes on most systems. This indicates that the array elements are stored contiguously in memory. All elements of the first row are placed in adjacent memory locations before the elements of the second row are stored. This linear arrangement of rows in memory confirms that two-dimensional arrays in C and C++ are stored using row-major order.

1.3.2 Define and Access 2D Array Elements

In C++, a 2D array is defined using the following syntax:

```
data_type array_name[rows][columns];
```

For example:

```
int arr[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

The first index specifies the row, while the second index specifies the column.

The most common way to access all elements of a 2D array is by using nested loops:

```
for (int i = 0; i < 2; i++) { //ith row
    for (int j = 0; j < 3; j++) { //jth column
        cout << arr[i][j] << " ";
    }
    cout << endl;
}
```

The outer loop iterates over the rows of the array, while the inner loop iterates over the columns within each row. During each iteration, an individual element is accessed using the notation `arr[i][j]`, where `i` represents the current row index and `j` represents the current column index.

In many textbooks, two-dimensional arrays are typically accessed using nested for loops to reflect their logical row-and-column structure. However, because memory is inherently linear and the elements of a 2D array are stored contiguously, it is also possible to access all elements using a single for loop. The following example demonstrates how nested loops can be avoided while still correctly traversing the elements of a two-dimensional array.

Code Example 1-9: Access 2D array elements using pointer arithmetic with a single loop

```
1 #include <iostream>
2 using namespace std;
3
```

```

4 int main() {
5     int arr[2][3] = { {1, 2, 3}, {4, 5, 6} };
6
7     int* ptr = &arr[0][0];
8
9     for (int i = 0; i < 6; i++) {
10         cout << *(ptr + i) << " ";
11     }
12
13     return 0;
14 }

```

The program demonstrates how a two-dimensional array can be accessed as a contiguous block of memory using pointer arithmetic. The array `arr` is declared as a 2×3 integer array and initialized with values. The expression `&arr[0][0]` obtains the address of the first element of the array, which is stored in the pointer variable `ptr`. Because a 2D array in C++ is stored in row-major order, all elements are placed contiguously in memory. The for loop iterates six times, corresponding to the total number of elements in the array, and on each iteration the expression `*(ptr + i)` accesses the next element in memory. As a result, the program prints all elements of the 2D array sequentially using a single loop, illustrating that multidimensional arrays can be traversed linearly through pointer arithmetic.

Nested Loops vs. Pointer Arithmetic in 2D Traversal

Whether you traverse a two-dimensional array with nested for loops or with a single loop using pointer arithmetic, the amount of work done is essentially the same — each element of the array is visited once in both cases. The real difference lies in how easy the code is to read and understand.

Nested for loops are usually preferred because they clearly reflect the row-and-column layout of a two-dimensional array. This makes the code more intuitive, easier to follow, and simpler to maintain or debug, especially in academic or teaching environments where clarity is more important than clever tricks.

1.3.3 *Space Utilization in 2D Arrays*

In C++, a two-dimensional (2D) array uses a contiguous block of memory in which all rows and columns are fully populated with data elements. The total memory space required depends on the number of rows, the number of columns, and the size of the data type stored in the array. For example, an integer 2D array declared as `int arr[4][5]` contains $4 \times 5 = 20$ integer elements. Since an int typically occupies 4 bytes, the total memory allocated for this array is $20 \times 4 = 80$ bytes, stored contiguously in row-major order. This design works efficiently when every row has the same number of columns and all elements are meaningful. However, if the data is jagged — meaning different rows require different numbers of columns — a

C-style 2D array is not an ideal choice. In such cases, alternative data structures—such as arrays of pointers or standard library containers—are more efficient because they allow memory to be allocated only where needed, reducing unnecessary waste.

1.3.4 *Dynamic 2D Arrays*

As discussed earlier, a static 2D array allocates a single contiguous block of memory at compile time, with all rows laid out back-to-back in row-major order. In contrast, a dynamic 2D array determines its size at runtime and allocates memory on the heap rather than on the stack. Because this allocation happens at runtime, a dynamic 2D array does not have a fixed memory layout by default. If a dynamic array is created using an array of pointers, each row is allocated separately and only row-wise contiguity is guaranteed. However, if a single block of memory is allocated to store all elements, the dynamic 2D array can also be stored contiguously in heap, closely mirroring the layout of a static 2D array.

Array of pointers

The following example demonstrates the creation of a simple dynamic two-dimensional array using array of pointers

Code Example 1-10: Dynamic 2D array using array of pointers

```

1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int rows = 3;
6      int cols = 4;
7
8      // declare array of pointers (rows) on the heap
9      int** arr = new int*[rows];
10
11     // allocate columns for each row
12     for (int i = 0; i < rows; i++) {
13         arr[i] = new int[cols];
14     }
15
16     // initialize the array
17     for (int i = 0; i < rows; i++) {
18         for (int j = 0; j < cols; j++) {
19             arr[i][j] = i * cols + j;
20         }
21     }
22
23     // print the array
24     for (int i = 0; i < rows; i++) {
25         for (int j = 0; j < cols; j++) {
26             cout << arr[i][j] << " ";
27         }
28         cout << endl;

```

```

29     }
30
31     // deallocate memory
32     for (int i = 0; i < rows; i++) {
33         delete[] arr[i];
34     }
35     delete[] arr;
36
37     return 0;
38 }

```

Output (addresses may vary)

```

At row address starting from 0x10175db00----->0 1 2 3
At row address starting from 0x10175db10----->4 5 6 7
At row address starting from 0x10175d930----->8 9 10 11

```

This program demonstrates how a dynamic two-dimensional array can be created, used, and properly released in C++. The number of rows and columns is determined at runtime using the variables **rows** and **cols**.

Line 9: Allocates memory for a dynamic array of pointers and is the first step in creating a dynamic 2D array in C++. Here, **arr** is declared as a pointer to a pointer to int (**int****). This means **arr** will point to an array whose elements are themselves pointers to integers. The expression **new int*[rows]** allocates memory on the heap for rows number of elements, where each element is of type **int***. At this stage, no actual integer data is stored yet—only space for the row pointers is created.

Lines 12–13: This loop allocates memory for the columns of each row in the dynamic 2D array. Each element **arr[i]** is a pointer that is meant to represent one row. However, at that point, these pointers do not yet point to any actual data. Inside the loop, the statement allocates a contiguous block of memory for cols integers on the heap and assigns its address to **arr[i]**. As a result, **arr[i]** now points to the first element of a one-dimensional array that represents row **i** of the 2D array.

Once the memory is allocated, the program initializes the array using nested loops. Each element is assigned a value based on its row and column position, illustrating how elements are accessed using the familiar **arr[i][j]** notation. The array is then printed to the console, again using nested loops to traverse all rows and columns.

Finally, the program carefully deallocates the dynamically allocated memory. Each row is released individually using **delete[]**, followed by deleting the array of pointers itself. This step is essential to prevent memory leaks. Overall, the code illustrates the fundamental mechanics of dynamic 2D arrays including runtime allocation, access, and proper memory management.

The following example demonstrates the creation of a simple dynamic two-dimensional array using a single contiguous block and single loop (no row pointers).

Code Example 1-11: Dynamic 2D array using a single-loop

```

1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int rows = 3;
6      int cols = 4;
7
8      // allocate a single contiguous block
9      int* arr = new int[rows * cols];
10
11     // initialize the array
12     for (int i = 0; i < rows * cols; i++) {
13         arr[i] = i;
14     }
15
16     // print the array
17     for (int i = 0; i < rows * cols; i++) {
18         cout << arr[i] << " ";
19     }
20
21     // deallocate memory
22     delete[] arr;
23
24     return 0;
25 }

```

```
0 1 2 3 4 5 6 7 8 9 10 11
```

This program demonstrates the dynamic allocation and use of a one-dimensional array that stores data in a single contiguous block of heap memory. At runtime, memory is allocated to hold $rows \times cols$ integers, and the array is initialized using a single loop that assigns each element its linear index value. Although the variables *rows* and *cols* suggest a two-dimensional interpretation, the array is accessed and printed purely as a one-dimensional structure, with elements displayed sequentially in memory order. Finally, the program releases the dynamically allocated memory using `delete[]`, illustrating proper heap memory management.

The main limitation of using the single-loop, 1D-only approach is that it loses the logical two-dimensional structure of the data, even though the memory is intended to represent a 2D layout.

Because the array is accessed only through a single index, there is no explicit notion of rows and columns in the code. This makes it difficult to express operations that naturally depend on row-column relationships, such as accessing a specific row, iterating over a column, or applying matrix-style logic. Any time row or column information

is needed, it must be reconstructed manually using arithmetic (for example, computing $row = index/cols$ and $col = index \% cols$), which reduces readability and increases the likelihood of errors.

In short, single-loop works correctly for linear initialization or traversal, it obscures the programmer's intent when the data is conceptually two-dimensional, making the code harder to understand, maintain, and extend.

Code Example 1-11 is rewritten using nested for loops to explicitly reflect the logical two-dimensional structure of the data. Although the array is stored as a single contiguous block of memory, the nested loops reintroduce the familiar row-column access pattern, making the relationship between rows and columns clear and improving readability. This approach helps align the implementation with the conceptual model of a 2D array, which is especially beneficial for understanding, maintenance, and further extension of the code.

Code Example 1-12: Dynamic 2D array using a nested-loop

```

1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int rows = 3;
6      int cols = 4;
7
8      // allocate a single contiguous block
9      int* arr = new int[rows * cols];
10
11     // initialize the array
12     for (int i = 0; i < rows; i++) {
13         for (int j = 0; j < cols; j++) {
14             arr[i * cols + j] = i * cols + j;
15         }
16     }
17
18     // print the array
19     for (int i = 0; i < rows; i++) {
20         for (int j = 0; j < cols; j++) {
21             cout << arr[i * cols + j] << " ";
22         }
23         cout << endl;
24     }
25
26     // deallocate memory
27     delete[] arr;
28
29     return 0;
30 }
```

```
0 1 2 3
4 5 6 7
8 9 10 11
```

This program demonstrates how a two-dimensional data structure can be represented using a single dynamically allocated, contiguous block of memory. At runtime, the program allocates enough heap space to store $rows \times cols$ integers using a one-dimensional array. Although the memory is physically one-dimensional, the program logically treats it as a two-dimensional array by mapping each pair of indices (i, j) to a linear index using the formula $(i * cols + j)$, which corresponds to row-major order. The nested loops first initialize the elements using this mapping and then print them in a row-by-row format, giving the appearance of a 2D array. Finally, the dynamically allocated memory is released using `delete[]`, ensuring proper resource management.

The following illustrates how the array index `arr[i * cols + j]` is computed and progresses through memory.

```
i represent rows = 3
j represent cols = 4
When i = 0, j = 0 -> arr[i * cols + j] -> arr[0] = 0
When i = 0, j = 1 -> arr[i * cols + j] -> arr[1] = 1
When i = 0, j = 2 -> arr[i * cols + j] -> arr[2] = 2
When i = 0, j = 3 -> arr[i * cols + j] -> arr[3] = 3
...
When i = 2, j = 3 -> arr[i * cols + j] -> arr[11] = 11
```

Why Nested Loops Are Still Used?

Although a dynamic contiguous 2D array is stored as a one-dimensional block and can be initialized with a single loop, nested loops are commonly used to preserve the logical row-column structure and improve code readability and clarity.

Figure 1.4 illustrates the differences in memory allocation between static and dynamic two-dimensional arrays, highlighting both contiguous memory layouts and dynamic implementations with non-contiguous rows using an array of pointers.

On the left hand side of the figure shows a single, solid rectangular block of memory. All rows and columns are stored contiguously, with no gaps between rows. The green cells represent actual data elements, and the arrow indicates how memory progresses in a continuous manner. This illustrates that a static/dynamic 2D array occupies one uninterrupted block of memory, where rows are placed back-to-back in row-major order.

The right side of the figure illustrates the output produced by the code shown in

Example 1-10. Each row is shown in green but is located at a different position in memory, with gaps in between. The annotations indicate that each row begins at a distinct memory address—for example, Row 0, Row 1, and Row 2 all have different starting addresses. This layout illustrates how a dynamic 2D array, implemented using an array of pointers, allocates each row independently on the heap rather than as a single contiguous block on the stack.

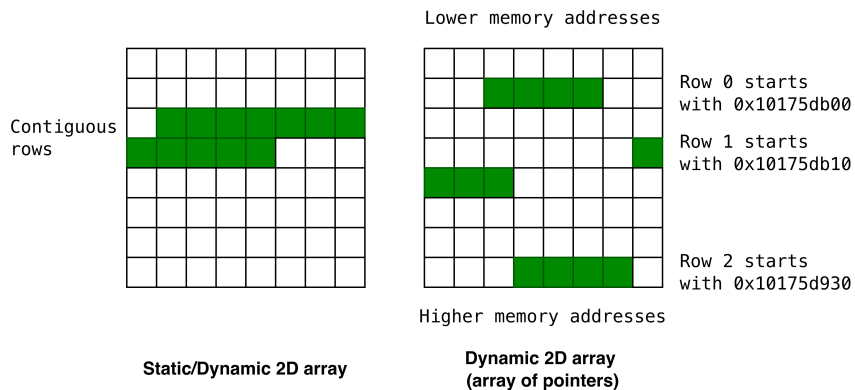


FIGURE 1.4 Memory map of static and dynamic 2D arrays

In real-world applications, data is often irregular, with different rows containing different numbers of columns. Such data is commonly referred to as jagged data meaning the rows do not all have the same length and therefore do not form a perfect rectangular structure. Using a rigid row-by-column memory block to load large jagged data sets can lead to inefficient memory usage, since space must be reserved for elements that may never be used. Although memory has become cheaper over time, this does not remove the responsibility of a programmer to write memory-efficient code. Thoughtful data structure choices still matter, especially when working with large or complex data sets. Arrays of pointers provide a flexible solution for handling jagged data by allowing each row to be sized according to its actual needs. The next section discusses how jagged data can be efficiently managed using arrays of pointers.

Assume you are working with a data set consisting of 5 observations (rows), where the number of parameters (columns) varies from one observation to another. To represent such jagged data efficiently, an array of pointers is used to store the starting address of each row, along with a separate one-dimensional array that records the length of each row. For example, `int parameters[5] = {4, 7, 9, 12, 3};` indicates that row 0 contains 4 parameters, row 1 contains 7 parameters, and so on with each value specifying the number of columns allocated for the corresponding row.

Code Example 1-13: Dynamic 2D array using array of pointers to handle jagged data

```
1 #include <iostream>
2 using namespace std;
```

```

3
4 int main() {
5     int rows = 5;
6     int* obs[rows]; // declare array of 5 pointers in stack
7     int par[5] = {4,7,9,12,3};
8
9     // allocate columns for each row
10    for (int i = 0; i < rows; i++) {
11        obs[i] = new int[par[i]]; // making space available per row
12    }
13
14    int value = 1;
15    for (int i = 0; i < rows; i++) {
16        cout << "obs[" << i << "]: ";
17        for (int j = 0; j < par[i]; j++) {
18            obs[i][j] = value++; // example user-defined data
19            cout << obs[i][j] << " ";
20        }
21        cout << endl;
22    }
23
24    // deallocate memory
25    for (int i = 0; i < rows; i++) {
26        delete[] obs[i];
27    }
28
29    return 0;
30 }

```

```

obs[0]: 1 2 3 4
obs[1]: 5 6 7 8 9 10 11
obs[2]: 12 13 14 15 16 17 18 19 20
obs[3]: 21 22 23 24 25 26 27 28 29 30 31 32
obs[4]: 33 34 35

```

Line 5: Defines an integer variable named **rows** and initializes it to 5. This value represents the number of rows (or observations) in the data set and is stored on the stack.

Line 6: Declares an array of **rows** pointers to integers. This creates space to five pointer variables on the stack, where each pointer is intended to reference a dynamically allocated one-dimensional array representing a row of data.

Line 7: Declares and initializes a fixed-size integer array on the stack. Each element of this array specifies the number of columns associated with a particular row, thereby defining the jagged structure of the data.

Lines 10–11: Dynamically allocates a contiguous block of memory on the heap with a size determined by **par[i]**. The starting address of this block is stored in **obs[i]**, effectively assigning memory for that specific row.

Line 14: Loop begins the process of traversing and initializing each row of the jagged array.

Line 15: Prints a label indicating the current row index, helping distinguish the output corresponding to each row.

Line 16: Iterates over the columns of the current row. The loop limit `par[i]` ensures that the program accesses only the valid elements allocated for that row.

Line 17: Computes and stores a value at the current row and column. This expression generates distinct, sequential values for each row, making the jagged structure clearly visible in the output.

Lines 24–25: The for loop begins the cleanup phase, iterating once per row to deallocate the heap memory previously allocated for row `i`. Using `delete[]` is essential because the memory was allocated with `new[]`.

Why jagged data is typically handled on the heap? Jagged data refers solely to the logical structure of the data—specifically, the fact that different rows may contain different numbers of elements—and is independent of how memory is allocated or where the data is stored. In practice, jagged data is typically stored on the heap because its irregular structure often requires runtime-determined sizes, flexible lifetimes, and independent row allocation. In many real-world scenarios, such data is populated dynamically from external sources such as files, databases, network APIs, or user input, where row lengths cannot be known in advance. While stack-based representations are possible when all dimensions are known at compile time, heap allocation provides the scalability, flexibility, and adaptability required for most real-world applications.

Multidimensional Arrays and Linear Memory

In C++, all array storage is fundamentally one-dimensional at the memory level. A true two-dimensional array is laid out as a single contiguous linear block in row-major order, while an array-of-pointers representation consists of multiple independent one-dimensional arrays, each referenced by a pointer. Regardless of the storage model, techniques such as nested loops, a single loop with index mapping, or pointer-based access are simply different ways of traversing linear memory, not different kinds of memory themselves.

1.4 Functions and C-Style Arrays

In C++, arrays interact with functions in a manner that differs fundamentally from scalar variables. When a scalar value such as an `int` or `double` is passed to a function, it is commonly passed by value, resulting in the function receiving a copy of the variable. However, C++ also allows scalar values to be passed by reference, enabling the function to access and modify the original variable. Arrays, however, follow a different rule. When an array is passed to a function, it decays into a pointer to its first element. As a result, the function does not receive a copy of the array but rather an address that refers to the original memory. Any modification performed on the

array elements inside the function therefore directly affects the original array.

This behavior applies uniformly to static arrays (allocated at compile time, usually on the stack) and dynamic arrays (allocated at runtime on the heap). From the perspective of a function, there is no semantic difference between the two; in both cases, the function operates on memory through pointers. Consequently, the programmer must explicitly supply any necessary size or structural information to the function.

1.4.1 Static One-Dimensional Arrays and Functions

A static one-dimensional array has a fixed size determined at compile time. When such an array is passed to a function, only the address of its first element is transferred. The size information is not preserved, which is why functions that process arrays must receive the array size as a separate parameter.

Code Example 1-14: Static 1D arrays and functions

```

1 #include <iostream>
2 using namespace std;
3
4 void printArray(int arr[], int size) {
5     for (int i = 0; i < size; i++) {
6         cout << arr[i] << " ";
7     }
8 }
9
10 int main() {
11     int data[5] = {1, 2, 3, 4, 5};
12     printArray(data, 5);
13 }
```

In this example, the parameter `int arr[]` is syntactic sugar for `int* arr`. The function operates directly on the original array stored in the caller's scope. This illustrates that static arrays are not copied when passed to functions, and their contents can be accessed or modified through the received pointer.

1.4.2 Dynamic One-Dimensional Arrays and Functions

Dynamic one-dimensional arrays are allocated at runtime using operators such as `new`. Despite the difference in allocation mechanism, their interaction with functions is identical to that of static arrays. The function still receives a pointer to the first element, and size information must be passed explicitly.

Code Example 1-15: Dynamic 1D arrays and functions

```

1 #include <iostream>
2 using namespace std;
```

```

3
4 void printArray(int* arr, int size) {
5     for (int i = 0; i < size; i++) {
6         cout << arr[i] << " ";
7     }
8 }
9
10 int main() {
11     int size = 5;
12     int* data = new int[size]{1, 2, 3, 4, 5};
13
14     printArray(data, size);
15
16     delete[] data;
17 }

```

In this version, the array is allocated dynamically on the heap using the `new[]` operator. The function parameter is declared as `int*`, which reflects the fact that a dynamically allocated one-dimensional array is accessed through a pointer to its first element. From the function's perspective, there is no difference between a static and a dynamic one-dimensional array, as both are passed as a pointer along with their size. The responsibility for managing the lifetime of the dynamically allocated memory lies with the programmer, which is why the array is explicitly deallocated using `delete[]` after use.

1.4.3 Static Two-Dimensional Arrays and Functions

A static two-dimensional array in C++ is stored in contiguous memory using row-major order, meaning that all elements of the first row are stored sequentially, followed immediately by all elements of the second row, and so on. When such an array is passed to a function, the array name decays into a pointer to its first row. In Code Example 1-16, a function parameter declared as `int mat[][COL]`, where `COL` is a compile-time constant representing the number of columns, the parameter internally is adjusted by the compiler to `int (*mat)[COL]`. This indicates that `mat` is a pointer to an array of `COL` integers. Each increment of this pointer advances it by exactly one full row in memory, that is, by `sizeof(int[COL])` bytes, allowing the compiler to correctly compute memory offsets between successive rows.

To compute the address of an element `mat[i][j]`, the compiler applies the following calculation:

$$\text{element_address}(i, j) = \text{base_address} + i \times (\text{number_of_columns}) + j \quad (1.1)$$

The number of columns is essential because it defines the memory stride between successive rows. Without knowing the column size, the compiler cannot determine where one row ends and the next begins. Consequently, when declaring a two-dimensional array as a function parameter in C++, the compiler requires the column

size to be specified, while the row size may be left unspecified.

In contrast, the number of rows plays no role in this address calculation. The compiler does not need to know how many rows exist in order to compute the address of any individual element. The row count is instead a logical property used by the program at runtime to control iteration and prevent out-of-bounds access, which is why it is typically passed as a separate function parameter.

The following example demonstrates the concept discussed above.

Code Example 1-16: Static 2D arrays and functions

```

1  #include <iostream>
2  using namespace std;
3
4  const int COL = 4;
5
6  void printMatrix(int mat[][COL], int rows) {
7      for (int i = 0; i < rows; i++) {
8          for (int j = 0; j < COL; j++) {
9              cout << mat[i][j] << " ";
10         }
11         cout << endl;
12     }
13 }
14
15 int main() {
16     int matrix[3][COL] = {
17         {1, 2, 3, 4},
18         {5, 6, 7, 8},
19         {9, 10, 11, 12}
20     };
21
22     printMatrix(matrix, 3);
23 }
```

This program demonstrates how a static two-dimensional array is passed to and processed by a function in C++. The constant `COL` defines the number of columns and is used in the function parameter to ensure that the column size is known at compile time. When the array `matrix` is passed to `printMatrix()`, the array name decays into a pointer to its first row, and the function parameter is treated as a pointer to an array of `COL` integers. This allows the compiler to correctly compute memory offsets between rows using row-major order.

The function `printMatrix()` receives the number of rows as a separate argument, which is used to control the outer loop and ensure safe traversal of the array. Inside the function, nested loops iterate over each row and column, accessing elements using standard two-dimensional indexing and printing them to the output. In `main()`, a static two-dimensional array with three rows and `COL` columns is declared and initialized, and the function is called with the array and the row count. This example highlights that, for static

two-dimensional arrays, the column dimension must be specified in the function signature, while the row dimension can be supplied at runtime.

1.4.4 *Dynamic Two-Dimensional Arrays and Functions*

As we already discussed that a dynamic two-dimensional array may be implemented as a single contiguous block of memory allocated at runtime. Although this structure is conceptually two-dimensional, it is physically represented as a one-dimensional array whose elements are arranged in row-major order. Access to individual elements is achieved through explicit row-column arithmetic, using the same expression as mentioned in Equation 1.1. Because the array is represented by a simple pointer (for example, `int*`), no dimensional information is embedded in the type. As a result, any function that operates on such a structure must be explicitly provided with both the number of rows and the number of columns. The column count is required to compute correct memory offsets between rows, while the row count is needed to control iteration and ensure safe traversal of the allocated memory.

Code Example 1-17: Dynamic 2D arrays and functions

```

1 void printMatrix(int* mat, int rows, int cols) {
2     for (int i = 0; i < rows; i++) {
3         for (int j = 0; j < cols; j++) {
4             cout << mat[i * cols + j] << " ";
5         }
6         cout << endl;
7     }
8 }
9
10 int main() {
11     int rows = 3, cols = 4;
12     int* matrix = new int[rows * cols];
13     printMatrix(matrix, rows, cols);
14     delete[] matrix;
15 }
```

This program demonstrates how a dynamic two-dimensional array can be represented as a single contiguous block of memory and processed using a function. In `main()`, memory is allocated at runtime using `new` to create a one-dimensional array large enough to hold all elements of the conceptual two-dimensional structure. Although the array is logically treated as having `rows` rows and `cols` columns, it is physically stored as a linear block of memory, and the pointer `matrix` refers to the first element of this block.

The function `printMatrix()` receives a pointer to this contiguous memory along with the number of rows and columns. Because the pointer itself contains no information about the array's dimensions, both `rows` and `cols` must be supplied explicitly. Inside the function, nested loops traverse the array, and each two-dimensional index (`i, j`) is mapped to a one-dimensional offset using the expression `i * cols + j`. This row-column arithmetic

reflects row-major order and allows correct access to each element within the linear memory block.

After the function call, the dynamically allocated memory is released using `delete[]`, which is necessary to prevent memory leaks. This example highlights the key difference between static and dynamic two-dimensional arrays: in dynamic contiguous arrays, neither dimension is embedded in the type, so all structural information must be provided at runtime.

Functions and Arrays

The interaction between functions and arrays in C++ reveals an essential principle: functions do not receive arrays as complete objects, but rather operate on memory through pointers. Whether an array is static or dynamic, one-dimensional or multi-dimensional, the function's correctness depends on accurate structural information supplied by the programmer. Understanding this behavior is critical for writing safe, efficient, and maintainable C++ programs and serves as a conceptual bridge to pointers, dynamic memory management, and modern container abstractions.

1.4.5 Jagged Arrays and Functions

The following example shows how jagged arrays allocated on the heap and implemented as arrays of pointers are passed to a function.

Code Example 1-18: Jagged arrays and functions

```

1  #include <iostream>
2  using namespace std;
3
4  void printJagged(int** mat, int rows, int cols[]) {
5      for (int i = 0; i < rows; i++) {
6          for (int j = 0; j < cols[i]; j++) {
7              cout << mat[i][j] << " ";
8          }
9          cout << endl;
10     }
11 }
12
13 int main() {
14     int rows = 3;
15
16     // Number of columns in each row (jagged structure)
17     int cols[3] = {2, 4, 3};
18
19     // Allocate array of pointers (rows)
20     int** matrix = new int*[rows];

```

```

21
22     // Allocate each row with a different length
23     for (int i = 0; i < rows; i++) {
24         matrix[i] = new int[cols[i]];
25     }
26
27     // Initialize the jagged array
28     int value = 1;
29     for (int i = 0; i < rows; i++) {
30         for (int j = 0; j < cols[i]; j++) {
31             matrix[i][j] = value++;
32         }
33     }
34
35     // Pass jagged data to the function
36     printJagged(matrix, rows, cols);
37
38     // Deallocate memory
39     for (int i = 0; i < rows; i++) {
40         delete[] matrix[i];
41     }
42     delete[] matrix;
43
44     return 0;
45 }

```

In this example, jagged data is created in `main()` using an array of pointers, where each pointer refers to a dynamically allocated one-dimensional array representing a row. The array `cols` stores the number of elements in each row and provides the structural metadata required for safe traversal. The function `printJagged()` receives a pointer to pointers (`int**`), the number of rows, and the per-row column counts, allowing it to correctly iterate over rows of varying lengths. This approach illustrates how jagged arrays are commonly implemented when row sizes are determined at runtime.

1.4.6 Limitations of C-Style Arrays

C-style arrays support a limited but fundamental set of operations defined by the C++ language. Because C-style arrays are not objects, they do not provide member functions; instead, all supported operations are expressed through language syntax and built-in operators. For example, the total size of an array can be obtained using the `sizeof` operator, as shown in Code Example 1-2. The value returned by `sizeof` represents the total size of the array in bytes and can be used to determine the array length (i.e., the number of elements) by dividing it by the size of an individual element.

 **`sizeof()` is an Operator, Not a Function**

The `sizeof(array)` operation may appear similar to a function call due to its use of parentheses; however, it is not a function. The parentheses are optional, and the expression can also be written as `sizeof array;`, which yields the same result.

The following operations are widely supported by C++ standard library containers and by arrays in many other programming languages, but are not available for C-style arrays. It is therefore important to be aware of the restrictions associated with their use.

Copy assignment of entire arrays

Copy assignment of entire arrays refers to the operation in which all elements of one array are assigned to another array of the same type and size using a single assignment statement. In many programming languages and in C++ standard library containers, this operation is supported and results in an element-by-element copy of the array contents. However, in C-style arrays, copy assignment of entire arrays is not permitted. C-style arrays are not objects and therefore do not define an assignment operator; as a result, an expression such as `b = a` is invalid when `a` and `b` are C-style arrays. Instead, copying the contents of a C-style array must be performed explicitly, either through element-wise assignment or by using appropriate library utilities.

```
int a[3] = {1, 2, 3};
int b[3];
b = a;    // compile-time error
```

Bounds-checked access

Bounds-checked access ensures that an index used to access an array element falls within the valid range. Unlike C-style arrays, which perform no bounds checking and can lead to undefined behavior when accessed out of range.

Dynamic resizing

Dynamic resizing refers to the ability of a data structure to change its size at runtime as elements are added or removed. C-style arrays do not support dynamic resizing; their size is fixed at compile time (or at allocation time for dynamically allocated arrays).

Member function calls

C-style arrays do not support member function calls because they are not objects. They are a built-in language construct and therefore provide no member functions like `.size()`, `.at()`, or `.resize()`. Any operations on C-style arrays must be performed using language operators (e.g., indexing, `sizeof`) or external library functions.

Array-to-pointer decay

When a C-style array is passed to a function, it undergoes array-to-pointer decay, meaning that the array is converted into a pointer to its first element. As a consequence, the function receives only the memory address of the array data, and the original array size information is no longer available. Operations such as `sizeof(array)`

inside the function therefore yield the size of the pointer rather than the size of the array. Since the array length is not preserved, the function cannot determine how many elements the array contains on its own. Consequently, the array length must be provided explicitly as a separate argument when passing a C-style array to a function.

```
void print(const int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << endl;
    }
}
```

1.5 From C-Style Arrays to Modern C++ Arrays

C-style arrays form the foundation of array handling in C++; however, they exhibit several inherent limitations, as discussed in the preceding section. As programs grow in size and complexity, these limitations can lead to bugs, unsafe memory access, and code that is difficult to maintain. Modern C++ addresses many of these shortcomings by providing safer and more expressive array abstractions.

This section introduces modern C++ array abstractions, highlights how they differ from traditional C-style arrays, and demonstrates how they improve safety, readability, and reliability without compromising performance.

C++ provides the `std::array` container as a safer and more modern alternative to fixed-size C-style arrays. In C++, a container is an object used to store a collection of elements and to provide a consistent interface for accessing and working with those elements. Containers are a central part of the Standard Template Library (STL). The STL is a built-in part of C++ that provides ready-made tools for working with data. It includes common data structures, such as dynamic arrays and lists, along with functions to perform operations like sorting and searching. STL helps programmers write simpler, cleaner, and more efficient code by using well-tested components instead of creating everything from scratch.

`std::array` is a fixed-size container whose size is specified at the time of creation and remains constant for its entire lifetime. Because this size must be known at compile time and is encoded directly into the type (`std::array<T, N>`) where `T` is the “element type” and `N` is the “number of elements”. `std::array` does not support runtime size allocation and cannot grow or shrink after it is created, much like a C-style array. Its elements are stored contiguously in memory, which provides efficient access and predictable performance. In terms of memory utilization, `std::array` stores its elements directly within the object itself, so where the data resides depends entirely on where the object is created. For example, when declared as a local variable, both the container and its elements are stored on the stack. However, when a `std::array` is dynamically allocated, the `std::array` object itself is placed on the heap. Because

Container

Standard Template
Library (STL)

`std::array` stores its elements directly within the object, the elements reside in the same memory block as the object rather than in a separate heap allocation. The pointer used to access the object may still be stored on the stack, depending on where it is declared. Regardless of its storage location, the fixed size and contiguous layout of `std::array` remain unchanged.

However, unlike C-style arrays, `std::array` is a first-class object and it behaves like a regular C++ object, not a special or limited language construct like a C-style array.

Concretely, a first-class object can:

- Be assigned and copied using normal assignment semantics
- Be passed to functions by value, reference, or const reference
- Be returned from functions
- Have member functions such as `.size()` or `.at()`
- Participate naturally in templates and STL algorithms

`std::array` can be initialized using either copy-list initialization or uniform (direct-list) initialization.

```
#include <array>
std::array<int, 5> arr = {1, 2, 3, 4, 5}; // copy-list initialization
```

The first line includes the header, which provides the definition of `std::array` from the C++ Standard Library. The second line declares a variable named `arr` as an object of type `std::array<int, 5>`. This means `arr` is a fixed-size container that can store exactly five elements of type `int`. The value 5 is a compile-time constant and represents the size of the array. The initializer list `{1, 2, 3, 4, 5}` assigns values to the five elements in order. Internally, these elements are stored contiguously in memory as part of the `std::array` object. When declared as a local variable, this object is typically allocated on the stack, similar to a C-style array. Unlike a C-style array, however, `std::array` is a proper object that knows its own size and integrates directly with STL algorithms.

`std::array` is written with the prefix `std::` because it belongs to the C++ Standard Library namespace, named `std`. Writing `std::array` explicitly tells the compiler that you are referring to the array type provided by the C++ Standard Library, not some other array that might exist in your program or in another library.

`std::array` can also be initialized as follows:

```
std::array<int, 5> arr{1, 2, 3, 4, 5}; // direct-list initialization
```

Both copy-list initialization and uniform (direct-list) initialization initialize a `std::array` in the same way. Since `std::array` is an aggregate type, both forms perform aggregate initialization: the elements are initialized directly in place, with no constructors, temporaries, or copy operations involved. The difference is primarily

stylistic. For clarity and readability, programmers should choose one initialization form and use it consistently throughout their code.

Take a look at another example.

```
#include <array>
auto p = new std::array<int, 5> {1,2,3,4,5};
delete p;
p = nullptr;
```

The above code dynamically allocates a fixed-size `std::array<int, 5>` on the heap using direct initialization and initializes its elements with the values 1, 2, 3, 4, and 5. The `new` operator reserves heap memory and constructs the `std::array` object there, while `auto` deduces `p` to be of type `std::array<int, 5>*`. If `p` is a local variable, the pointer itself resides on the stack and simply holds the address of the heap-allocated object. Because `std::array` stores its elements directly within the object, the five integers are part of the same heap allocation and are not stored in a separate memory block. Since the object is dynamically allocated, it must be explicitly released using `delete p` when it is no longer needed to avoid a memory leak, and assigning `p = nullptr` afterward clears the pointer to prevent accidental use of a dangling pointer.

1.5.1 Member Functions of `std::array`

When working with data in C++, we don't just store values—we also use functions that are designed to work specifically with that data. These functions are called member functions. At this stage, you can think of a member function as a built-in function that comes with a specific data type and is used to access or manipulate the data stored in a variable of that type. Member functions are called using the variable name and a dot (`.`), and they provide a safe and convenient way to interact with the data without needing to know how it is stored internally. The following member functions can be used with `std::array`.

1) Element Access Functions: Element access functions provide controlled ways to read or modify individual elements stored in an array.

TABLE 1.2 *Element Access Member Functions of `std::array`*

Function	Description	Example
<code>operator[]</code>	Direct access (no bounds checking)	<code>arr[2];</code>
<code>at()</code>	Bounds-checked access (throws <code>std::out_of_range</code>)	<code>arr.at(2);</code>
<code>front()</code>	First element	<code>arr.front();</code>
<code>back()</code>	Last element	<code>arr.back();</code>

Function	Description	Example
<code>data()</code>	Pointer to underlying raw array	<code>int* p = arr.data();</code>

The following example illustrates how various functions can be used to access elements within an array.

Code Example 1-19: Example of element access functions (addresses may vary)

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     array<int,10> arr = {1,3,5,7,9,2,4,6,8,10};
6     int* p = arr.data(); // integer pointer points to the base address
7     cout << arr[5] << endl; // 2
8     cout << arr[12] << endl; // -1882333868 (some garbage)
9     cout << arr.at(5) << endl; // 2
10    cout << arr.at(12) << endl; // throws std::out_of_range:
11    cout << arr.front() << endl; // 1
12    cout << arr.back() << endl; // 10
13    cout << p[4] << endl; // pointer points to element 4
14    cout << &arr << endl; // points to the base address of arr (0x16babb188)
15    cout << &arr[0] << endl; // points to the base address of arr
16    // (0x16babb188)
17    cout << p << endl; // points to the base address of arr (0x16babb188)
18 }
```

Line 5: Declares a `std::array` named `arr` that can hold exactly ten integers. This is a fixed-size container whose size is known at compile time, and all elements are stored contiguously in memory.

Line 6: Obtains a raw pointer to the first element of the array. The `data()` member function returns the base address of the contiguous block of integers managed by the `std::array`. As a result, `p` now points to the same memory location as `&arr[0]`, enabling pointer-style indexing.

Line 8: Attempts to access an element outside the valid index range of the array (0 to 9). This results in undefined behavior. The program may print a seemingly random or “garbage” value, may crash, or may appear to work, but none of these outcomes are guaranteed. This illustrates the danger of unchecked array access.

Line 10: Attempts to access the element at index 12 of the array using the `at()` member function. Since `arr` is a `std::array` of size 10, the valid index range is from 0 to 9. The index 12 is therefore out of bounds. Unlike the subscript operator `[]`, the `at()` function performs runtime bounds checking. When it detects that the requested index is invalid, it throws a `std::out_of_range` exception. If this exception is not caught using a try-catch block, the program terminates, making this line a clear demonstration of how `at()` provides safe access by preventing silent undefined behavior.

Line 13: The line uses pointer-style indexing. Since `p` points to the first element of the array, `p[4]` is equivalent to `*(p + 4)` and accesses the element at index 4 of the array. The value printed is 9, showing that pointer arithmetic works the same way as array indexing when the data is contiguous.

Line 16: Prints the value stored in the pointer `p`, which is a memory address. Since `p` was initialized using `arr.data()`, it holds the address of the first element of the `std::array`. As a result, this line outputs the base address of the array's underlying contiguous storage. In other words, it confirms that `p` points to the first element of the array, and pointer arithmetic on `p` can be used to access the array elements sequentially.

2) Capacity Functions: Capacity functions report information about how many elements a container can hold. They do not access elements and do not modify the container.

TABLE 1.3 *Capacity Member Functions of `std::array`*

Function	Description	Example
<code>size()</code>	Number of elements	<code>arr.size();</code>
<code>max_size()</code>	Same as <code>size()</code> (fixed-size container)	<code>arr.max_size();</code>
<code>empty()</code>	Always false unless size is 0	<code>arr.empty();</code>

The following example demonstrates how different functions can be used to determine the size of an array.

Code Example 1-20: Example of capacity access functions

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     array<int,10> arr = {1,3,5,7,9,2,4,6,8,10};
6     array<int,0> arr2; // uninitialized array
7     cout << arr.size() << endl; // 10
8     cout << arr.max_size() << endl; // 10
9     cout << arr.empty() << endl; // 0 -> False
10    cout << arr2.empty() << endl; // 1 -> True
11 }
```

Line 6: Declares another `std::array` of 10 integers without providing initial values. This creates the container with space for 10 integers; whether the integers are initialized depends on how the object is created, but the key point is that the array still has 10 elements.

Line 7: `arr.size()` returns the current number of elements in the container. For `std::array<int, 10>`, the container always contains exactly 10 elements—no more, no less—so `size()` returns 10.

Line 8: `arr.max_size()` returns the maximum number of elements the container is allowed to hold. Since a `std::array` cannot grow or shrink, its maximum possible size is also fixed at 10. Therefore, `max_size()` also returns 10.

Line 9: Prints whether `arr` is empty. `empty()` returns true only if the array has 0 elements. Because `arr` has 10 elements, it returns false, which prints as 0.

Line 10: Checks whether `arr2` has zero elements. Since `arr2` is `array<int, 0>`, it truly has no elements, so `empty()` returns true, and `cout` prints that as 1.

i size() vs max_size() With `std::array`, the result of `size()` and `max_size()` are always identical. With dynamic containers like `std::vector`, they are usually different, because the container can grow or shrink over time.

3) Iterator Functions: Iterator functions in `std::array` provide a uniform way to traverse (move through) the elements of the array without needing index variables. They allow `std::array` to work seamlessly with STL algorithms such as `std::sort`, `std::find`, and range-based for loops. The table below shows the iterator functions of `std::array`.

TABLE 1.4 *Iterator Member Functions of `std::array`*

Iterator Function	What It Returns	What It Does	Typical Use
<code>begin()</code>	Iterator	Points to the first element	Start traversal
<code>end()</code>	Iterator	Points one past the last element	Stop condition
<code>cbegin()</code>	Const iterator	Read-only iterator to first element	Prevent modification
<code>cend()</code>	Const iterator	Read-only iterator past last element	Safe traversal
<code>rbegin()</code>	Reverse iterator	Points to last element	Reverse traversal
<code>rend()</code>	Reverse iterator	Points before first element	Reverse stop
<code>crbegin()</code>	Const reverse iterator	Read-only reverse iterator	Const reverse traversal
<code>crend()</code>	Const reverse iterator	Read-only reverse end	Safe reverse traversal

The following is a minimal example of every iterator-related function provided by `std::array`, using one common array so the differences are easy to see.

Code Example 1-21: Example of iterator functions

```

1 #include <array>
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     array<int, 5> arr = {10, 20, 30, 40, 50};
7
8     // Iterate forward (modifiable elements)
9     for (auto it = arr.begin(); it != arr.end(); it++) {
10         *it += 1; // allowed
11         cout << *it << " "; // 11 21 31 41 51
12     }
13     cout << endl;
14
15     // Iterate forward (read-only elements)
16     for (auto it = arr.cbegin(); it != arr.cend(); it++) {
17         // *it += 1; // not allowed (const)
18         cout << *it << " ";
19     }
20     cout << endl;
21
22     // Iterate backward (modifiable elements)
23     for (auto it = arr.rbegin(); it != arr.rend(); it++) {
24         *it -= 1; // allowed
25         cout << *it << " "; // 50 40 30 20 10
26     }
27     cout << endl;
28
29     // Iterate backward (read-only elements)
30     for (auto it = arr.crbegin(); it != arr.crend(); it++) {
31         // *it -= 1; // not allowed (const)
32         cout << *it << " ";
33     }
34     cout << endl;
35 }

```

4) Modifier Functions: Modifier functions are member functions that change the contents of a container. In the case of `std::array`, modifier functions operate only on the existing elements because the size of the array is fixed at compile time. They do not add or remove elements, nor do they change the capacity of the container. Instead, they provide controlled ways to update values or exchange contents. For example, the `fill()` function assigns the same value to all elements in the array, while `swap()` exchanges the contents of two arrays of the same type and size. Thus, modifier functions allow efficient and safe modification of an array's data without altering its structure.

TABLE 1.5 *Modifier Member Functions of `std::array`*

Modifier Function	Description	Example
<code>fill()</code>	Assigns the same value to all elements of the array	<code>arr.fill(0);</code>
<code>swap()</code>	Exchanges the contents of two arrays of the same type and size	<code>arr1.swap(arr2);</code>

The following example illustrates the use of modifier functions.

```
std::array<int, 4> arr = {1, 2, 3, 4};
std::array<int, 4> arr2 = {10, 20, 30, 40};

arr.fill(5);      // arr becomes {5, 5, 5, 5}
arr.swap(arr2);   // arr ↔ arr2 contents exchanged
```

5) Comparison Operators: Comparison operators allow `std::array` objects to behave like built-in values by supporting direct equality and ordering comparisons. These comparisons are performed lexicographically between two `std::array` objects of the same type and size, meaning elements are compared sequentially from the first position onward. The comparison stops as soon as a difference is encountered, in the same way words are compared in a dictionary.

When two `std::array` objects are compared lexicographically:

1. The first elements of both arrays are compared.
2. If they are different, the result of that comparison determines the outcome.
3. If they are equal, the comparison moves to the next element.
4. This continues until a difference is found or all elements are equal.

For example,

```
std::array<int, 3> a = {1, 3, 2};
std::array<int, 3> b = {1, 2, 4};
cout << (a < b) << endl;    // 0 -> False -> comparison stops at the first
↪ differing element (3 > 2)
```

The following example demonstrates how strings are compared. When comparing `std::array<std::string>` objects, the comparison is performed lexicographically at two levels. First, `std::array` compares its elements from left to right. When an element comparison is required, the corresponding `std::string` objects are compared using their own lexicographical rules. A `std::string` comparison proceeds character by character, and each character is evaluated based on its underlying character encoding value (typically ASCII or an ASCII-compatible encoding). The comparison

stops as soon as a difference is encountered, and the result is determined at that point. This is why, in the example, “cat” is considered greater than “car”: the characters ‘t’ and ‘r’ are the first to differ, and ‘t’ has a higher encoding value than ‘r’.

The comparison operators `<`, `>`, `<=`, and `>=` all use lexicographical comparison; only the direction of comparison changes.

```
std::array<string, 1> a = {"cat"};
std::array<string, 1> b = {"car"};
cout << (a < b) << endl;    // 0 -> False -> comparison stops at the first
    ↪ differing element (t > r)
```

1.5.2 `std::array` and Functions

`std::array` works very naturally with functions because it is a first-class object that knows its size at compile time. Unlike C-style arrays, it does not decay to a pointer, which makes function interfaces safer, clearer, and more expressive.

The first example is passing `std::array` by value, where the entire array is copied into the function’s parameter. This approach is simple and safe because the function works on its own copy and cannot modify the original array, avoiding side effects and aliasing issues. It is reasonable for small arrays, where the cost of copying is minimal and clarity is more important than performance. However, copying all elements can become inefficient for large arrays or arrays of complex types, especially when the function only needs read-only access. For this reason, passing `std::array` by value is best used for small data sets or when isolation is intentional.

Code Example 1-22: Passing `std::arrays` by value

```
1 #include <array>
2 #include <iostream>
3
4 void print(std::array<int, 4> arr) {
5     for (int x : arr)
6         std::cout << x << " ";
7 }
8
9 int main() {
10     std::array<int, 4> data = {1, 2, 3, 4};
11     print(data);
12 }
```

This program shows how a `std::array` can be passed by value to a function. In `main()`, a `std::array<int, 4>` named `data` is created and initialized with four integers. When `print(data)` is called, the entire array is copied into the function parameter `arr` because the function takes its argument by value. Inside the print function, a range-based for loop iterates over this local copy and prints each element. Since the function operates on a copy, any changes made to `arr` would not affect the original data in `main()`. This example highlights

that `std::array` behaves like a regular object: it preserves its size, can be passed safely to functions, and does not decay to a pointer as C-style arrays do.

The second example is passing `std::array` by reference, which avoids copying the array and allows the function to work directly with the original data. This approach is efficient and well suited for large arrays or performance-critical code, as it eliminates copy overhead while still preserving the array's size information. It also allows the function to modify the original array when a non-const reference is used. However, because the function operates on the same object, changes are visible to the caller, which may introduce unintended side effects if not carefully designed. For this reason, passing by `const` reference is commonly preferred when only read-only access is needed, balancing efficiency with safety.

Code Example 1-23: Passing `std::arrays` by non-const reference

```
void print(std::array<int, 4>& arr) {
    for (int& x : arr)
        std::cout << x << " ";
}
```

The third example is passing `std::array` by `const` reference, where the function receives a reference to the original array without creating a copy and is explicitly prevented from modifying it. This allows efficient access to the array's elements while avoiding the overhead of copying all values. The `const` qualifier enforces read-only behavior, making the function safer and easier to reason about by eliminating unintended side effects. Inside the function, the range-based loop simply reads and prints each element. This approach is widely considered best practice when a function only needs to observe or display the contents of a `std::array`.

Code Example 1-24: Passing `std::arrays` by `const` reference

```
void print(const std::array<int, 4>& arr) {
    for (int x : arr)
        std::cout << x << " ";
}
```

In the fourth example shown in Code Example 1-25, when a function returns a `std::array` by value, it may appear that the array is first created inside the function and then copied into the variable in the caller `mai()`. This mental model was common in older C++ practices. However, modern C++ compilers apply Return Value Optimization (RVO) to avoid this overhead. RVO is a compiler optimization technique that constructs the returned object directly in the caller's memory, thereby eliminating unnecessary copy or move operations when an object is returned from a function.

Return Value
Optimization

With RVO, the compiler can determine at compile time that the object being returned

Guaranteed Copy
Elision

will immediately be used to initialize another object in the caller. Instead of allocating memory for a temporary array inside the function `createArray()`, the compiler reuses the caller's storage and constructs the array directly in the memory location of the receiving variable. As a result, the function does not create a local array and then return it; it builds the array directly in the caller's space. However, it is important to note that RVO was not mandatory in pre-C++17 standards—compilers were allowed, but not required, to apply this optimization.

Since C++17, this optimization is no longer optional in many common cases—it is guaranteed by the language standard (known as guaranteed copy elision). This means that no temporary object exists at runtime, no copy constructor is called, and no move occurs. From a runtime perspective, only one `std::array` is ever constructed, even though the source code appears to involve a return.

Another way to think about it is this: the caller allocates space for the result first, and the function simply fills that space. This is why returning `std::array` (and other value types) is both safe and efficient in modern C++. It gives you clean, expressive code without the performance penalties that older C++ programmers were trained to fear.

Code Example 1-25: `std::array` returning from a function

```

1 #include <array>
2 #include <iostream>
3
4 std::array<int, 3> createArray() {
5     return {10, 20, 30};
6 }
7
8 int main() {
9     std::array<int, 3> data = createArray();
10
11     for (int x : data)
12         std::cout << x << " ";
13 }
```

Line 4: Declares a function named `createArray` that returns a `std::array` of 3 integers. The return type is `std::array<int, 3>`.

Line 5: Creates a `std::array<int, 3>` initialized with 10, 20, 30 and returns it (in modern C++, it's typically constructed directly into the caller's array due to copy elision/RVO).

Line 9: Declares a `std::array<int, 3>` named `data` and initializes it with the array returned by `createArray()`.

Lines 11-12: A range-based for loop iterates through `data` from the first element to the last, and for each iteration, the current element is copied into `x` and printed to the output followed by a space.

i Difference between RVO and Guaranteed Copy Elision

Return Value Optimization (RVO) is a compiler optimization technique that allows an object returned by value to be constructed directly in the caller's memory, thereby avoiding unnecessary copy or move operations. Before C++17, RVO was optional—most compilers applied it, but they were not required to do so, and copy or move constructors still needed to be accessible in case the optimization was not performed.

Guaranteed copy elision, introduced in C++17, is a language rule rather than an optimization. In specific return scenarios, the C++ standard mandates that no temporary object is created at all, and the returned object is constructed directly in the caller's storage. In these cases, copy and move constructors are not invoked and do not even need to exist.

1.5.3 Design Constraints of `std::array`

Before delving into the design constraints of `std::array`, we first outline the design goals and motivations behind its introduction.

The design thinking behind `std::array` was to modernize the traditional C-style array without changing its fundamental performance or memory model, while fixing its most painful usability problems. C-style arrays are extremely efficient, but they are not real objects: they don't know their own size, decay to pointers when passed to functions, cannot be returned by value safely, and don't integrate well with the Standard Template Library. `std::array` was introduced to preserve the same contiguous, zero-overhead storage as a built-in array, while wrapping it in a proper C++ object that behaves predictably and safely in generic code.

A key design goal of `std::array` was zero-cost abstraction. A `std::array<T, N>` where 'T' is a data type and 'N' is the number of elements, is essentially a thin wrapper around `T[N]`. The elements are stored directly inside the array object, and the container performs no dynamic memory allocation. The container does not use hidden pointers, and its size is fixed at compile time rather than determined at runtime. Because the size `N` is known at compile time, the compiler can perform the same optimizations it would for a raw array. This makes `std::array` suitable for performance-critical code, embedded systems, and low-level components where dynamic allocation is undesirable or forbidden.

Another major motivation was STL compatibility and type safety. By making the array a first-class object, `std::array` can be copied, assigned, passed to and returned from functions by value, and used seamlessly with standard algorithms. The container exposes a small set of member functions that support clear and consistent interaction with the array. At the same time, the array size is part of the type itself (`std::array<T, N>`), allowing for stronger compile-time checks and helping prevent accidental size mismatches common with raw arrays.

Finally, `std::array` reflects the broader C++ design philosophy of making the safe

and expressive option the default when there is no performance penalty. It is not meant to replace `std::vector`, which handles dynamic resizing, but to replace C-style arrays in cases where the size is fixed and known at compile time. In that sense, `std::array` fills an important design gap: it provides the efficiency of C arrays, the expressiveness of modern C++, and predictable behavior that works cleanly with templates, algorithms, and generic programming.

The following design constraints reflect intentional trade-offs and may act as limitations when working with data that has dynamic, unpredictable, or variable structure. More concretely:

Runtime-sized data: Data whose size is only known at runtime, such as user input, file contents, network packets, or database query results. For example, reading ‘n’ (number of values) from input cannot be modeled with `std::array` because “n” is not known at compile time.

Growing or shrinking collections: Data that changes size over time, such as a list of active users, a log that keeps appending entries, or a container that supports insertion and removal. Since `std::array` cannot grow or shrink, it is not suitable for these use cases.

Highly flexible or irregular structures: Data with varying lengths or shapes, such as jagged arrays, variable-length records, or dynamically generated matrices. These structures typically require runtime allocation and flexible sizing.

Large data sets that risk stack exhaustion: Very large fixed-size data blocks that may not safely fit on the stack. While `std::array` can be heap-allocated, its fixed size still makes it less convenient than containers designed for large, dynamically managed data.

In short, `std::array` works best for small, fixed-size, well-defined data known at compile time (e.g., RGB values, fixed sensor readings, lookup tables). It becomes a limiting factor when the data’s size or structure must adapt at runtime.

Based on these design constraints, there is a clear need for a container that can manage data whose size and structure may change at runtime. This motivation leads naturally to the `std::vector` container, which is the topic of our next discussion.

1.6 The `std::vector` Container

In the previous sections, we examined fixed-size arrays—both C-style arrays and `std::array` and saw how they provide contiguous storage, predictable performance, and strong compile-time guarantees. These characteristics make fixed-size arrays an excellent choice when the number of elements is known in advance and remains constant throughout the program’s execution. However, many real-world problems do not fit this model. Data often arrives incrementally, grows or shrinks over time, or depends on values that are only known at runtime. This is where `std::vector` becomes essential.

`std::vector` is a dynamically sized sequence container that stores its elements contiguously in memory while allowing its size to change during execution. As a result, `std::vector` is one of the most widely used and foundational containers in the C++ Standard Library.

1.6.1 Memory Management

A `std::vector` stores its elements in one contiguous block of memory, just like a C-style array. The key difference is where and how that memory is obtained:

- A `std::vector` is a small control object that stores a pointer to data, size, and capacity; the vector object itself resides wherever it is declared (stack or heap),
- The elements are stored in a dynamically allocated block on the heap.

The following cases are discussed in the context of memory management. Memory behavior is examined using the GNU Debugger (gdb) on a Linux platform with the GCC compiler and the `libstdc++` standard library. Implementation-specific details are presented in the Appendix~8.1; however, the key observations and outcomes are discussed here.

Case 1: Vector declared on the stack with initialized members

A `std::vector` object typically maintains three internal pointers. The actual name of these pointers varies based on different implementations, however the purpose is same. These pointers are `start`, `finish` and `end_of_storage` pointers. The size and capacity of the vector are computed by taking differences between these pointers.

$$size = finish_pointer - start_pointer$$

$$capacity = end_of_storage_pointer - _start_pointer$$

where,

`start` points to the beginning of the contiguous memory block that stores the vector's elements.

`finish` points one position past the last constructed element, marking the logical end of the vector.

`end_of_storage` points one position past the end of the allocated memory block, marking the vector's total capacity.

The following code example demonstrates the concepts described above.

```

1 void f() {
2     std::vector<int> v = {1, 2, 3, 4}; // declares and initializes the vector
    ↪ with four integers
3     return 0;
4 }
```

```

stack:
v (0x7fffffff470)
+-----+
| 0x55555556aeb0 | --> heap memory (start pointer)
| 0x55555556aec0 | (finish pointer)
| 0x55555556aec0 | (end of storage pointer)
+-----+

heap memory:
+-----+-----+
| 0x55555556aeb0 | 1 | <-- start pointer
| 0x55555556aeb4 | 2 |
| 0x55555556aeb8 | 3 |
| 0x55555556aebc | 4 |
| 0x55555556aec0 | --- | <-- one-past-the-end (finish pointer)
+-----+-----+

```

The following are the key observations:

- 1) A closer inspection of a vector's internal memory allocation reveals that the one-past-the-end position does not represent a valid element in heap memory. It may not contain meaningful data, may not correspond to usable memory, and exists only as a pointer value used to mark the logical boundary of the element range. Here, finish pointer marks this end boundary using the begin-inclusive, end-exclusive range model adopted by the C++ standard library.
- 2) In this example, the size and capacity of the vector are the same (4); however, this is not always the case. The size represents the number of elements currently stored in the vector, while the capacity represents the amount of storage that has been allocated. When a vector is initialized using an initializer list, common standard library implementations typically allocate exactly enough storage to hold the elements, resulting in no unused capacity. The following calculates the size of the vector.

$$\begin{aligned}
 \text{vector_size} &= \text{finish_pointer} - \text{start_pointer} \\
 &= (0x\ldots\text{AEC0} - 0x\ldots\text{AEB0}) / \text{sizeof(int)} \\
 &= 0x10 \text{ bytes} / 4 \text{ bytes} \\
 &= 16 / 4 \\
 &= 4
 \end{aligned}$$

- 4) The vector **v** is allocated on the stack, but it internally holds pointers that point to its elements stored in heap memory.

Case 2: Vector declared on the stack with initialized members and reallocated memory space

This example demonstrates vector memory reallocation. The vector is first initialized with a fixed number of elements, after which additional elements are appended to show how its internal storage changes as the vector grows. Initially, the vector has a size and capacity of 4.

```

1 void f() {
2     std::vector<int> v = {1, 2, 3, 4}; // declares and initializes the vector
    ↪ with four integers
3     v.push_back(5);
4     v.push_back(6);
5     return 0;
6 }

```

The memory allocation for the first four elements is consistent with the layout discussed earlier; however, the specific memory addresses may vary.

```

stack:
v (0x7fffffff470)
+-----+
| 0x55555556ceb0 | --> heap memory (start pointer)
| 0x55555556cec0 | (finish pointer)
| 0x55555556cec0 | (end of storage pointer)
+-----+

heap memory:
+-----+-----+
| 0x55555556ceb0 | 1 | <-- start pointer
| 0x55555556ceb4 | 2 |
| 0x55555556ceb8 | 3 |
| 0x55555556cebc | 4 |
| 0x55555556cec0 | --- | <-- one-past-the-end (finish pointer)
+-----+-----+

```

When elements such as `v.push_back(5);` and `v.push_back(6);` are appended to a vector that has already reached its capacity, the standard library performs a reallocation by allocating a new, larger block of heap memory. The existing elements are copied or moved to the new location, usually with extra capacity reserved to accommodate future insertions. After the transfer completes, the previously allocated heap memory is released back to the allocator immediately and does not persist until the program ends. The resulting memory layout is illustrated below.

```

stack:
v (0x7fffffff470)
+-----+
| 0x55555556ced0 | --> heap memory (start pointer)
| 0x55555556cee8 | (finish pointer)
| 0x55555556cef0 | (end of storage pointer)

```

```

+-----+
heap memory (new allocation)
+-----+-----+
| 0x55555556ced0 | 1 | <-- start pointer
| 0x55555556ced4 | 2 |
| 0x55555556ced8 | 3 |
| 0x55555556cedc | 4 |
| 0x55555556cee0 | 5 |
| 0x55555556cee4 | 6 |
| 0x55555556cee8 | 0 | <-- available space (finish pointer)
| 0x55555556ceec | 0 | <-- available space
| 0x55555556cef0 | --- | <-- one-past-the-end (end of storage
    ↪ pointer)
+-----+-----+

```

The initial heap memory allocation starts at address ced0. When elements are added to a vector that has already reached its capacity, a new block of heap memory is allocated beginning at address ced0. As part of this reallocation, the allocator expands the capacity from four to eight elements. The vector's size and capacity can be calculated as follows:

$$\begin{aligned}
 \text{vector_size} &= \text{finish_pointer} - \text{start_pointer} \\
 &= (0x\ldots CEE8 - 0x\ldots CED0) / \text{sizeof}(\text{int}) \\
 &= 0x18 \text{ bytes} / 4 \text{ bytes} \\
 &= 24 / 4 \\
 &= 6
 \end{aligned}$$

$$\begin{aligned}
 \text{vector_capacity} &= \text{end_of_storage_pointer} - \text{start_pointer} \\
 &= (0x\ldots CEF0 - 0x\ldots CED0) / \text{sizeof}(\text{int}) \\
 &= 0x20 \text{ bytes} / 4 \text{ bytes} \\
 &= 32 / 4 \\
 &= 8
 \end{aligned}$$

How vector stores strings

2

Classes & Inheritance

3

Virtual Functions & Abstract Classes

Virtual Functions	58
Virtual Destructors	62
Abstract Class	68
Examples - A Polymorphic Design	71
Concept Check	75

Imagine you have a remote control with a button labeled “Power.”

- If the remote is connected to a TV, pressing Power button turns the TV on.
- If the same remote is connected to a Blu-ray player, pressing Power button starts the player and display the name of the movie.

A button represents a function, and pressing it (calling the function) triggers behavior specific to the device. **Polymorphism** describes the idea of one function producing many behaviors, while virtual functions are the C++ feature that makes this behavior work.

 The term polymorphism originates from the Greek poly (many) and morphē (form), meaning “many forms,” as defined in the Oxford English Dictionary. [1]

3.1 Virtual Functions

A virtual function is a member function of a class that is declared using the keyword **virtual** and is meant to be overridden in a derived class so that the function call is resolved at runtime, not at compile time.

Without virtual, C++ uses **compile-time binding** - in other words, the compiler decides which function to call before the program runs, based solely on the information available at compile time, such as the type of the variable or pointer. For example:

Code Example 3-1: Behavior of classes without virtual functions

```

1  #include <iostream>
2  using namespace std;
3
4  class Animal {
5  public:
6      void speak() const {
7          cout << "Animal makes a sound" << endl;
8      }
9  };
10
11 class Dog : public Animal {
12 public:
13     void speak() const {
14         cout << "Dog barks" << endl;
15     }
16 };
17
18 int main() {
19     Animal* a = new Dog();
20     a->speak();
21     delete a;

```

```

22     return 0;
23 }

```

Lines 6 & 13: The class **Dog** defines a member function **speak()** with the same name and signature as the function in the base class **Animal**.

Line 19: A base-class pointer of type **Animal** is used to point to a **Dog** object.

Line 20: Function call **a->speak()** is resolved at compile time. The version of **a->speak()** defined in the **Animal** class is invoked, and the output is “Animal makes a sound”. The function calls are determined by the pointer type rather than the actual object type.

Virtual functions determine which function implementation is called at runtime.

Code Example 3-2: Behavior of class member functions using virtual functions

```

1 class Animal {
2 public:
3     virtual void speak() const {
4         cout << "Animal makes a sound" << endl;
5     }
6 };
7 class Dog : public Animal {
8 public:
9     void speak() const override {
10         cout << "Dog barks" << endl;
11     }
12 };

```

The compiler decides to run **speak()** method in the derived class and prints “Dog barks”. Adding **virtual** changed the program from compile-time (static) binding to run-time (dynamic) binding. In short: adding **virtual** tells the compiler to decide which function to call at runtime based on the object, not the pointer.

Line 9: **override** is not required, but it helps the compiler make sure you are actually overriding a virtual function and not making a mistake. Override **can only be used if a virtual function exists in the base class, or get a compile-error**.

Runtime polymorphism

The above code uses runtime polymorphism - allows the same function call to behave differently for different objects. Polymorphism is often described as “one interface, many forms”.

i Some textbooks loosely use the term polymorphism for compile-time mechanisms, while others reserve polymorphism strictly for runtime behavior. So both of these are acceptable depending on context.

The following example illustrates a compilation error that occurs when a base-class pointer attempts to access a member function that is not declared in the base class.

Code Example 3-3: Base-class pointer accessing a non-declared member function (compilation error)

```

1 class Animal {
2 public:
3     virtual void speak() const {
4         cout << "Animal makes a sound" << endl;
5     }
6     void print() const {
7         cout << "I am an animal" << endl;
8     }
9 };
10 class Dog : public Animal {
11 public:
12     void speak() const override {
13         cout << "Dog barks" << endl;
14     }
15     void info() const {
16         cout << "I am a Dog" << endl;
17     }
18 };
19
20 int main() {
21     Animal* a = new Dog();
22     a->speak();    // valid
23     a->print();    // valid
24     a->info();     // error: no member named 'info' in 'Animal'
25 }

```

Although the pointer `a` refers to a `Dog` object, it is declared as an `Animal*`. At compile time, C++ allows access only to the members defined in `Animal`. Because `info()` exists only in `Dog` class, the compiler does not allow the call `a->info()`, resulting in a compilation error. If a method is declared in `Animal` and marked as `virtual`, the version defined in derived-class `Dog` will be executed at runtime. Since a method `info()` exists only in the `Dog` class and is not declared in `Animal`, attempting to call it through pointer `a` will result in a compilation error. By contrast, the call `a->speak()` is valid because `speak()` is declared in the base class and marked as `virtual`. This allows the function call to be resolved at runtime, so the `Dog` version of `speak()` is executed. However, virtual dispatch does not extend the set of accessible members; it only determines which implementation of an already-declared function is called. In summary, a base-class pointer can only access members that are part of the base-class interface, and `info()` is inaccessible through `Animal*` because it is not declared in `Animal`.

Lets extend Code Example 3-2 and add another derived class and update the main function.

Code Example 3-4: Behavior of virtual member functions across derived classes

```

1 class Cat : public Animal {
2 public:
3     void speak() const override {
4         cout << "Cat meows" << endl;
5     }
6 };
7
8 int main() {
9     Animal* a[2];
10    a[0] = new Dog();
11    a[1] = new Cat();
12
13    for (int i = 0; i < 2; i++) {
14        a[i]->speak();
15    }
16
17    for (int i = 0; i < 2; i++) {
18        delete a[i];
19    }
20
21    return 0;
22 }

```

Line 3: Declares a member function `speak`. `const` means the function does not modify the object. `override` tells the compiler this function overrides a virtual function in `Animal`.

Line 9: Declares an array of two pointers to `Animal`. Each element can point to any object derived from `Animal`.

Lines 10 & 11: Dynamically creates a `Dog` and `Cat` objects on the heap and stores its address in `a[0]` and `a[1]` as an `Animal*` respectively. Both pointers are of type `Animal` but they point to different derived objects. This sets up runtime polymorphism.

Line 13: Loops over both elements in the array

Line 14: Calls `speak()` through a base-class pointer. Because `speak()` is virtual in `Animal`, the function call is resolved at runtime. `Dog::speak()` is called for `a[0]`; `Cat::speak()` is called for `a[1]`

Line 18: Frees the memory allocated with `new`. Calls the derived class destructor first, then the base class destructor. This works correctly only if `Animal` has a virtual destructor.

Since we're talking about runtime polymorphism, it's worth also touching on static polymorphism so the difference between the two becomes clearer.

Static polymorphism

Static polymorphism is a form of polymorphism in which the function call is resolved at compile time, allowing the same function name to perform different operations based on different parameter types. Static polymorphism is commonly achieved through function overloading, operator overloading, and templates. Because the decision is made before the program runs, static polymorphism incurs no runtime

overhead and generally results in faster execution.

Code Example 3-5: Static polymorphism using function overloading

```

1 class Math {
2 public:
3     int add(int a, int b) const {
4         return a + b;
5     }
6
7     double add(double a, double b) const {
8         return a + b;
9     }
10 };
11
12 int main() {
13     Math m;
14     cout << m.add(2, 3) << endl;    // calls int version
15     cout << m.add(2.5, 3.5) << endl; // calls double version
16 }

```

The program demonstrates static polymorphism through function overloading. The **Math** class defines two member functions named **add()**, one that accepts integer parameters and another that accepts double-precision parameters. Although both functions share the same name, they differ in their parameter types, allowing the compiler to distinguish between them. In the **main** function, a **Math** object is created and each version of **add()** is invoked with arguments that match its parameter types. The compiler determines which function to call at compile time, based on the types of the arguments passed, illustrating how function overloading enables multiple behaviors under a single function name without using virtual functions.

- i** Function overloading cannot be achieved using different return types alone; the parameter list must differ. Function overloading selects a function at compile time based on different parameter lists, whereas function overriding replaces a base-class virtual function in a derived class and is resolved at run time.

Mention about the vtable

3.2 Virtual Destructors

Destructors are necessary in C++ because they provide a guaranteed and orderly way to release resources that an object acquires during its lifetime before the object is destroyed. While memory allocated on the stack is reclaimed automatically, many objects manage non-memory resources such as dynamically allocated heap memory,

open files, network connections, database handles, or system locks, which are not automatically released by the operating system at the correct time. A destructor ensures that these resources are properly cleaned up when an object goes out of scope or is deleted, preventing memory leaks, resource exhaustion, and undefined behavior.

In C++, destructors play a central role in the Resource Acquisition Is Initialization (RAII) principle by binding resource management to object lifetime. This design ensures that resources are released automatically when objects go out of scope, making programs safer, more predictable, and easier to maintain.

Without destructors, programmers would have to rely on manual cleanup, which is error-prone and difficult to guarantee, especially in large systems and in the presence of exceptions.

Virtual destructors in C++ are a language mechanism that ensure correct and complete object destruction in polymorphic class hierarchies. When a derived-class object is deleted through a base-class pointer, the absence of a virtual destructor in the base class results in static binding, causing only the base-class destructor to execute and leaving the derived portion of the object undisposed. Declaring the base-class destructor as virtual enables dynamic binding, ensuring that the destructor associated with the object's dynamic (runtime) type is invoked first, followed by base-class destructors in the proper sequence. This behavior is essential for the safe release of resources allocated by derived classes and for preventing memory leaks and undefined behavior. Consequently, any base class intended for polymorphic use should declare a virtual destructor to maintain correctness, robustness, and resource safety in object-oriented C++ programs.

The following two Code Examples 3-6 and 3-7 illustrates the behavior of destructors with and without a virtual destructor when runtime polymorphism is used.

Code Example 3-6: Without virtual destructor (memory leak)

```

1  #include <iostream>
2  using namespace std;
3
4  class Base {
5  public:
6      ~Base() {
7          cout << "Base destructor called" << endl;
8      }
9  };
10
11 class Derived : public Base {
12 private:
13     int* data;
14 public:
15     Derived() {
16         data = new int[100];    // allocate resource
17         cout << "Derived constructor allocated memory" << endl;

```

```

18     }
19
20     ~Derived() {
21         delete[] data;           // cleanup
22         cout << "Derived destructor freed memory" << endl;
23     }
24 };
25
26 int main() {
27     Base* obj = new Derived();
28     delete obj;    // deleting through base pointer
29     return 0;
30 }

```

Line 6: Declares the destructor of **Base**. This destructor is NOT virtual.

Line 7: Prints a message when a Base object is destroyed.

Line 16: Dynamically allocates an array of 100 integers on the heap. Derived now owns this memory.

Line 20: Destructor of Derived. Responsible for cleaning up resources owned by Derived.

Line 27: Creates a Derived object on the heap. Stores its address in a Base-class pointer. This enables polymorphism.

Line 28: Deletes the object through a base-class pointer. Since Base's destructor is not virtual, only Base class destructor is invoked. The derived-class destructor is not called, so the dynamically allocated data is never released, resulting in a memory leak and undefined behavior.

Summary: If a base class is used to delete derived objects, its destructor must be virtual, and the derived class must define its own destructor to properly release resources.

Code Example 3-7: With virtual destructor (no memory leak)

```

1  #include <iostream>
2  using namespace std;
3
4  class Base {
5  public:
6      virtual ~Base() {
7          cout << "Base destructor called" << endl;
8      }
9  };
10
11 class Derived : public Base {
12 private:
13     int* data;
14 public:
15     Derived() {
16         data = new int[100];    // allocate resource
17         cout << "Derived constructor allocated memory" << endl;

```



```

18     }
19
20     ~Derived() {
21         delete[] data;           // cleanup
22         cout << "Derived destructor freed memory" << endl;
23     }
24 };
25
26 int main() {
27     Base* obj = new Derived();
28     delete obj;    // correct polymorphic destruction
29     return 0;
30 }

```

When a derived-class object is deleted through a base-class pointer `delete obj` in Line 28, C++ first executes the destructor of the derived class, allowing all resources allocated at the derived level to be properly released. Subsequently, the base-class destructor `~Derived()` is invoked, completing the destruction process in the correct hierarchical order. This sequence guarantees comprehensive resource deallocation and preserves program correctness, thereby preventing memory leaks and ensuring robust and predictable behavior in polymorphic C++ programs.

i In this example, setting a `obj = nullptr` after deletion is not required if the pointer goes out of scope immediately, but it is recommended when the pointer remains in use to prevent dangling pointer errors.

3.2.1 Using Smart Pointers to Manage Memory

Thus far, memory has been managed manually. As codebases scale and multiple classes are introduced, this approach becomes increasingly difficult. Smart pointers ease this burden by providing automatic and efficient memory management.

Smart pointers provide automatic lifetime management by releasing ownership and invoking an appropriate deleter when they go out of scope, thereby reducing errors such as forgotten delete operations and double deletions. However, in polymorphic hierarchies, correct destruction of a derived object through a base-class pointer depends on the base class declaring a virtual destructor; the smart pointer merely triggers deletion, while the language's virtual dispatch mechanism determines whether the derived destructor is executed.

Code Example 3-7 is extended to incorporate smart pointers in order to illustrate how memory management is handled automatically.

Code Example 3-8: Memory management using smart pointers

```

1 #include <memory> // for smart pointers
2 using namespace std;
3
4 class Base {
5 public:
6     virtual ~Base() {
7         cout << "Base destructor called" << endl;
8     }
9 };
10
11 class Derived : public Base {
12 private:
13     int* data;
14 public:
15     Derived() {
16         data = new int[100]; // allocate resource
17         cout << "Derived constructor allocated memory" << endl;
18     }
19
20     ~Derived() {
21         delete[] data; // cleanup
22         cout << "Derived destructor freed memory" << endl;
23     }
24 };
25
26 int main() {
27     std::unique_ptr<Base> obj = std::make_unique<Derived>();
28     return 0;
29 }

```

The use of `std::unique_ptr` in this program expresses exclusive ownership of the dynamically allocated object, ensuring that only a single pointer is responsible for managing its lifetime. The object is **automatically deleted when the smart pointer goes out of scope**, thereby eliminating the need for explicit delete operations and reducing the risk of memory management errors such as leaks or double deletion. Furthermore, correct object destruction is guaranteed because the base class declares a virtual destructor, which enables dynamic binding and ensures that the derived-class destructor is invoked prior to the base-class destructor.

3.2.2 Compiler-generated Destructors

A compiler-generated (default) destructor is automatically provided by the C++ compiler when a class does not explicitly define one. This destructor is responsible for destroying the object's subobjects, including its non-static data members and base-class subobjects, by invoking their respective destructors in the correct order. However, the compiler-generated destructor performs only **shallow cleanup** and does not re-

lease dynamically allocated resources managed through raw pointers, as the compiler cannot infer ownership semantics. Consequently, classes that acquire resources manually using mechanisms such as `new`, file handles, or mutexes must define a custom destructor to ensure proper resource release. In contrast, when a class relies exclusively on RAII-compliant types (e.g., `std::vector`, `std::string`, or smart pointers), the compiler-generated destructor is sufficient, aligning with the Rule of Zero and promoting safer, more maintainable code.

Rule of Zero

The Rule of Zero is a C++ design principle that states that a class should not explicitly define any of the custom member functions—namely the destructor, copy constructor, copy assignment operator, move constructor, or move assignment operator—if it does not manage resources directly. Instead, resource ownership and lifetime management should be delegated to **RAII-compliant class data members**, typically provided by standard library types such as `std::vector`, `std::string`, and smart pointers. Class member functions then operate on these RAII-managed resources without performing explicit allocation or deallocation. When such types are used, the compiler-generated member functions are sufficient to correctly manage object lifetimes, copying, and destruction. Adhering to the Rule of Zero results in safer and more robust code by eliminating manual resource management, reducing errors such as memory leaks and double deletion, and improving maintainability and exception safety.

The Code Example 3-8 does not comply with the Rule of Zero. It relies on manual deletion of a dynamically allocated array, which the programmer must manage explicitly. In contrast, smart pointers handle resource cleanup automatically. Mixing manual and automatic memory management increases complexity and becomes difficult to maintain in larger projects, where significant effort is spent on correct and consistent resource handling.

Code Example 3-8 is rewritten to conform to the Rule of Zero.

Code Example 3-9: Adhere to the Rule of Zero

```

1  #include <iostream>
2  #include <memory>
3  #include <vector>
4  using namespace std;
5
6  class Base {
7  public:
8      virtual ~Base() = default;    // required for polymorphic deletion
9  };
10
11 class Derived : public Base {
12 private:
13     std::vector<int> data;    // RAII-managed dynamic array
14 public:
15     Derived() : data(100) {    // allocate 100 integers
16         cout << "Derived constructor allocated memory" << endl;

```

```

17     }
18     // no destructor needed (Rule of Zero)
19 };
20
21 int main() {
22     std::unique_ptr<Base> obj = std::make_unique<Derived>();
23     return 0;
24 }

```

Line 2: Includes support for smart pointers, such as `std::unique_ptr`.

Line 3: Includes the `std::vector` container, which provides a RAII-managed dynamic array.

Line 8: Declares a virtual destructor for the base class. `virtual` ensures correct polymorphic destruction when deleting through a `Base*`. The `= default` tells the compiler to generate the destructor automatically. No manual cleanup is needed in Base.

Line 13: Declares a `std::vector` of integers. This is a dynamic array managed by RAII. The vector automatically allocates and frees memory.

Line 15: Defines the constructor of Derived. The initializer list `data(100)` creates a vector with 100 integers. Memory allocation happens safely inside the vector. No destructor is written. `std::vector` cleans up its own memory.

Line 22: Creates a Derived object on the heap, stores it in a `std::unique_ptr<Base>`. This enables polymorphism and automatic memory management. When `obj` goes out of scope, the object is deleted safely.

In short, a default destructor automatically destroys stack-allocated members and Standard Library RAII types. However, raw pointers, C-style resources, for example `FILE* f = fopen("data.txt", "r");` and OS-level handles such as sockets, file descriptors, GPU contexts, and C-API mutexes are **not** released by a default destructor and therefore require explicit or via RAII wrapper cleanup.

-  Polymorphic base classes should declare a virtual destructor (often `= default`) to enable safe deletion through base-class pointers. Derived classes do not need their own destructors if all data members are RAII-compliant, as the compiler-generated destructors are sufficient. In non-polymorphic designs, neither base nor derived classes require destructors provided all resources are managed via RAII.

3.3 Abstract Class

An abstract class in C++ is a class that is intended to serve as a base for other classes and cannot be instantiated directly. It is characterized by the presence of at least one pure virtual function, which is declared by assigning `= 0` in its declaration. The primary purpose of an abstract class is to define a common interface and enforce a contract that derived classes must follow by providing implementations for the pure

Pure virtual
function

virtual functions. Abstract classes support runtime polymorphism, allowing objects of derived classes to be accessed through base-class pointers or references while ensuring the correct function implementations are invoked at runtime. In addition to pure virtual functions, an abstract class may contain data members, constructors, concrete member functions, and virtual destructors, enabling the sharing of both behavior and structure among related classes. A simple abstract class is shown below.

Code Example 3-10: Example of an abstract class

```

1 class Shape {
2 public:
3     virtual void draw() = 0;    // pure virtual function
4 };
5
6 int main() {
7     Shape s;    // ERROR
8     Shape* s;   // OK (pointer)
9 }

```

In this example, the class `Shape` is an abstract class because it contains a pure virtual function `draw()`, which is declared using `= 0`. In C++, abstract classes cannot be instantiated, because they represent incomplete types whose behavior is not fully defined. Therefore, the statement `Shape s;` results in a compilation error, as the compiler does not allow the creation of an object from an abstract class.

However, declaring a pointer of type `Shape* s` is valid. This is because a pointer does not create an object; it only reserves space to hold the address of an object. Such a pointer is typically used to point to an object of a derived class that provides implementations for all pure virtual functions. This mechanism enables runtime polymorphism, allowing the program to interact with different derived objects through a common base-class interface while deferring the actual function calls to runtime.

i The term **abstract function** is not defined in the C++ standard. It is commonly used informally in teaching, borrowed from languages like Java and C#, and serves as a conceptual shortcut to mean a pure virtual function—that is, a function with no implementation that must be overridden in a derived class.

Abstract data type

It is worth mentioning another closely related concept to an abstract class, known as an Abstract Data Type (ADT). An ADT is a conceptual model that defines a data type in terms of its behavior and permitted operations, rather than its internal representation. An ADT specifies what operations can be performed on the data and what those operations mean, while deliberately hiding how the data is stored or implemented. This separation of interface from implementation allows the same ADT to have multiple underlying implementations without affecting the code that uses it. In C++, ADTs are commonly expressed using abstract classes with pure virtual func-

tions, though templates and other design techniques can also be used to achieve the same abstraction.

i The term ADT is language-independent and belongs to computer science theory rather than to any specific programming language.

The following example implements ADT using pure virtual functions.

Code Example 3-11: ADT implementation using pure virtual functions

```
1 class Stack {
2 public:
3     virtual void push(int) = 0;
4     virtual void pop() = 0;
5     virtual int top() const = 0;
6     virtual bool empty() const = 0;
7     virtual ~Stack() = default;
8 };
```

This code defines an abstract class `Stack` that models the stack ADT by declaring its required operations; `push()`, `pop()`, `top()`, and `empty()` as pure virtual functions. The class specifies what a stack must do without defining how it does it, forcing derived classes to provide concrete implementations. The virtual destructor ensures safe polymorphic deletion when a stack object is accessed through a base-class pointer.

Code Example 3-12: Implementation of virtual functions

```
1 class ArrayStack : public Stack {
2     std::vector<int> data;
3 public:
4     void push(int x) override { data.push_back(x); }
5     void pop() override { data.pop_back(); }
6     int top() const override { return data.back(); }
7     bool empty() const override { return data.empty(); }
8 };
```

This code defines `ArrayStack`, a concrete implementation of the `Stack` ADT that uses a `std::vector<int>` as its underlying storage. The class inherits from `Stack` and overrides all pure virtual functions, providing concrete definitions for `push()`, `pop()`, `top()`, and `empty()`. By storing elements in a `std::vector`, the stack supports dynamic resizing while preserving LIFO (Last-In, First-Out) behavior. Because all required functions are implemented, `ArrayStack` can be instantiated and used polymorphically through a `Stack` pointer or reference.

Interface class

An abstract class that consists solely of pure virtual functions and contains no data members, as shown in Code Example 3-11, is called an interface class, and it defines a strict contract that derived classes must implement while providing no implementation details of its own. An abstract class may contain a mix of implemented and pure virtual functions, along with data members, whereas an interface class contains only pure virtual functions and no data members, serving solely to define a contract for derived classes.

In practice, an interface class is used to decouple code, enable polymorphism, and enforce a clear contract between components. Common practical uses include:

- **Multiple implementations:** Allow different classes to provide different implementations of the same behavior (e.g., array-based vs. linked-list stack) while being used interchangeably.
- **Loose coupling:** Client code depends on the interface, not the concrete class, making the system easier to modify and extend.
- **Polymorphic behavior:** Enable runtime substitution of objects through base-class pointers or references.
- *Testability:* Facilitate mocking or stubbing in unit tests by swapping real implementations with test doubles.
- **Plugin and framework design:** Define extension points where users can supply custom implementations without changing existing code.
- **Clear API boundaries:** Precisely specify what a component guarantees, improving maintainability and collaboration in large codebases.

i If a system has a single, stable implementation and is unlikely to change, introducing an interface class may add unnecessary complexity. However, if the code is expected to evolve, support multiple implementations, enable polymorphic use, or be tested in isolation, an interface class provides long-term benefits by clearly separating what a component does from how it does it.

3.4 Examples - A Polymorphic Design

3.4.1 *Payment Processing System*

Consider online payment systems such as credit cards, PayPal, and bank transfers. Although the internal mechanisms of these payment methods differ significantly, they all exhibit a common set of responsibilities: each must support payment processing and refund operations. This combination of shared behavioral requirements with distinct underlying implementations makes online payment systems a representative real-world use case for abstract classes where a common interface can be defined while allowing specialized behavior in derived classes.

The UML class diagram in Figure 3.1 illustrates in a payment hierarchy based on

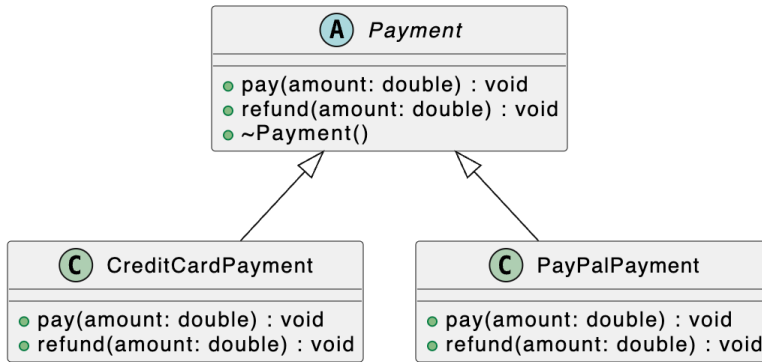


FIGURE 3.1 UML - Payment processing system

abstraction and inheritance. The **Payment** class is an abstract class defining the common operations `pay` and `refund`, along with a virtual destructor for safe polymorphic destruction; it is marked by an **A** in a circle, indicating an abstract class. The **CreditCardPayment** and **PayPalPayment** classes are concrete subclasses that implement these operations and are marked by a **C** in a circle, denoting concrete, instantiable classes. The inheritance relationships show how concrete payment types specialize the abstract **Payment** interface to support runtime polymorphism.

Code Example 3-13: Interface class

```

1 #include <iostream>
2 #include <memory>
3 using namespace std;
4
5 class Payment {
6 public:
7     virtual void pay(double amount) = 0;
8     virtual void refund(double amount) = 0;
9
10    // Rule of Zero: no custom destructor logic
11    virtual ~Payment() = default;
12 };
  
```

The code defines an abstract base class **Payment** that represents a generic interface for different types of payment mechanisms. It declares two pure virtual functions, `pay()` and `refund()`, which establish a contractual requirement that all derived classes must implement payment processing and refund behavior. Because these functions are pure virtual, **Payment** cannot be instantiated directly and is intended to be used polymorphically through base-class pointers or references. The destructor is declared virtual and explicitly defaulted, ensuring that derived objects are destroyed correctly when deleted through a **Payment** pointer while adhering to the Rule of Zero—no custom resource management is introduced in the base

class. Overall, this design cleanly separates interface from implementation and enables safe runtime polymorphism.

Code Example 3-14: Derived classes

```

1 class CreditCardPayment : public Payment {
2 public:
3     void pay(double amount) override {
4         cout << "Paid $" << amount << " using Credit Card\n";
5     }
6
7     void refund(double amount) override {
8         cout << "Refunded $" << amount << " to Credit Card\n";
9     }
10 };
11
12 class PayPalPayment : public Payment {
13 public:
14     void pay(double amount) override {
15         cout << "Paid $" << amount << " using PayPal\n";
16     }
17
18     void refund(double amount) override {
19         cout << "Refunded $" << amount << " via PayPal\n";
20     }
21 };

```

The code defines two concrete classes, **CreditCardPayment** and **PayPalPayment**, both of which publicly inherit from the abstract base class **Payment**. Each class provides its own implementation of the pure virtual functions **pay** and **refund**, thereby fulfilling the interface contract imposed by **Payment**. The **override** keyword is used to explicitly indicate that these member functions override virtual functions from the base class, enabling compile-time checking and improving code correctness. Through this design, objects of these derived classes can be treated polymorphically as **Payment** types while exhibiting behavior specific to the chosen payment method, illustrating runtime polymorphism and the separation of interface from implementation.

Code Example 3-15: Main function

```

1 int main() {
2     auto payment1 = make_unique<CreditCardPayment>();
3     auto payment2 = make_unique<PayPalPayment>();
4
5     payment1->pay(150.0);
6     payment2->pay(80.0);

```

```

7
8     payment1->refund(30.0);
9     payment2->refund(20.0);
10
11     // Rule of Zero:
12     // No delete, no destructor calls, no cleanup code
13 }

```

The main function demonstrates the use of polymorphism and RAII through smart pointers in C++. Two objects, **CreditCardPayment** and **PayPalPayment**, are dynamically created using **std::make_unique** and stored in **std::unique_ptr** instances, ensuring exclusive ownership and automatic resource management. The objects are accessed through pointer semantics to invoke the pay and refund functions, which are resolved at runtime based on the dynamic type of the objects, illustrating runtime polymorphism. When main exits, the **std::unique_ptr** instances automatically destroy the managed objects, invoking the appropriate destructors without requiring explicit delete statements. This design adheres to the Rule of Zero by eliminating manual memory management and cleanup code, resulting in safer and more maintainable C++ code.

3.4.2 Data Cleaning Pipeline

A data-cleaning pipeline represents a structured sequence of transformations applied to raw data before analysis or modeling. Rather than embedding all cleaning logic in a single block of code, the pipeline decomposes the task into distinct stages, each responsible for a specific operation such as removing invalid values, handling outliers, or normalizing data. This separation makes the overall process easier to understand, maintain, and extend.

In a polymorphic design, each cleaning stage implements a common interface that defines how data should be cleaned. The pipeline interacts only with this interface, not with concrete implementations. As a result, different cleaning strategies can be substituted or reordered without modifying the pipeline itself. For example, one stage may replace negative values with zeros, while another caps extreme values to a specified maximum, yet both are invoked in exactly the same way.

By applying cleaners sequentially to the same dataset, the pipeline ensures that each stage operates on the output of the previous one, mirroring real-world data preparation workflows. New cleaning steps can be introduced simply by adding new classes that implement the interface, leaving existing code unchanged. This approach demonstrates how polymorphism enables flexible, extensible, and well-structured data-cleaning pipelines in C++.

A UML class diagram for a data-cleaning pipeline illustrates how different cleaning strategies are organized around a common interface. The diagram shows an interface class that defines the cleaning operation and multiple concrete classes that implement this operation in different ways, such as removing invalid values or capping outliers.

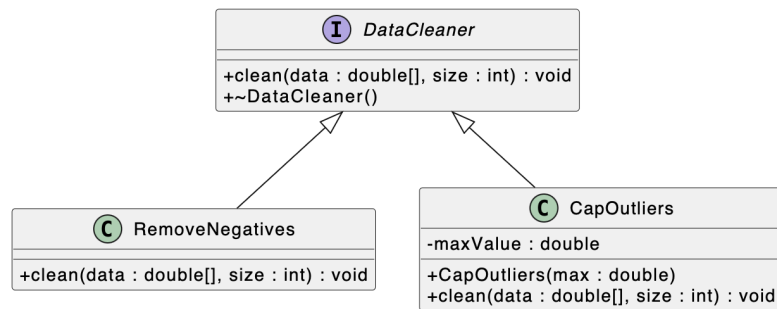


FIGURE 3.2 UML - Data cleaning pipeline

By highlighting the inheritance relationships, the UML makes it clear how polymorphism allows each cleaner to be used interchangeably within the same pipeline.

3.5 Concept Check

Virtual functions

1. *What problem do virtual functions solve in object-oriented programming?*

Virtual functions in C++ solve the problem of incorrect function dispatch when using base-class pointers or references. By default, C++ uses static binding, meaning a function call is resolved at compile time based on the pointer type, which can cause the base-class version of a function to be called even when the object is of a derived type. Declaring a function as virtual enables dynamic (runtime) binding, ensuring that the function corresponding to the object's actual (dynamic) type is executed. This mechanism is essential for runtime polymorphism, allowing a common interface to support different behaviors in derived classes.

2. *What does it mean for a function call to be resolved at runtime?*

A function call being resolved at runtime means that the decision about which function implementation to execute is made while the program is running, rather than at compile time. In C++, this occurs for virtual functions called through base-class pointers or references, where the call is dispatched based on the object's dynamic type (the actual object created) instead of the pointer's static type. This runtime resolution enables polymorphic behavior, allowing the correct overridden function in a derived class to be executed.

3. *Is it mandatory to use the override keyword?*

No, the override keyword is not strictly necessary in C++. A function in a derived class will still override a virtual base-class function as long as the function signature matches, even without writing override.

4. *Why does runtime polymorphism in C++ require a base-class pointer or reference?*

Runtime polymorphism in C++ requires a base-class pointer or reference because dynamic binding only occurs when the compiler cannot determine the object's exact type at compile time. When a virtual function is called through a base-class pointer or reference, C++ defers the decision until runtime and selects the function based on the object's dynamic type. In contrast, when a function is called on an object directly (not via a pointer or reference), the object's type is known at compile time, so the call is resolved statically and polymorphism does not occur.

5. *Can a virtual function exist even if no base-class pointer is ever used?*

Yes, a virtual function can exist even if no base-class pointer or reference is ever used. Declaring a function as virtual simply makes it eligible for dynamic binding, but runtime polymorphism only occurs when the function is called through a base-class pointer or reference. If all calls are made directly on objects, the function will still compile and work correctly, but the calls are resolved at compile time, and the virtual mechanism is never actually exercised.

6. *What happens if a derived class does not override a virtual function from its base class?*

If a derived class does not override a virtual function from its base class, it automatically inherits the base-class implementation. When the function is called through a base-class pointer or reference, the base version is executed because no overridden version exists in the derived class. The function remains virtual, so further derived classes can still override it, but until that happens, the base-class behavior is used.

7. *Why are virtual function calls generally slower than non-virtual calls?*

Virtual function calls are generally slower than non-virtual calls because they are resolved at runtime using dynamic dispatch. Instead of calling a function directly, the program must look up the correct function address—typically through a virtual table (vtable)—based on the object's dynamic type. This extra level of indirection prevents certain compile-time optimizations, such as inlining, and adds a small overhead compared to direct (statically bound) function calls.

8. *Can constructors be virtual?*

No, constructors cannot be virtual in C++. Constructors are responsible for creating and initializing an object, and during construction the object's dynamic type is not yet fully established. Virtual dispatch relies on the object already having a valid dynamic type and vtable, which is not the case while a base-class constructor is running. As a result, C++ resolves constructor calls statically, and allowing virtual constructors would be conceptually meaningless and unsafe.

9. *When should a function be declared as virtual?*

Make a function virtual if different derived classes should provide different behavior, and you want the correct behavior to be chosen at runtime.

10. *What restriction applies when calling member functions through a base-class*

pointer in C++?

When using a base-class pointer, you can only call methods declared in the base class, not those defined exclusively in the derived class.

Virtual destructors

1. *What happens if a polymorphic-base class does not have a virtual destructor and an object is deleted through a base-class pointer?*

If a base class does not declare a virtual destructor and an object is deleted through a base-class pointer, the compiler will still generate a default (non-virtual) base-class destructor if none is explicitly defined. When delete is applied to the base-class pointer, only the compiler-generated base-class destructor is called, while the derived-class destructor is skipped. This results in undefined behavior, commonly leading to resource leaks because the derived portion of the object is not properly destroyed.

2. *What happens if the base class has a virtual destructor but the derived class does not define its own destructor?*

If the base class declares a virtual destructor and the derived class does not define one, the compiler implicitly generates a destructor for the derived class, and deleting the object through a base-class pointer correctly calls the derived destructor first, followed by the base destructor. The compiler-generated destructor calls the destructors of all data members, but it **only** correctly releases resources for RAII-compliant members; raw pointers and manually managed resources are not automatically cleaned up.

3. *Should every class have a virtual destructor?*

No, not every class should have a virtual destructor. A destructor only needs to be virtual if the class is intended to be used polymorphically, meaning objects may be deleted through a base-class pointer or reference. Making a destructor virtual adds a small runtime and memory overhead, so for classes that are not used as base classes—or are never deleted via base pointers—a non-virtual destructor is sufficient and preferable. As a rule of thumb, declare a virtual destructor only in classes designed to serve as polymorphic base classes.

4. *How does declaring a destructor as `= default` relate to the Rule of Zero?*

Declaring a destructor as `= default` aligns with the Rule of Zero by explicitly stating that the class relies entirely on RAII-managed members for resource cleanup. It lets the compiler generate the correct destructor automatically, avoids manual resource management, and preserves the class's trivial, safe behavior—just as if no destructor were written at all.

5. *How do smart pointers support RAII?*

Smart pointers support RAII by tying resource ownership to object lifetime: they acquire resources when created and automatically release them in their destructors when they go out of scope. This guarantees deterministic cleanup, prevents leaks, and eliminates the need for manual delete.

6. *Why is `std::unique_ptr` often preferred over raw pointers?*

`std::unique_ptr` is often preferred over raw pointers because it enforces single ownership, automatically releases resources via RAII, prevents accidental copying, and makes ownership explicit and safe—eliminating common errors like memory leaks and double deletion.

7. *Why must the base class have a virtual destructor when managing objects via smart pointers?*

Because smart pointers delete objects through the static type they hold. If a smart pointer holds a `Base*` to a `Derived` object, the base class must have a virtual destructor to ensure the derived destructor is called, allowing complete and correct resource cleanup. Without it, destruction is incomplete and results in undefined behavior.

8. *What happens to the managed object when a smart pointer goes out of scope?*

When a smart pointer goes out of scope, its destructor is called, which automatically deletes the managed object (and releases the resource), ensuring proper cleanup via RAII.

9. *Can smart pointers eliminate all memory leaks?*

No. Smart pointers greatly reduce memory leaks by automating ownership and cleanup, but they cannot eliminate all leaks. Leaks can still occur due to issues like reference cycles with `std::shared_ptr`, holding smart pointers in long-lived structures unintentionally, or leaking non-memory resources that are not managed by RAII.

Abstract class

1. *Can an abstract class contain data members?*

Yes, an abstract class in C++ can contain data members. Abstractness is determined solely by the presence of at least one pure virtual function, not by whether the class has data. Data members are often used to store shared state or common implementation details that derived classes can reuse, while pure virtual functions define the interface that derived classes must implement.

2. *Why can abstract classes not be instantiated?*

Abstract classes cannot be instantiated because they contain at least one pure virtual function with no implementation, meaning the class represents an incomplete type. Since the behavior of these functions is undefined at the abstract-class level, the compiler prevents object creation to avoid calling functions that have no concrete definition. Abstract classes are intended to serve as base classes that define a contract, requiring derived classes to provide implementations before objects can be created.

3. *Why does C++ not have a dedicated interface keyword like Java?*

C++ does not have a dedicated interface keyword because the language already provides the necessary flexibility through abstract classes and pure virtual functions. An abstract class with only pure virtual functions (and typically no data

members) naturally serves the role of an interface, while still allowing C++ to support multiple inheritance, default implementations, and fine-grained control over object layout and performance. This design reflects C++'s philosophy of offering powerful low-level mechanisms rather than rigid high-level constructs, giving programmers more control over how interfaces and implementations are expressed.

4. *When is it appropriate to use an interface class instead of a concrete base class?*

It is appropriate to use an interface class when you want to define a pure contract that specifies what operations a class must support without prescribing how they are implemented. Interface classes are ideal when multiple, unrelated classes need to conform to the same API, when you are designing libraries or frameworks for others to extend, or when you want to decouple components to reduce dependencies. In contrast, a concrete base class is more suitable when shared data or common implementation can be meaningfully reused across derived classes.

5. *Can an abstract class provide implementations for some member functions?*

Yes, an abstract class in C++ can provide implementations for some of its member functions. A class is considered abstract only because it contains at least one pure virtual function; other virtual or non-virtual functions may have full implementations. This allows an abstract class to supply shared behavior or default functionality while still requiring derived classes to implement the remaining pure virtual functions.

4

Linked Lists

Basic Concept of Linked Lists	82
Linked Lists Using Smart Pointers	101
Linked Lists Using the STL	101
Applications of Linked Lists	106

Arrays are among the most fundamental data structures in computer science, offering efficient constant-time $O(1)$ access to elements through direct indexing. However, despite their simplicity and performance advantages in read operations, arrays exhibit several inherent limitations that make them less suitable for dynamic and frequently changing datasets. One of the primary constraints of arrays is their fixed size. In many programming environments, the size of an array must be determined at the time of allocation, and although dynamic arrays (such as vectors in C++) provide some flexibility, they still rely on underlying resizing mechanisms that can incur additional computational overhead.

A significant drawback of array-based storage arises in the context of insertion and deletion operations. Since arrays store elements in contiguous memory locations, inserting or removing an element at any position other than the end requires shifting subsequent elements to maintain order. This results in a time complexity of $O(n)$, which becomes increasingly inefficient as the size of the dataset grows. Such behavior makes arrays less suitable for applications where frequent updates to the data structure are required.

Another limitation is related to memory management and reallocation. When a dynamic array exceeds its current capacity, it must allocate a new block of memory, typically larger than the original, and copy all existing elements to the new location. This resizing operation is costly, both in terms of time and system resources, and may lead to memory fragmentation. Furthermore, repeated reallocations can degrade performance in systems that demand consistent and predictable execution times.

These limitations motivate the need for alternative data structures that can efficiently handle dynamic data. Linked structures, such as linked lists, address many of these challenges by using non-contiguous memory allocation and pointer-based connections between elements. This design eliminates the need for shifting elements during insertion or deletion and allows the structure to grow or shrink dynamically without requiring large-scale memory reallocation. Consequently, linked lists provide a more flexible and adaptable approach for managing data in environments characterized by frequent modifications.

This chapter first explores linked lists using raw pointers, allowing the reader to build a strong conceptual foundation and understand the role of pointer manipulation. Later, the chapter transitions to the use of linked lists in production environments, where they are implemented and managed through the Standard Template Library (STL).

4.1 Basic Concept of Linked Lists

A linked list is a fundamental linear data structure that organizes elements in a sequential manner using a collection of interconnected nodes rather than contiguous memory locations. Each node in a linked list is a composite structure consisting of two primary components: the data field, which stores the value of the element, and a

pointer (or reference) field, which stores the address of the next node in the sequence. This design enables the construction of a chain-like structure in which each node is logically linked to its successor, thereby forming a dynamic sequence of elements.

Unlike arrays, linked lists do not require contiguous memory allocation. Nodes can be stored in arbitrary locations in memory, and their logical order is maintained through pointer connections rather than physical adjacency. This non-contiguous allocation provides significant flexibility, allowing the data structure to grow or shrink dynamically without the need for memory reallocation or element shifting. As a result, linked lists are particularly well-suited for applications involving frequent insertions and deletions.

The entry point to a linked list is defined by a special pointer known as the head pointer. The head pointer stores the address of the first node in the list and serves as the primary means of accessing and traversing the entire structure. If the head pointer is null, the list is considered empty. Traversal of the linked list begins at the head and proceeds sequentially by following the pointer from one node to the next until a null pointer is encountered, indicating the end of the list.

Conceptually, a linked list can be visualized as a series of nodes connected by directional links, where each node points to its successor; this structure is commonly referred to as a unidirectional linked list because traversal proceeds in only one direction. This visualization emphasizes the logical relationships between elements rather than their physical placement in memory. Such a representation is instrumental in understanding the behavior of linked lists, particularly in operations such as traversal, insertion, and deletion, where pointer manipulation plays a central role.

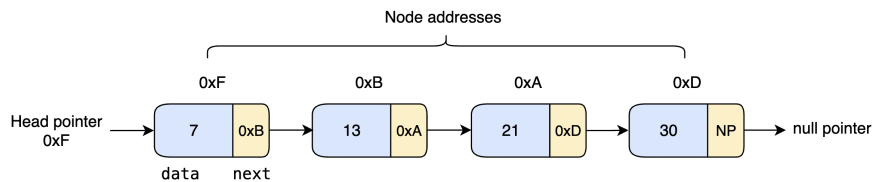


FIGURE 4.1 *Unidirectional linked list*

i Head and null pointers

The head pointer is a reference (or variable) that stores the memory address of the first node in a linked list. It serves as the primary access point to the entire data structure. All operations on the linked list—such as traversal, insertion, and deletion—begin from the head pointer. Importantly, the head itself is not a node and does not store any data element; rather, it simply points to the first node. If the linked list is empty, the head pointer contains a null value **nullptr**, indicating that no nodes are present.

A null pointer, represented in C++ by **nullptr**, is a special value that signifies

the absence of a valid memory address. In the context of linked lists, it is used to indicate two important conditions. First, when the head pointer is equal to `nullptr`, it denotes that the list is empty. Second, within the list, the pointer field of the last node is set to `nullptr`, marking the termination of the list. The null pointer does not refer to any actual node or memory location; it simply indicates that there is no subsequent element.

The given program shown in Code example 4-1 demonstrates the fundamental construction of a unidirectional (singly) linked list in C++ through explicit dynamic memory allocation and pointer manipulation. The `Node` structure defines the basic building block of the linked list, where each node consists of two components: an integer data field (`data`) that stores the value, and a pointer (`next`) that holds the address of the subsequent node in the sequence. This design reflects the core characteristic of linked lists, in which elements are not stored contiguously in memory but are instead connected through pointers.

Within the main function, four nodes are dynamically allocated on the heap using the `new` operator. Each node is initialized with a specific integer value (7, 13, 21, and 13) and a `nullptr` pointer, indicating that, at the time of creation, the nodes are not yet linked to any successor. The pointer variable `head` is then assigned the address of the first node, thereby establishing the entry point to the linked list. It is important to note that the head pointer does not store any data itself; rather, it serves as a reference to the first node, enabling access to the entire structure.

Subsequently, the nodes are connected by explicitly assigning the next pointer of each node to the address of its successor. Specifically, the first node points to the third node, the third node points to the fifth node, and the fifth node points to the seventh node. The final node retains its `nullptr` value in the next field, signifying the termination of the list. This sequential linking establishes a chain of nodes that can be traversed starting from the head pointer. Overall, the program provides a clear illustration of how a singly linked list is constructed and organized in memory using pointers.

The code uses `struct` for simplicity. In C++, the primary difference between a `struct` and a class is that members of a `struct` are public by default, whereas members of a class are private by default.

Since a linked list node requires direct access to its members (e.g., `data` and `next`) for pointer manipulation, using a `struct` avoids the need to explicitly declare them as public. Although a class could also be used with a `public:` section, `struct` is preferred here as it represents a simple data container and keeps the code concise and clear.

Code Example 4-1: Creating nodes in a linked list

```
1 struct Node
2 {
3     int data;
4     Node* next;
```

```

5 };
6
7 int main()
8 {
9     // Create 4 nodes
10    Node* first = new Node{7, nullptr};
11    Node* third = new Node{13, nullptr};
12    Node* fifth = new Node{21, nullptr};
13    Node* seventh = new Node{30, nullptr};
14
15    // Head pointer points to first node
16    Node* head = first;
17
18    // Link the nodes
19    first->next = third;
20    third->next = fifth;
21    fifth->next = seventh;
22
23    return 0;
24 }

```

Lines 1–5: Defines a `Node`, the fundamental unit of a singly linked list, containing an integer value (data) and a pointer (next) to the next node in the sequence. The pointer `Node* next` stores the memory address of the next node in the list, thereby linking nodes together. It enables traversal by allowing each node to point to its successor, forming a chain structure.

Lines 10–13: Dynamically creates four nodes on the heap using the `new` operator. Each pointer (first, third, fifth, and seventh) stores the address of a newly allocated `Node` initialized with a specific integer value (7, 13, 21, and 30) and a `nullptr` in the next field. At this stage, the nodes exist independently in memory and are not yet linked to one another.

Line 16: This statement assigns the pointer `head` to the first node, making it the entry point of the linked list. Through `head`, the entire list can be accessed and traversed, as it points to the beginning of the chain.

Lines 19–21: These statements link the nodes together by assigning each node's next pointer to the address of its successor. Specifically, the first node points to the third, the third points to the fifth, and the fifth points to the seventh, thereby forming a sequential chain of nodes that constitutes the linked list. The seventh node does not need to be explicitly linked to `nullptr`, as it was already initialized with `nullptr` during its creation, indicating the end of the list.

Traversal in a singly linked list refers to the process of visiting each node sequentially, starting from the head pointer and following the chain of references until the end of the list is reached. Since each node contains a pointer only to its immediate successor, traversal must proceed in a strictly linear fashion, without the ability to skip elements or access nodes directly by index. Consequently, in the worst case, all n nodes must

be visited to locate a particular element or to process the entire structure. This results in a time complexity of $O(n)$, making traversal inherently linear in nature.

Code Example 4-2: Traversing a linked list

```

1 void traverse(Node* head) {
2     Node* temp = head;
3     while (temp != nullptr)
4     {
5         cout << temp->data << " -> ";
6         temp = temp->next;
7     }
8     cout << "nullptr" << endl;
9 }
```

The function `traverse` performs a sequential traversal of a singly linked list starting from the head node. A temporary pointer, `temp`, is initialized to the head and iteratively advances through each node using the `next` pointer until a `nullptr` is encountered, indicating the end of the list. During each iteration, the data stored in the current node is printed, followed by an arrow to represent linkage, and finally `nullptr` is printed to denote termination. The traversal logic can be readily customized, for example, to search for a specific node or value during iteration instead of merely printing the elements.

Insertion in a singly linked list is an efficient operation when performed at specific locations. Inserting a new node at the beginning of the list can be achieved in constant time $O(1)$, as it requires only updating the new node's pointer to the current head and then reassigning the head pointer to the new node. Insertion at the end of the list can also be performed in $O(1)$ time if a tail pointer is maintained, as the new node can be directly appended and the tail updated accordingly. However, in the absence of a tail pointer, insertion at the end requires traversal of the entire list to locate the last node, resulting in a time complexity of $O(n)$. Similarly, inserting a node at an arbitrary position within the list necessitates traversing the list to locate the desired position, followed by appropriate pointer adjustments, resulting in an overall time complexity of $O(n)$. This case is illustrated in Example 4-3. It should be noted that the `traverse()` function in this example incorporates search functionality and thus differs slightly in behavior from the `traverse()` function described in Example code 4-2.

Code Example 4-3: Inserting a node at an arbitrary position within the list

```

1 Node* traverse(Node* head, int search_data) {
2     Node* temp = head;
3     while (temp != nullptr)
4     {
5         if (temp->data == search_data) {
6             return temp;
7         }
8     }
9     return nullptr;
10 }
```

```

7         }
8         temp = temp->next;
9     }
10    return nullptr;
11 }
12
13 void insert(Node* head, int search_data, int new_data) {
14     Node* found_node = traverse(head, search_data);
15
16     if (found_node == nullptr) {
17         cout << "Data not found" << endl;
18         return;
19     }
20
21     Node* new_node = new Node{new_data, found_node->next};
22     found_node->next = new_node;
23 }

```

Line 1: The function `Node* traverse(Node* head, int search_data)` is designed to search the linked list for a node containing a specified value. Its return type, `Node*`, indicates that the function returns the address of a node rather than a simple value. The parameter `head` represents the starting point of the linked list, while `search_data` specifies the target value being sought.

Line 2: Within this function, the statement `Node* temp = head;` creates a temporary pointer named `temp` and initializes it with the address stored in `head`. This means that `temp` initially points to the first node in the list. The purpose of using `temp` is to traverse the list without altering the original head pointer.

Lines 3–10: The loop `while (temp != nullptr)` continues execution as long as `temp` points to a valid node. This condition ensures that traversal proceeds node by node and stops when the end of the list is reached. Inside the loop, the condition `if (temp->data == search_data)` compares the data stored in the current node with the target value. If the two values are equal, the statement `return temp;` immediately returns the address of that node, thereby allowing the calling function to access it directly. If the current node does not contain the desired value, the statement `temp = temp->next;` advances the temporary pointer to the next node in the sequence. In this way, the function examines each node one at a time. If the loop completes without finding a matching value, the statement `return nullptr;` is executed, indicating that no node in the list contains the requested data.

Line 13: The function `void insert(Node* head, int search_data, int new_data)` performs the insertion operation. Its purpose is to insert a new node after the node containing `search_data`. The parameter `head` identifies the beginning of the list, `search_data` specifies the existing node after which insertion should occur, and `new_data` is the value to be stored in the newly created node.

Line 14: The statement `Node* found_node = traverse(head, search_data);` calls the traversal function to locate the node containing the specified search value. The returned pointer is stored in `found_node`, which represents the node after which the new node will be inserted.

Lines 16–18: The conditional statement `if (found_node == nullptr)` checks whether the search was unsuccessful. If `traverse` did not find the target node, it returned `nullptr`, and in that case the insertion cannot proceed. The statement `cout << "Data not found" << endl;` outputs an error message, and `return;` terminates the function immediately.

Line 21: If a valid node is found, the statement `Node* new_node = new Node(new_data, found_node->next);` dynamically allocates memory for a new node. This new node is initialized so that its data member stores `new_data`, and its next pointer stores the address that was previously held in `found_node->next`. In other words, the new node is prepared to point to the node that originally followed `found_node`.

Line 22: Finally, the statement `found_node->next = new_node;` updates the next pointer of the `found_node` node so that it now points to the newly created node. This completes the insertion process. As a result, the new node is placed immediately after the node containing `search_data`, while the remainder of the list remains properly connected.

Overall, this code provides a clear example of how insertion in a singly linked list is achieved through explicit pointer manipulation using raw pointers.

Deletion operations in a singly linked list also depend on the position of the node being removed. Deleting the first node is a constant-time operation $O(1)$, as it involves updating the head pointer to reference the next node in the list. In contrast, deleting the last node in a singly linked list requires traversal to identify the penultimate node, since there is no backward pointer to the previous node. This makes deletion at the end an $O(n)$ operation. Deletion at a specific position similarly necessitates traversal to locate the preceding node, after which the pointer is adjusted to bypass the target node.

Accordingly, deletion at an arbitrary position exhibits a time complexity of $O(n)$. This scenario is demonstrated in Example 4-4, where the `traverse()` function is reused from Example 4-3.

Code Example 4-4: Deleting a node at an arbitrary position within the list

```

1 void deleteNode(Node* head, int search_data) {
2     Node* ptr_search_data = traverse(head, search_data);
3
4     // If node not found or no next node to delete
5     if (found_node == nullptr || found_node->next == nullptr) {
6         cout << "Deletion not possible" << endl;
7         return;
8     }
9
10    // Node to delete
11    Node* nodeToDelete = found_node->next;
12
13    // Update links (bypass the node)
14    found_node->next = nodeToDelete->next;
15

```



```

16 // Free memory
17 delete nodeToDelete;
18 }

```

Line 1: The function `deleteNode(Node* head, int search_data)` is designed to remove a node from a singly linked list based on a specified value, using a previously defined traversal routine. The function begins by invoking the `traverse(head, search_data)` function and storing its result in the pointer variable `found_node`. This call attempts to locate the node containing `search_data`, and if successful, returns a pointer to that node. Conceptually, this node serves as the predecessor of the node to be deleted.

Lines 4–8: The conditional statement checks whether deletion is feasible. The condition `found_node == nullptr` indicates that the traversal did not locate any node containing the specified value, implying that the target position for deletion does not exist. The condition `found_node->next == nullptr` signifies that the located node is the last node in the list and therefore does not have a successor. Since the deletion strategy employed in this function removes the node following `found_node`, the absence of a successor makes deletion impossible. In either case, an appropriate message is displayed and the function terminates early using the `return` statement.

Line 11: If deletion is possible, the pointer `nodeToDelete` is introduced and assigned the value `found_node->next`. This pointer now references the node that is intended to be removed from the list. Storing this reference is essential to ensure that the node can be safely deallocated after the list structure has been updated.

Line 14: The next statement, `found_node->next = nodeToDelete->next`, performs the critical pointer manipulation required for deletion. Specifically, it updates the next pointer of the `found_node` node so that it bypasses the node to be deleted and instead points directly to its successor. This operation preserves the continuity of the linked list, ensuring that all remaining nodes remain properly connected.

Line 17: Finally, the statement `delete nodeToDelete`; deallocates the memory associated with the removed node. This step is necessary in environments using manual memory management, such as C++, to prevent memory leaks.

Overall, the function demonstrates how deletion in a singly linked list can be effectively achieved through a combination of traversal and precise pointer manipulation.

The linked list as shown in the Example code 4-4 is depicted using a figure shown in Figure 4.2 to illustrate the deletion process through pointer adjustments. The memory addresses shown (e.g., 0xF, 0xB, 0xA, 0xD) are assumed for illustration purposes and do not represent actual memory locations.

Initially, the head pointer references the first node (0xF), which links to the node at 0xB (`found_node`). The node at 0xB points to the node to be deleted (0xA), which in turn points to the next node (0xD).

To perform deletion, the next pointer of `found_node` (0xB) is updated from 0xA to 0xD, thereby bypassing the node at 0xA.

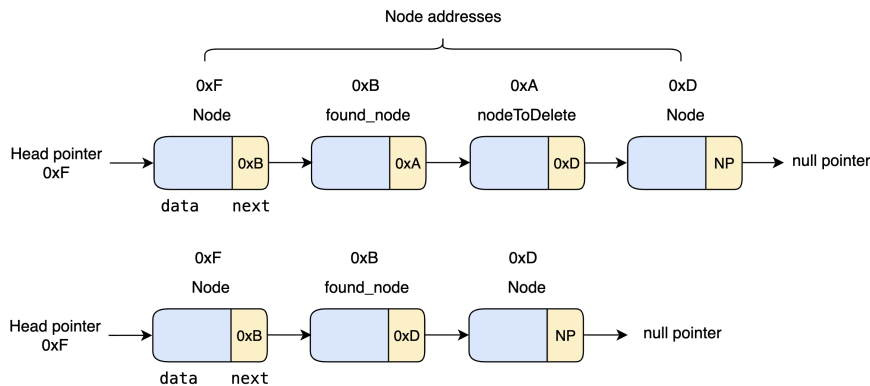


FIGURE 4.2 Deletion operation in a unidirectional linked list

After this pointer reassignment, the node at 0xA becomes unreachable from the head and can be safely deallocated. The resulting list maintains continuity by directly linking 0xB to 0xD.

i Pointer adjustment in unidirectional linked lists

Pointer adjustments are fundamental to the correct functioning of a unidirectional (singly) linked list. During insertion or deletion operations—whether at the beginning, in the middle, or at the end of the list—the manner in which pointers are updated may vary slightly depending on the position of the node involved. However, the underlying principle remains consistent: the integrity of the list is maintained by correctly reassigning the next pointers to preserve connectivity between nodes.

The following table summarizes the time complexity of insertion and deletion operations in a unidirectional (singly) linked list based on the position of the operation.

TABLE 4.1 Time complexity of insertion and deletion in a unidirectional linked list

Operation	Beginning	Middle	End
Insertion	$O(1)$; direct update of the head pointer.	$O(n)$; requires traversal to locate the insertion point.	$O(n)^1$; traversal required unless a tail pointer is maintained.

Deletion	$O(1)$; head pointer is updated.	$O(n)$; requires traversal to locate the predecessor node.	$O(n)$; traversal required to locate the second-last node.
----------	-----------------------------------	---	---

4.1.1 Bidirectional Linked List

A bidirectional linked list, commonly referred to as a doubly linked list, is a linear data structure in which each node maintains two pointers: one referencing the subsequent node and another referencing the preceding node. Formally, a node in a bidirectional linked list contains three components: a data field, a pointer to the next node, and a pointer to the previous node, illustrated in Figure 4.3. This dual-link structure enables traversal of the list in both forward and backward directions, thereby providing greater flexibility compared to a unidirectional (singly) linked list.

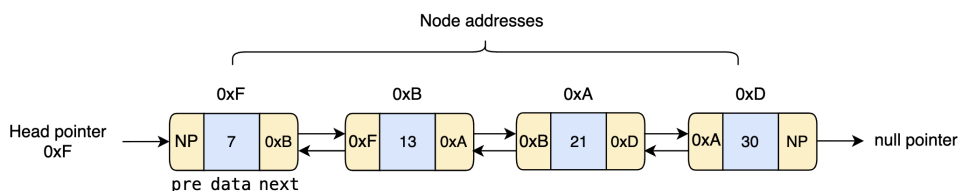


FIGURE 4.3 Bidirectional linked list

The primary advantage lies in its efficient navigation and manipulation capabilities. Since each node has access to both its successor and predecessor, operations such as insertion and deletion can be performed more conveniently, particularly when the position of the target node is already known. For example, deletion of a node does not require traversal to locate its predecessor, as is necessary in singly linked lists; instead, the previous pointer directly provides access to the preceding node. Consequently, certain operations that may require traversal in a singly linked list can be executed more efficiently in a bidirectional structure.

The time complexity of insertion and deletion operations depends on whether the location of the target node is known in advance. When insertion or deletion is performed at an arbitrary position specified by a key or index, a traversal from the head node is typically required to locate the desired position. This traversal incurs a time complexity of $O(n)$, where n denotes the number of nodes in the list. Consequently, the overall time complexity of both insertion and deletion in such cases is dominated by the search phase and is therefore $O(n)$.

¹If a tail pointer is maintained, insertion at the end can be performed in $O(1)$ time. However, deletion at the end still requires traversal to locate the predecessor node in a singly linked list.

Once the appropriate node or position has been identified, the actual insertion or deletion operation can be performed in constant time, $O(1)$, due to the availability of both prev and next pointers. For insertion, this involves adjusting the pointers of the neighboring nodes to correctly incorporate the new node into the sequence. Similarly, deletion requires updating the adjacent nodes so that they bypass the node being removed, followed by deallocation of memory. Despite these constant-time pointer manipulations, the necessity of traversal in general scenarios results in an overall time complexity of $O(n)$.

The Table 4.3 provides a concise summary of the time complexity of insertion and deletion operations in a bidirectional linked list across different positions, consolidating the key results discussed earlier.

TABLE 4.3 Time complexity of insertion and deletion in a bidirectional linked list

Operation	Beginning	Middle	End
Insertion	$O(1)$; direct update of head and backward link.	$O(n)$; requires traversal to locate the insertion point.	$O(n)^1$; traversal required unless a tail pointer is maintained.
Deletion	$O(1)$; head pointer and backward link are updated.	$O(n)^2$; traversal required to locate the target node.	$O(n)^1$; traversal required unless a tail pointer is maintained.

The creation and deletion of nodes in bidirectional linked lists will be discussed in the following sections; these operations are largely analogous to those in unidirectional linked lists, with minor variations.

Creating nodes

The Example code 4-5 illustrates the construction of a bidirectional linked list, demonstrating how nodes are created and subsequently linked through both forward and backward pointers.

Code Example 4-5: Node creation in bidirectional linked list

```

1 struct Node {
2     int data;
3     Node *prev;
4     Node *next;

```

¹If a tail pointer is maintained, both insertion and deletion at the end can be performed in $O(1)$ time.

²If a direct reference to the target node is available, deletion can be performed in $O(1)$ time.

```

5 };
6 int main() {
7     // Create 4 nodes
8     Node *first = new Node{1, nullptr, nullptr};
9     Node *third = new Node{3, nullptr, nullptr};
10    Node *fifth = new Node{5, nullptr, nullptr};
11    Node *seventh = new Node{7, nullptr, nullptr};
12
13    // Head pointer points to the first node
14    Node *head = first;
15    // Link the nodes
16    first->prev = nullptr;
17    first->next = third;
18
19    third->prev = first;
20    third->next = fifth;
21
22    fifth->prev = third;
23    fifth->next = seventh;
24
25    seventh->prev = fifth;
26    seventh->next = nullptr;
27 }

```

Lines 1–4: The code begins with the definition of a Node structure, which contains three members: an integer **data** to store the value, a pointer **prev** to reference the previous node, and a pointer **next** to reference the subsequent node. This structure enables traversal in both forward and backward directions.

Lines 8–11: Within the **main()** function, four nodes are dynamically allocated using the **new** operator. Each node is initialized using uniform initialization syntax, where the first value represents the data stored in the node, and the remaining two values initialize the **prev** and **next** pointers to **nullptr**. Specifically, **Node *first = new Node{1, nullptr, nullptr};** creates the first node with data value 1, and both pointers initially set to null, indicating that it is not yet connected to any other node. Similarly, three additional nodes—**third**, **fifth**, and **seventh**—are created with values 3, 5, and 7, respectively, each initially isolated.

Line 14: The pointer head is then declared and assigned the address of the first node. This establishes the entry point to the linked list, from which traversal operations can begin.

Lines 16–26: The subsequent statements explicitly establish the bidirectional links between the nodes. For the first node, **first->prev = nullptr;** correctly indicates that it has no predecessor, while **first->next = third;** connects it to the second node in the sequence. For the node third, the statement **third->prev = first;** creates a backward link to the first node, and **third->next = fifth;** establishes a forward link to the next node. This pattern continues for the node fifth, where **fifth->prev = third;** and **fifth->next = seventh;** connect it to its neighboring nodes in both directions. Finally, for the last node, **seventh->prev = fifth;** establishes the backward connection, while **seventh->next = nullptr;** indicates the termination of the list in the forward direction. This use of **nullptr** at both ends of the list clearly defines the boundaries of the data structure.

Overall, the code provides a clear and explicit illustration of how a bidirectional linked list is constructed through manual pointer assignments, emphasizing the importance of maintaining both forward and backward links to ensure proper traversal and structural integrity.

Inserting a node

The following Example code illustrates three cases of node insertion: insertion at the beginning of the list, insertion at an arbitrary position, and insertion at the end of the list. The `traverse()` function is reused from Example code 4-2.

Code Example 4-6: Node insertion in bidirectional linked list

```

1 void insert(Node *&head, int search_data, int new_data, bool
  ↪ insert_at_beginning) {
2
3     // Case 1: Insert at the beginning of the list
4     if (insert_at_beginning) {
5         Node *new_node = new Node{new_data, nullptr, nullptr};
6         new_node->next = head;
7         head = new_node;
8     } else {
9         Node *found_node = traverse(head, search_data);
10        if (found_node == nullptr) {
11            cout << "Data not found" << endl;
12            return;
13        }
14        // Case 2: Insert at the end of the list
15        if (found_node->next == nullptr) {
16            Node *new_node = new Node(new_data, found_node, nullptr);
17            found_node->next = new_node;
18        } else {
19            // Case 3: Insert at an arbitrary position
20            Node *new_node = new Node{new_data, found_node,
  ↪ found_node->next};
21            // Fix a backward link of the next node after the new node
22            found_node->next->prev = new_node;
23            // Fix a forward link
24            found_node->next = new_node;
25        }
26    }
27 }

```

Line 1: The function `insert(Node *&head, int search_data, int new_data, bool insert_at_beginning)` implements insertion in a bidirectional (doubly) linked list under three distinct scenarios. The head pointer is passed by reference (`Node *&head`), allowing the function to update the original head of the list when necessary. The parameter `search_data` is used to locate a target node, while `new_data` represents the value to be

stored in the newly created node. The boolean flag `insert_at_beginning` determines whether the insertion should occur at the start of the list.

Lines 4–7: The function begins by evaluating the condition `if (insert_at_beginning)`. When this condition is `true`, a new node is dynamically allocated using `Node *new_node = new Node(new_data, nullptr, nullptr)`, where the node is initialized with the given data and its `prev` and `next` pointers set to `nullptr`. The statement `new_node->next = head`; then establishes a forward link from the new node to the current head of the list. Subsequently, `head = new_node`; updates the `head` pointer so that it now points to the newly inserted node, effectively placing it at the beginning of the list.

Lines 9–12: If the insertion is not at the beginning, the function proceeds to locate the target node using `Node *found_node = traverse(head, search_data)`. This call searches the list for a node containing the value `search_data`. If the target node is not found, indicated by `found_node == nullptr`, the function outputs an error message and terminates early using `return`;

Lines 15–17: If the target node is successfully located, the function next determines whether insertion should occur at the end of the list by checking the condition `if (found_node->next == nullptr)`. If this condition holds, it implies that the located node is the last node. In this case, a new node is created using `Node *new_node = new Node(new_data, found_node, nullptr)`; where the `prev` pointer which is `found_node` is set to the current last node and the `next` pointer is set to `nullptr`. The statement `found_node->next = new_node`; then updates the forward link of the last node to include the new node, thereby extending the list.

Lines 19–24: In the alternative case, where the target node is neither at the beginning nor at the end, the function performs insertion at an arbitrary position. A new node is created using `Node *new_node = new Node(new_data, found_node, found_node->next)`; where the `prev` pointer is set to the located node and the `next` pointer is set to its successor. The statement `found_node->next->prev = new_node`; updates the backward link of the successor node so that it now points to the newly inserted node. Finally, `found_node->next = new_node`; updates the forward link of the located node to complete the insertion.

Overall, the function demonstrates how insertion in a bidirectional linked list is achieved through careful manipulation of both forward and backward pointers, ensuring that the structural integrity of the list is preserved in all cases.

Deleting a node

The following Example code illustrates three cases of node deletion: deleting a node at the beginning of the list, at an arbitrary position, and at the end of the list. The `traverse()` function is reused from Example code 4-2.

Code Example 4-7: Node deletion in bidirectional linked list

```
1 void deleteNode(Node *&head, int search_data) {
2     Node *ptr_search_data = traverse(head, search_data);
3     if (ptr_search_data == nullptr) {
4         cout << "Data not found" << endl;
5         return;
```

```

6     }
7     // Case 1: Deleting the node at the beginning of the list
8     if (ptr_search_data->prev == nullptr) {
9         head = ptr_search_data->next;
10        head->prev = nullptr;
11        delete ptr_search_data;
12    }
13    // Case 2: Deleting the node at the end of the list
14    else if (ptr_search_data->next == nullptr) {
15        ptr_search_data->prev->next = nullptr;
16        delete ptr_search_data;
17    } else {
18        // Case 3: Deleting the node at an arbitrary position
19        ptr_search_data->next->prev = ptr_search_data->prev;
20        ptr_search_data->prev->next = ptr_search_data->next;
21        delete ptr_search_data;
22    }
23 }

```

Line 1: The function `deleteNode(Node *&head, int search_data)` is designed to remove a node containing a specified value from a bidirectional linked list. The parameter `head` is passed by reference so that the function can modify the head pointer itself when necessary, particularly if the node to be deleted is the first node in the list. The parameter `search_data` represents the value of the node that is intended to be removed.

Line 2: The statement `Node *ptr_search_data = traverse(head, search_data);` calls the `traverse()` function to search the linked list for the node containing `search_data`. If such a node is found, the function returns a pointer to that node, which is stored in `ptr_search_data`. If the value is not found, the function returns `nullptr`. Thus, this line performs the search operation and stores the address of the target node for subsequent deletion.

Line 3: The next conditional statement, `if (ptr_search_data == nullptr)`, checks whether the search was unsuccessful. If the pointer is `null`, it means that no node in the list contains the requested value. In that case, the statement `cout << "Data not found" << endl;` displays an error message to indicate that deletion cannot be performed. The subsequent `return;` statement immediately terminates the function, thereby preventing any attempt to access an invalid pointer.

Lines 7–12: The first deletion case is handled by the condition `if (ptr_search_data->prev == nullptr)`. In a bidirectional linked list, if the `prev` pointer of a node is null, then that node must be the first node in the list. The statement `head = ptr_search_data->next;` updates the head pointer so that it now points to the second node, which becomes the new first node after deletion. The next statement, `head->prev = nullptr;`, resets the backward pointer of the new head node to `null`, thereby preserving the structural property that the first node has no predecessor. Finally, `delete ptr_search_data;` deallocates the memory occupied by the removed node.

Lines 13–17: The second case is expressed by `else if (ptr_search_data->next == nullptr)`. If the next pointer of the target node is `null`, then the node is the last node in the

list. The statement `ptr_search_data->prev->next = nullptr;` updates the next pointer of the preceding node so that it becomes the new last node. This effectively disconnects the original last node from the list. The statement `delete ptr_search_data;` then frees the memory associated with the deleted node.

Lines 17–22: The final case is handled by the `else` block, which corresponds to deletion at an arbitrary position in the middle of the list. In this situation, the node to be deleted has both a predecessor and a successor. The statement `ptr_search_data->next->prev = ptr_search_data->prev;` updates the `prev` pointer of the next node so that it points back to the node preceding the one being deleted. Then, the statement `ptr_search_data->prev->next = ptr_search_data->next;` updates the next pointer of the previous node so that it points forward to the node following the one being deleted. Together, these two pointer updates bypass the target node and reconnect the remaining parts of the list. The statement `delete ptr_search_data;` then releases the memory occupied by the removed node.

Overall, the above code demonstrates the three principal cases involved in deleting a node from a bidirectional linked list: deletion at the beginning, deletion at the end, and deletion at an intermediate position. In each case, the surrounding pointers are adjusted to preserve the continuity of the list before the target node is deallocated.

One important practical note, however, is that the first case assumes that the list contains more than one node. If the list contains only a single node, then after the statement `head = ptr_search_data->next;`, the head becomes `nullptr`, and the subsequent statement `head->prev = nullptr;` would be invalid. Therefore, an additional check would be needed to make the implementation fully robust.

4.1.2 Unidirectional Circular Linked List

A unidirectional circular linked list is a variant of the singly linked list in which each node contains a data field and a single pointer that references the next node in the sequence, but unlike a standard linear list, the final node does not point to a null reference. Instead, the last node links back to the first node, thereby forming a closed loop. Formally, if `head` denotes the first node, then the next pointer of the last node satisfies `last → next = head`. This circular structure eliminates the notion of a terminal node and allows traversal to continue indefinitely by cycling through the list.

One of the primary characteristics of a unidirectional circular linked list is that traversal can begin at any node and still visit all elements, eventually returning to the starting point. This property is particularly useful in applications that require repetitive or cyclic processing, such as round-robin scheduling or buffering systems. Unlike a linear singly linked list, a circular linked list does not contain a `nullptr` to denote the end of the structure; instead, the final node links back to the head, forming a closed loop. As a result, traversal must be carefully controlled to avoid infinite iteration, and a stopping condition is typically defined by detecting when the traversal has returned to the starting node.

Insertion and deletion operations in a unidirectional circular linked list follow princi-

ples similar to those of a singly linked list but require careful handling of the circular linkage. In particular, operations at the beginning or end of the list necessitate updating the pointer of the last node to ensure that the final node continues to reference the head, thereby preserving the circular structure. If a direct reference to the last node is maintained, insertion at both the beginning and end can be performed in $O(1)$ time; otherwise, traversal is required, resulting in a time complexity of $O(n)$. Deletion operations similarly require precise pointer adjustments, especially when removing boundary nodes such as the head or the last node.

From an asymptotic standpoint, the use of a circular structure does not alter the time complexity of fundamental operations as shown in Table 4.1. Traversal and search remain $O(n)$, and insertion or deletion at arbitrary positions also incur $O(n)$ due to the need for traversal. The primary advantage of the circular linked list lies not in improved time complexity, but in its ability to represent cyclic data without relying on a nullptr, enabling seamless and continuous traversal. Consequently, while the structure introduces additional considerations in pointer management, it retains the same Big-O performance characteristics as a standard singly linked list.

```
first->next = third;
third->next = fifth;
fifth->next = seventh;
seventh->next = head;
```

The nodes are linked sequentially using the statements `first->next = third;`, `third->next = fifth;`, and `fifth->next = seventh;`, thereby establishing the forward connections within the list. To complete the circular structure, the final node is linked using `seventh->next = head;`.

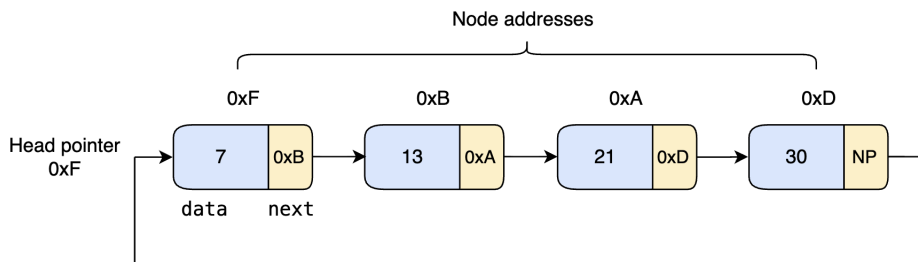


FIGURE 4.4 Circular unidirectional linked list

i Maintaining circular integrity via the head pointer

It is important to note that the last node should connect to the head pointer rather than to a specific node instance (such as `first`). This design choice ensures that the structural integrity of the circular linked list is preserved even if the head

pointer is later modified—for example, during an insertion at the beginning of the list. If the linkage were made to a fixed node (e.g., first), any subsequent update to the head would break the circular property. By consistently referencing the head pointer, the implementation remains robust, semantically accurate, and adaptable to dynamic changes in the list structure.

4.1.3 Bidirectional Circular Linked List

A bidirectional circular linked list, also referred to as a doubly circular linked list, is a dynamic linear data structure in which each node contains three components: a data field, a pointer to the next node, and a pointer to the previous node. Unlike a standard doubly linked list, the last node in a bidirectional circular linked list does not point to a null reference; instead, its next pointer refers back to the first node, and the first node's previous pointer refers to the last node. This circular linkage eliminates terminal boundaries and allows traversal to proceed indefinitely in both forward and backward directions. As illustrated in the Figure 4.5, each node maintains both forward and backward references, while the first and last nodes are interconnected to preserve the circular structure. As a result, the structure is particularly well suited for applications requiring cyclic iteration, such as round-robin scheduling, navigation systems, and buffer management.

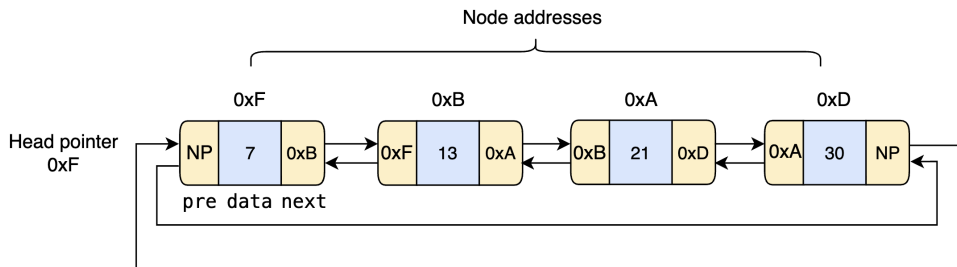


FIGURE 4.5 Circular bidirectional linked list

Insertion operations in a bidirectional circular linked list require careful adjustment of both next and previous pointers to maintain the integrity of the circular structure. Insertion at the beginning involves creating a new node and updating four references: the new node's next pointer is set to the current head, its previous pointer is set to the last node, the previous pointer of the current head is updated to the new node, and the next pointer of the last node is updated to the new node; finally, the head pointer is reassigned to the new node. If a direct reference to the last node (tail) is maintained, this operation can be performed in constant time, i.e., $O(1)$. Similarly, insertion at the end also takes $O(1)$ time with a tail pointer, as it involves linking the new node between the last node and the head while preserving circularity. However, insertion at an arbitrary position requires traversal of the list to locate the appropriate node, resulting in a time complexity of $O(n)$.

Deletion operations follow analogous principles and also require updating both forward and backward links. Deleting the first node involves reassigning the head pointer to the second node, updating the new head's previous pointer to the last node, and modifying the last node's next pointer to point to the new head. With a maintained tail pointer, this operation can also be completed in $O(1)$ time. Deletion at the end similarly takes $O(1)$ time when the tail pointer is available, as it requires updating the second-to-last node's next pointer to the head and adjusting the head's previous pointer accordingly. In contrast, deletion of a node at an arbitrary position necessitates traversal to locate the node, followed by pointer adjustments to bypass it, yielding a time complexity of $O(n)$.

In terms of asymptotic analysis, the bidirectional circular linked list exhibits time complexities comparable to those of standard doubly and singly linked lists. Traversal and search operations require $O(n)$ time, as each node may need to be visited in the worst case. Insertions and deletions at known positions (such as the beginning or end, given appropriate pointers) can be performed in $O(1)$ time, while operations at arbitrary positions incur $O(n)$ due to traversal. Thus, while the circular and bidirectional properties enhance flexibility and eliminate boundary conditions, they do not fundamentally alter the overall time complexity characteristics of the data structure.

The bidirectional circular linkage can be established by explicitly updating both the forward (next) and backward (prev) pointers, as illustrated in the following code segment.

```
first->next = third;
third->prev = first;

third->next = fifth;
fifth->prev = third;

fifth->next = seventh;
seventh->prev = fifth;

seventh->next = head;
head->prev = seventh;
```

The nodes are linked sequentially using the statements `first->next = third;`, `third->next = fifth;`, and `fifth->next = seventh;`, thereby establishing the forward connections within the list. To introduce bidirectional traversal, corresponding backward links are created using `third->prev = first;`, `fifth->prev = third;`, and `seventh->prev = fifth;`. Finally, to maintain the circular structure, the last node is connected back to the head using `seventh->next = head;`, and the head node's backward link is updated using `head->prev = seventh;`, completing the bidirectional circular linkage.

4.2 Linked Lists Using Smart Pointers

4.3 Linked Lists Using the STL

Standard Template
Library

A linked list in the C++ Standard Library is implemented through the container classes `std::list` and `std::forward_list`. The former represents a doubly linked list, while the latter provides a singly linked list implementation. The Standard Template Library (STL) is a part of the C++ Standard Library that offers generic, reusable components—such as containers, algorithms, and iterators—for efficient data storage and manipulation. In these structures, elements are stored in dynamically allocated nodes connected through pointers, rather than in contiguous memory locations.

In `std::list`, each node maintains links to both its previous and next elements, enabling bidirectional traversal. In contrast, `std::forward_list` maintains only a forward link to the next node, resulting in a more memory-efficient structure but with limited traversal capability. Both containers abstract away the complexity of manual pointer manipulation, allowing programmers to avoid explicit memory management and focus on higher-level logic. This abstraction enhances safety, reduces errors, and aligns with modern C++ design principles.

From a computational perspective, both `std::list` and `std::forward_list` provide constant-time, $O(1)$, insertion and deletion when an iterator to the position is available. However, element access requires sequential traversal, leading to $O(n)$ time complexity. As a result, neither container supports direct indexing, distinguishing them from random-access containers such as vectors.

The container `std::list` supports bidirectional iterators and offers a rich set of member functions, including `push_front`, `push_back`, `insert`, `erase`, `remove`, `sort`, and `splice`. Notably, the `splice` operation enables efficient transfer of elements between lists by relinking nodes without copying data. On the other hand, `std::forward_list` provides a more limited interface tailored to forward traversal, including operations such as `push_front`, `insert_after`, and `erase_after`.

The following example illustrates the basic usage of `std::list`:

Code Example 4-8: Basic usage of linked list using `std::list`

```

1 #include <iostream>
2 #include <list>
3 using namespace std;
4
5 int main() {
6     list<int> numbers;
7
8     // Insertion operations
9     numbers.push_back(10); // insert at end
10    numbers.push_back(20);

```

```

11     numbers.push_front(5);    // insert at beginning
12
13     // Traversal
14     cout << "List elements: ";
15     for (int x : numbers) {
16         cout << x << " ";
17     }
18
19     // Insert at a specific position
20     auto it = numbers.begin();
21     it++; // move iterator to second position
22     numbers.insert(it, 15);
23
24     // Deletion
25     numbers.remove(20); // remove value 20
26
27     cout << "\nUpdated list: ";
28     for (int x : numbers) {
29         cout << x << " ";
30     }
31
32     return 0;
33 }

```

The common member functions of `std::forward_list` and `std::list` are summarized in Appendix~8.1.

Circular linked List and STL limitation

The container `std::list` implements a doubly linked list and does not support circular linking. Its design uses a special `end()` iterator instead of connecting the last node back to the first, ensuring clear traversal termination. A true circular linked list forms a loop with no natural endpoint, which conflicts with STL iterator semantics. Therefore, circular behavior must be implemented manually if required.

4.3.1 Operations Unique to Linked Lists in the STL

Specialized member functions in linked-list-based containers such as `std::list` and `std::forward_list` warrant careful study, as they expose capabilities that are largely unique to linked-list structures within the STL. Operations such as `splICE()`, `merge()`, and `unique()` exploit the pointer-based organization of nodes, enabling efficient structural modifications without the need for element copying or shifting—an approach that is not feasible in contiguous containers such as `std::vector`.

These operations are considered unique because linked lists store elements as independently allocated nodes connected through pointers, allowing transformations to

be performed via relinking of nodes. In contrast, contiguous containers rely on sequential memory layouts, where analogous operations necessitate data movement or reallocation, thereby incurring additional computational overhead. Consequently, the study of these member functions provides valuable insight into the distinctive performance characteristics of linked lists and clarifies the contexts in which they offer a meaningful advantage in practical applications.

The `splice()` operation

The `splice()` operation in `std::list` is a specialized member function that transfers elements from one list to another by relinking nodes rather than copying them. Conceptually, instead of creating new nodes and copying values, `splice()` simply adjusts the internal pointers so that the existing nodes are detached from one list and attached to another. As a result, the operation is extremely efficient and runs in constant time $O(1)$ for single-element transfers (and linear time only when transferring a range due to traversal, not copying).

This behavior is possible because `std::list` is implemented as a doubly linked list, where each node maintains pointers to its neighboring nodes. The `splice()` function manipulates these pointers to “cut” a node (or a sequence of nodes) from one list and “paste” it into another location, without allocating new memory or invoking copy/move constructors.

There are three common forms of `splice()` as illustrated in Table~4.5.

TABLE 4.5 Variants of `splice()` operation in `std::list`

Description	Example
Transfers all elements from <code>lst2</code> into the current list at position <code>pos</code> . After the operation, <code>lst2</code> becomes empty.	<code>lst1.splice(pos, lst2);</code>
Transfers a single element pointed to by <code>it</code> from <code>lst2</code> into the current list at position <code>pos</code> .	<code>lst1.splice(pos, lst2, it);</code>
Transfers a range of elements from <code>lst2</code> (from <code>first</code> to <code>last</code>) into the current list at position <code>pos</code> .	<code>lst1.splice(pos, lst2, first, last);</code>

The `merge()` operation

The `merge()` operation in `std::list` is a specialized member function used to combine two sorted lists into a single sorted list. Unlike typical merging approaches that involve copying elements into a new container, `merge()` operates by relinking nodes

from one list into another. Consequently, no element copying or memory reallocation occurs, making the operation highly efficient.

A key requirement of `merge()` is that both lists must already be sorted according to the same comparison criterion. During the merge process, elements from the second list are compared and inserted into appropriate positions in the first list while preserving sorted order. Importantly, duplicate elements are allowed; if both lists contain equal values, all occurrences are retained in the final merged list. After the operation, the source list becomes empty, as all its nodes are transferred.

Code Example 4-9: the `merge()` operation

```

1 #include <iostream>
2 #include <list>
3 using namespace std;
4
5 int main() {
6     list<int> lst1 = {1, 3, 5};
7     list<int> lst2 = {1, 3, 4, 6};
8
9     lst1.merge(lst2);
10
11     for (int x : lst1)
12         cout << x << " ";
13 }
```

In this example, both lists are sorted in ascending order. The `merge()` function interleaves elements from `lst2` into `lst1` by adjusting node pointers. Notice that duplicate values (e.g., 1 and 3) are preserved, demonstrating that `merge()` does not eliminate duplicates.

The `merge()` operation can also be performed in descending order, provided that both lists are already sorted in descending order. In such cases, the `merge()` function is invoked with an appropriate comparison function as follows.

```
lst1.merge(lst2, greater<int>());
```

where, the expression `greater<int>()` is an instantiation of the function object template `greater` defined in the C++ Standard Library header `<functional>`. This template class provides a comparison mechanism by overloading the function-call operator `operator()`, thereby allowing objects of this type to behave like functions. Specifically, `greater<int>` defines a binary predicate that takes two arguments of type `int` and returns true if the first argument is greater than the second. The syntax `greater<int>()` creates an instance of this functor, which can then be passed to algorithms or container member functions (such as `merge()` or `sort()`) to specify a custom ordering. This design reflects the generic programming paradigm of the

STL, where behavior—such as comparison logic—is parameterized through callable objects, enabling flexibility and extensibility beyond fixed ordering schemes.

The `unique()` operation

The `unique()` operation in `std::list` is a specialized member function used to remove consecutive duplicate elements from a linked list. Unlike general duplicate removal techniques, `unique()` only **eliminates adjacent equal elements**, meaning that the list must be appropriately arranged (typically sorted) if the goal is to remove all duplicates.

Internally, `unique()` operates by traversing the list and relinking nodes, removing duplicates without shifting elements or allocating new memory. This makes it efficient, as deletions are performed through pointer adjustments rather than data movement.

Code Example 4-10: the `unique()` operation

```

1 #include <iostream>
2 #include <list>
3 using namespace std;
4
5 int main() {
6     list<int> nums = {1, 1, 2, 3, 3, 3, 4, 4, 5};
7
8     nums.unique();
9
10    for (int x : nums)
11        cout << x << " ";
12 }
```

In this example, only consecutive duplicates are removed. Since identical values are adjacent, all duplicates are eliminated effectively. The output comes out to be **1 2 3 4 5**.

The following table presents a comparative analysis of `std::list` and `std::vector`, highlighting their structural characteristics, performance trade-offs, and typical use cases in modern C++ programming.

TABLE 4.7 Comparison between `std::list` (linked list) and `std::vector`

Feature	<code>std::list</code> (Linked List)	<code>std::vector</code>
Memory Layout	Non-contiguous; elements stored as separate nodes linked via pointers.	Contiguous; elements stored in adjacent memory locations.
Access Time	Sequential access only; $O(n)$.	Random access supported; $O(1)$.

Insertion/Deletion	Efficient ($O(1)$) with iterator; no shifting required.	Costly ($O(n)$); elements must be shifted.
Memory Overhead	Higher due to additional pointer storage (prev/next).	Lower; no extra pointer overhead.
Cache Performance	Poor cache locality due to non-contiguous storage.	Good cache locality due to contiguous memory.
Iterator Stability	Stable; insertions/deletions do not invalidate iterators.	May invalidate iterators due to reallocation.
Traversal	Bidirectional traversal supported.	Forward traversal (and random access) supported.
Sorting	Provides member function <code>sort()</code> .	Uses generic <code>std::sort()</code> algorithm.
Special Operations	Supports <code>splice()</code> , <code>merge()</code> , <code>unique()</code> .	No equivalent operations without copying data.
Use Case	Frequent insertions/deletions in the middle.	Frequent access and read-heavy operations.

4.4 Applications of Linked Lists

Linked lists are widely used in practical computing scenarios where dynamic memory management and efficient insertion or deletion operations are required. One common application is in the implementation of **dynamic data structures** such as stacks, queues, and deques, where elements are frequently added or removed without the need for contiguous memory allocation. Unlike arrays, linked lists allow these operations to be performed without shifting elements, making them more efficient in such contexts.

Another important application is in **memory management systems**, particularly in the maintenance of free lists. Operating systems often use linked lists to keep track of available memory blocks, enabling efficient allocation and deallocation during program execution.

Similarly, linked lists are used in **file systems**, where blocks of data may not be stored contiguously on disk; instead, pointers are used to link scattered blocks together.

In **hash tables**, linked lists are employed in separate chaining to handle collisions. When multiple keys hash to the same index, they are stored in a linked list at that location, allowing efficient storage and retrieval even in the presence of collisions. This is a common technique in many programming languages and libraries.

Linked lists are also useful in **graph representations**, particularly in adjacency list representations, where each vertex maintains a linked list of its neighboring vertices. This approach is memory-efficient compared to adjacency matrices, especially for sparse graphs.

Additionally, linked lists are used in applications requiring **bidirectional traversal**, such as navigation systems (e.g., undo/redo functionality in text editors or browser history), where a doubly linked list allows movement both forward and backward through a sequence of states.

Overall, linked lists provide flexibility and efficiency in scenarios where the size of the data structure changes frequently, or where insertions and deletions are more common than random access operations.

4.4.1 *Limitations of Linked Lists*

Linked lists, while flexible and efficient for certain operations, are not ideal in scenarios where fast random access, memory efficiency, or cache performance is critical. One primary limitation is that linked lists do not support **direct (indexed) access** to elements. Accessing the i -th element requires sequential traversal from the head, resulting in a time complexity of $O(n)$. In contrast, arrays provide $O(1)$ access, making them more suitable for applications that frequently access elements by index, such as numerical computing or matrix operations.

Linked lists are also not ideal in **memory-constrained environments**. Each node requires additional storage for pointer(s) (e.g., next and possibly previous), which increases memory overhead compared to arrays that store only the data. Furthermore, because nodes are allocated non-contiguously in memory, linked lists suffer from poor cache locality. Modern processors rely heavily on caching mechanisms, and contiguous structures like arrays benefit significantly from spatial locality, leading to faster access times.

Another limitation arises in applications that require **frequent searching or sorting**. Since linked lists lack efficient random access, algorithms such as binary search cannot be applied directly, and even simple searches require linear time. Sorting algorithms that rely on indexing are also less efficient on linked lists, although specialized algorithms like merge sort can be adapted.

Linked lists are also less suitable when **predictable performance and simplicity** are required. Pointer manipulation introduces additional complexity and increases the likelihood of programming errors, particularly when using raw pointers. This makes them less ideal in safety-critical systems unless managed carefully (e.g., with smart pointers).

Finally, linked lists are not well suited for applications that require **compact data representation and minimal** overhead, such as embedded systems or high-performance numerical applications. In such cases, array-based structures or vectors are generally preferred due to their simplicity, lower memory usage, and superior performance.

characteristics.

In general, linked lists are not ideal when fast indexed access, memory efficiency, cache performance, or algorithmic simplicity are primary requirements.

Application: LRU Cache Using a Doubly Linked List

A practical and widely adopted application of linked lists is the implementation of a Least Recently Used (LRU) cache, a data structure designed to manage limited storage efficiently by discarding the least recently accessed elements. LRU caches are commonly employed in operating systems, database systems, and web browsers to optimize memory usage and improve access performance.

An LRU cache operates under the principle that elements accessed recently are more likely to be used again, whereas elements that have not been accessed for a long time are candidates for removal. The cache is constrained by a fixed capacity; once this capacity is exceeded, the least recently used element must be evicted.

To achieve efficient performance, the LRU cache combines two data structures: a doubly linked list and a hash table. The doubly linked list maintains the order of usage, where the most recently used element is positioned at the front (head), and the least recently used element is positioned at the rear (tail). The hash table provides constant-time access to elements based on their keys. This hybrid design ensures that both retrieval and update operations can be performed in constant time, i.e., $O(1)$.

In this structure, each node of the doubly linked list stores a key-value pair along with two pointers: one pointing to the next node and another pointing to the previous node. When an element is accessed, its corresponding node is moved to the front of the list, reflecting its recent usage. When a new element is inserted, it is also placed at the front. If the cache has reached its maximum capacity, the node at the tail of the list—representing the least recently used element—is removed. The presence of both forward and backward pointers allows these operations to be performed efficiently without requiring traversal of the list.

Concept Check

■ Why can the tail address in a forward linked list not be maintained to enable deletion at the end in $O(1)$ time?

Maintaining a tail pointer in a singly linked list allows direct access to the last node, but it does not make deletion at the end a constant-time operation. This is because deletion of the last node requires updating the pointer of the preceding (second-to-last) node so that it points to null. In a singly linked list, each node contains only a reference to its next node and does not store any information about its predecessor. As a result, even if the tail pointer is available, there is no direct way to identify the node before the tail.

To locate the preceding node, it is necessary to traverse the list from the head until the penultimate node is reached. This traversal takes linear time, $O(n)$. Therefore, the presence of a tail pointer improves access to the last node but does not eliminate the need for traversal when deleting it. Consequently, deletion at the end in a singly linked list remains an $O(n)$ operation.

■ Is a Python list implemented using a linked list internally?

No, a Python list does not use a linked list under the hood.

Instead, it is implemented as a dynamic array, meaning elements are stored in contiguous memory locations. This design allows Python lists to provide constant-time indexing and efficient iteration.

5

Stacks & Queues

6

Hash Tables

Hash Tables	114
Principles of Hash Function Design	117
Principles of Compression Function Design	128
Hash Collisions: Inevitability and Impact	133
Collision Resolution Strategies	136
Hash Table Implementations	144
Hash Table Applications	154

In data structures, a dynamic environment refers to a setting in which the dataset is frequently changing due to insertions, deletions, or updates, and where the size or ordering of the data may not remain fixed over time. Unlike static environments—where data remains relatively stable and preprocessed—dynamic environments require algorithms that can efficiently adapt to continuous modifications. Comparison-based searching techniques such as linear search and binary search have significant limitations in such contexts. Linear search, while simple and adaptable, becomes inefficient as data grows because each search may require scanning many elements $O(n)$, which is costly when searches are frequent. Binary search improves search time to $O(\log n)$, but it requires the data to remain sorted; maintaining sorted order in a dynamic dataset introduces additional overhead. Insertions and deletions in sorted arrays often require shifting elements $O(n)$, making updates expensive and reducing overall efficiency. Furthermore, comparison-based methods rely solely on element-to-element comparisons, limiting their scalability when rapid, real-time lookups are needed. The main challenges in dynamic environments therefore include maintaining performance under frequent updates, balancing search and modification costs, preserving structural invariants (such as sorted order), and ensuring scalability as the dataset grows. These limitations motivate the use of alternative structures such as hash tables and balanced trees, which are specifically designed to handle dynamic data more efficiently.

6.1 Hash Tables

A hash table is a data structure that stores data as key–value pairs and provides very fast average-case performance for insertion, deletion, and search operations. It works by using a hash function to transform a key (such as a string, integer, or object) into an integer index that corresponds to a position in an underlying array. Instead of scanning through elements one by one (as in linear search) or relying on sorted order (as in binary search), a hash table computes where the data should be stored and accesses it directly. This is why hash tables typically achieve $O(1)$ average time complexity for lookup, insertion, and deletion.

Before proceeding with the discussion of hash tables, it is important to clarify the scope and underlying assumptions of this chapter. First, the hash tables considered here are intended for use in data structures and algorithm design, rather than for cryptographic or security purposes. Second, the discussion assumes that all data is stored in main memory (internal storage), and does not address hash table implementations designed for external storage systems such as disk-based databases.

Hash Tables in Modern Programming Languages

Hash tables are referred to by different names across programming languages—for example, **HashMap** in Rust and Java; **std::unordered_map** in C++; **dict** in Python; **map** in Go, as few examples.

Figure 6.1 illustrates how hash tables store data in memory using an underlying array as the primary storage structure. This array consists of a fixed number of slots, commonly referred to as buckets, and each bucket corresponds to an index derived through a two-step process involving a hash function and a compression function. When a key is inserted, the hash function first transforms the key into a large integer value using a specified hashing algorithm. This integer hash value is then passed to the compression function, which maps it into a valid array index within the bounds of the underlying array. Once the index is computed, the key–value pair is stored at the corresponding location in the contiguous array structure, conceptually labeled as $A_1, A_2, A_3, \dots, A_n$. The colored cells in the figure visually represent how different keys are mapped to different buckets. Importantly, this placement is determined by the computed hash and compression processes rather than by the order of insertion, thereby enabling efficient and direct access to stored elements.

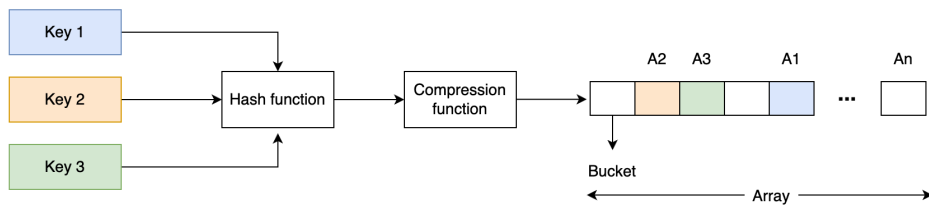


FIGURE 6.1 Hash Table

The following example illustrates how a key is mapped to an index within an array. Its purpose is not to present the full theoretical framework of hashing, but rather to demonstrate the basic operational mechanism of a hash implementation and provide the reader with an initial intuitive understanding. Hashing is a conceptually rich and mathematically nuanced topic; therefore, many of its deeper properties—such as collision behavior, distribution quality, robustness considerations, and performance analysis—will be examined systematically in the subsequent sections.

Suppose, the key is “CAT”. The ASCII value of CAT is 67, 65 and 84 respectively. For simplicity, a polynomial rolling hash function is used to compute the index. A polynomial rolling hash is a string hashing technique that treats a string like a polynomial and evaluates it at a chosen base number to produce a numeric hash value. It is called “polynomial” because each character contributes to the hash as a term in a polynomial expression, and “rolling” because the hash can be efficiently updated when sliding over substrings (commonly used in algorithms like Rabin–Karp). The hash function will be discussed further in next section.

The hash can be calculated using the following equation.

$$\text{hash}(s) = c_1 \cdot p^{n-1} + c_2 \cdot p^{n-2} + \dots + c_n \cdot p^0 \quad (6.1)$$

where n is the number of characters in the string, s is the input string of length n , c_i is the numeric value (e.g., ASCII or Unicode) of the i^{th} character, p is a chosen base (commonly a small prime such as 31 or 37), p^k is the positional weight assigned to each character and $\text{hash}(s)$ is the resulting integer hash value.

Use Equation 6.1, the hash value of CAT can be computed as follows:

$$\begin{aligned}\text{hash}(\text{CAT}) &= c_1 \times 31^2 + c_2 \times 31^1 + c_3 \times 31^0 \\ &= 67 \times 31^2 + 65 \times 31^1 + 84 \times 31^0 \\ &= 66486\end{aligned}$$

The prime number used in a polynomial rolling hash is a deliberate design choice intended to promote uniform distribution of hash values and mitigate structured collisions. Its selection is grounded in number-theoretic principles and empirical performance considerations rather than being derived from the input string itself. For example, Java's `String.hashCode()` employs a polynomial hash with a base of 31, an odd prime chosen to balance computational efficiency and distribution quality [2]. Similarly, in algorithmic and competitive programming contexts, primes such as 31 or 53 are commonly recommended, with the specific choice often influenced by the size of the character set. These primes help achieve effective dispersion of hash values while maintaining efficient arithmetic operations.

The next step is to apply a compression function to the computed hash value in order to obtain a valid array index. Lets assume that the division method is used.

$$\text{index} = \text{hash}(\text{key}) \bmod \text{array_length} \quad (6.2)$$

where `array_length` is the total number of buckets in the hash table and the `mod` is the modulo operator (returns the remainder). The modulo operation guarantees that $0 \leq \text{index} < \text{array_length}$, ensuring that the computed index always falls within the valid bounds of the underlying array. When a hash table is created, its implementation allocates a contiguous array consisting of x buckets. Each bucket represents a storage slot within the table. The value of x is implementation-dependent and varies across programming languages and standard library implementations.

Terminology Clarification

In many textbooks, the term `table length` is used to denote the number of buckets in a hash table, which corresponds directly to the length of the underlying array. For the sake of clarity and terminological uniformity, we will use the term `array length` throughout this discussion.

For this example, suppose that the array contains 16 buckets (i.e., `array_length = 16`) and that the previously computed hash value of CAT is 66486. By applying Equation

6.2, we calculate $66486 \bmod 16$, which produces an index value of 6. This index identifies the specific bucket within the underlying array where the associated key–value pair is stored.

In most hash table implementations, keys are treated as case-sensitive, particularly when the key type is a string. Consequently, the strings “CAT” and “cat” are considered distinct keys because their character encodings differ at the byte level. It is important to note that case sensitivity is not an inherent property of the hash table data structure itself; rather, it is determined by the key’s data type and the equality comparison mechanism defined by the programming language or implementation.

The above example illustrates the basic idea behind hash and compression functions in simple terms. In the next section, we will examine their design in greater depth and explore different approaches to constructing hash and compression functions.

6.2 Principles of Hash Function Design

In this section, we examine hash functions as they are used in hash tables for efficient data storage and retrieval. The emphasis is on properties such as uniform key distribution, minimization of collisions, and computational efficiency, all of which directly influence the average-case performance of fundamental operations.

This discussion does not address cryptographic hash functions (e.g., SHA-256), which are designed to satisfy security-oriented properties such as strong collision resistance, preimage resistance, and one-way behavior.

6.2.1 *Deterministic mapping*

A hash function used in a hash table is formally defined as

$$h : \mathcal{K} \rightarrow \mathbb{Z} \quad (6.3)$$

Here, $\mathcal{K}$ denotes the set of keys and $\mathbb{Z}$ denotes the set of integers. By the definition of a mathematical function, each key $k \in \mathcal{K}$ is associated with a unique integer $h(k) \in \mathbb{Z}$. Therefore, the mapping defined in Equation 6.3 is deterministic: repeated evaluations of h on the same key must produce the same integer value. Formally, using the universal quantifier symbol $\forall$ (read as “for all”),

$$\forall k \in \mathcal{K}, \quad h(k) \text{ is fixed} \quad (6.4)$$

The necessity of determinism follows from the operational behavior of a hash table. During insertion at time t_1 , the storage location of a key is computed from $h(k)$. During lookup at time t_2 , the same computation is performed. Correct retrieval therefore requires

$$h_{t_1}(k) = h_{t_2}(k) \quad (6.5)$$

Equation 6.5 follows directly from the definition of the function in Equation 6.3. If the hash function were allowed to vary between evaluations, the computed bucket during lookup could differ from the bucket used during insertion, thereby violating correctness.

Determinism is also essential for referential consistency during structural operations such as table resizing. When the table capacity changes from m to $\bar{m}$, only the compression function is modified. The underlying hash value defined in Equation 6.4 must remain invariant so that reinsertion preserves the identity of each key.

Therefore, determinism is a fundamental requirement for correctness, stability, and referential consistency in hash table implementations.

6.2.2 Uniform Distribution

The performance of a hash table depends fundamentally on how evenly keys are distributed across its m buckets. Let

$$h : \mathcal{K} \rightarrow \{0, 1, \dots, m-1\} \quad (6.6)$$

be a hash function that maps keys from the universe $\mathcal{K}$ into an array of size m . The set $\{0, 1, \dots, m-1\}$ represents all valid bucket indices of the table. For efficient performance, it is desirable that each bucket be equally likely to receive any given key. Formally, this requirement is expressed as

$$\Pr[h(k) = i] = \frac{1}{m}, \quad \forall i \in \{0, 1, \dots, m-1\}. \quad (6.7)$$

The Equation 6.7 states that the probability that a key k is mapped to a specific bucket i is exactly $1/m$. Since there are m buckets, each bucket receives an equal fraction of the total probability mass. Consequently, the probabilities sum to one.

$$\sum_{i=0}^{m-1} \frac{1}{m} = 1 \quad (6.8)$$

This uniformity condition forms the basis of the analytical model known as Simple Uniform Hashing (SUH), which assumes that each key is hashed independently and uniformly at random into one of the m buckets. Although practical hash functions are deterministic, SUH provides a probabilistic abstraction that enables rigorous average-case analysis.

Simple uniform
hash

Suppose n keys are inserted into the table. For a fixed bucket i , define the random variable, X_i to represent the number of keys that are mapped to bucket i by the hash

function. In other words, X_i counts how many of the n inserted keys hash to the specific bucket indexed by i .

Under SUH, each key independently maps to bucket i with probability $1/m$. Therefore, the number of keys in bucket i follows a binomial distribution:

$$X_i \sim \text{Binomial}(n, 1/m) \quad (6.9)$$

This distribution arises because there are n independent trials (one for each key), and each trial results in a “success” (the key hashes to bucket i) with probability $1/m$.

The expected number of keys in bucket i is given by the expectation of a binomial random variable $\mathbb{E}[X_i] = n \cdot \frac{1}{m}$. This simplifies to $\mathbb{E}[X_i] = \frac{n}{m}$. The quantity

$$\alpha = \frac{n}{m} \quad (6.10)$$

Load factor

is called the load factor, representing the average number of keys n stored per bucket m . Thus, $\mathbb{E}[X_i] = \alpha$. The load factor is a quantitative measure of how full a hash table is. It represents the average number of keys stored per bucket. There are three cases:

Case 1: If $\alpha = 1$, then on average each bucket contains one key.

Case 2: If $\alpha < 1$, the table is relatively sparse.

Case 3: If $\alpha > 1$, buckets (in separate chaining) contain multiple keys on average.

6.2.3 Low Collision Probability

A fundamental objective in the design of a hash function for hash tables is to minimize the probability of collisions. A collision occurs when two distinct keys $k_1, k_2 \in \mathcal{K}$, with $k_1 \neq k_2$, are mapped to the same bucket $h(k_1) = h(k_2)$.

Pigeonhole principle

Before examining collision behavior in hash tables, it is instructive to appeal to the Pigeonhole Principle as a fundamental counting argument. This principle provides a rigorous mathematical explanation for why collisions are unavoidable in hashing. By interpreting keys as objects and buckets as containers, the principle implies that whenever the number of keys exceeds—or is comparable to—the number of buckets, at least one bucket must contain more than one key.

In the setting of a hash table with m buckets storing n keys, collisions are therefore inevitable when $n \geq m$. Even in cases where $n \leq m$, collisions may still arise depending on the distribution induced by the hash function. Consequently, the objective in hash function design is not to eliminate collisions entirely, which is impossible in general, but to ensure that they occur with low probability and without systematic bias. The Pigeonhole Principle thus provides the theoretical foundation for analyzing collision probability and justifies the necessity of collision resolution mechanisms in hashing.

Probabilistic Model

Under the assumption of simple uniform hashing, each key is assumed to be mapped independently and with equal probability to any of the m buckets, as expressed in Equation 6.7.

For two distinct keys k_1 and k_2 , the probability that they collide is:

$$\Pr[h(k_1) = h(k_2)] = \frac{1}{m} \quad (6.11)$$

If n keys are inserted into the table, there are

$$\binom{n}{2} = \frac{n(n-1)}{2} \quad (6.12)$$

distinct pairs of keys.

The expression $\binom{n}{2}$, read as “ n choose 2,” denotes a binomial coefficient representing the number of distinct unordered pairs that can be selected from a set of n keys. More generally, the binomial coefficient is defined as

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

where $n!$ denotes the factorial of n .

Here $r = 2$, because a collision involves exactly two distinct keys, we obtain

$$\binom{n}{2} = \frac{n!}{2!(n-2)!} \quad (6.13)$$

Simplifying the factorial expression yields

$$\binom{n}{2} = \frac{n(n-1)}{2} \quad (6.14)$$

Equation 6.14 counts how many different pairs can be formed from n keys when the order does not matter. For example, if you have 4 keys, you can form 6 pairs like (Key1-Key2), (Key1-Key3), (Key1-Key4), (Key2-Key3), (Key2-Key4), and (Key3-Key4). In other words, $\binom{n}{2}$ simply tells us how many different key-pairs exist that could potentially collide.

To compute the expected total number of collisions, multiply number of possible pairs and the probability that any given pair collides.

$$\begin{aligned}
\mathbb{E}[\text{colliding pairs}] &= \binom{n}{2} \cdot \frac{1}{m} \\
&= \frac{n(n-1)}{2} \cdot \frac{1}{m} \\
&= \frac{n(n-1)}{2m}
\end{aligned} \tag{6.15}$$

i Effect of Increasing the Number of Buckets

Equation 6.15 shows that, for fixed n , the expected number of colliding pairs decreases as m increases, since the expression is inversely proportional to m . This conclusion holds under the assumption of simple uniform hashing. If the hash function does not distribute keys uniformly, increasing m alone may not yield a corresponding reduction in collisions.

6.2.4 Avalanche Effect

The avalanche effect refers to the property of a hash function whereby a small change in the input key produces a large and seemingly unpredictable change in the resulting hash value. In the context of hash tables, this property promotes effective key dispersion and helps prevent clustering caused by structurally similar inputs.

If the change in the hash value is also small when the input is slightly modified, the outputs become correlated. In such cases, similar keys may map to nearby or identical buckets, leading to non-uniform distribution. This correlation increases the likelihood of clustering, degrades average-case performance, and may cause the hash table to deviate from its expected $O(1)$ behavior. Therefore, insufficient sensitivity to small input variations is indicative of a weak hash function for practical hashing applications.

Let

$$h : \mathcal{K} \rightarrow \{0, 1\}^w \tag{6.16}$$

be a hash function whose output is represented as a binary string of length w bits.

$$k_1, k_2 \in \mathcal{K}, \quad k_1 \neq k_2, \tag{6.17}$$

such that k_2 differs only minimally from k_1 . For example, a single character change in a string, a single bit flip in an integer, and a small increment in a numeric key. To measure the difference between their hash outputs, we use the Hamming distance . The Hamming distance is a measure of the difference between two strings of equal

Hamming distance

length. It is defined as the number of positions at which the corresponding symbols differ.

Formally, let

$$x = (x_1, x_2, \dots, x_w), \quad y = (y_1, y_2, \dots, y_w) \quad (6.18)$$

where x and y denote two strings of length w over the same alphabet (commonly binary). The Hamming distance between x and y is defined as

$$d_H(x, y) = \#\{i \in \{1, \dots, w\} : x_i \neq y_i\} \quad (6.19)$$

Where, $d_H(x, y)$ is the number of positions at which the two strings differ, $\{1, \dots, w\}$ is the set of all index positions in the strings, $x_i \neq y_i$ are the two strings differ at position i , $\{i \in \{1, \dots, w\} : x_i \neq y_i\}$ represents the set of all positions where the two strings differ and $\#\{\dots\}$ denotes the cardinality of a set, i.e., counts how many elements are in that set.

Using Equation 6.19, the Hamming distance of k_1 and k_2 can be represented by

$$d_H(h(k_1), h(k_2))$$

Now, determine the expected Hamming distance between the two outputs.

Assume the hash output has w bits. For a small change in the input, each output bit flips with probability $\frac{1}{2}$ and each bit flip is approximately independent.

Let X_i be the indicator variable

$$X_i = \begin{cases} 1 & \text{if bit } i \text{ differs,} \\ 0 & \text{otherwise.} \end{cases}$$

Then, the probability that the i -th bit differs between the two hash outputs is one-half.

$$\Pr(X_i = 1) = \frac{1}{2}$$

The Hamming distance is the total number of differing bits:

$$d_H = \sum_{i=1}^w X_i$$

Since each X_i is Bernoulli with probability $\frac{1}{2}$,

$$\mathbb{E}[X_i] = \frac{1}{2}.$$

By linearity of expectation, the expected Hamming distance between the two outputs will be:

$$\mathbb{E}[d_H] = \sum_{i=1}^w \mathbb{E}[X_i] = w \cdot \frac{1}{2} = \frac{w}{2} \quad (6.20)$$

Equation 6.20 explains that each bit in the hash output has a 50% chance of changing when the input is slightly modified. Since there are w total bits and each contributes an average of $\frac{1}{2}$ difference, the total expected number of differing bits is $w \times \frac{1}{2} = \frac{w}{2}$. In simple terms, if each bit behaves like a fair coin flip, then about half of the bits will differ on average.

The explanation below will help readers better understand the concept.

Statistical Justification for the $\frac{w}{2}$ Expectation: The expected Hamming distance is not w because flipping all bits would produce a perfectly inverted and therefore highly structured output, indicating deterministic behavior rather than randomness. Conversely, if only a few bits change, the outputs remain strongly correlated, reflecting poor mixing. The most desirable behavior occurs when each bit flips independently with probability $\frac{1}{2}$, yielding maximal uncertainty and entropy. In this case, the hash output changes in a balanced and statistically unpredictable manner, which promotes effective dispersion.

Why Not Exactly $\frac{w}{2}$? The Hamming distance is a random variable with mean $\frac{w}{2}$ and variance $\frac{w}{4}$. Consequently, the value of d_H fluctuates around $\frac{w}{2}$; some instances will exhibit slightly fewer differing bits, while others will exhibit slightly more. Therefore, $\frac{w}{2}$ represents an expected value rather than an exact deterministic outcome.

6.2.5 Robustness to Structured Input Patterns

The theoretical analysis of hash functions often assumes that inputs are uniformly random and mutually independent. Under this assumption, performance guarantees—such as expected constant-time lookup in hash tables—are derived from probabilistic models like the Simple Uniform Hashing Assumption (SUH). However, real-world inputs are rarely random. They frequently exhibit structured, correlated, or patterned characteristics. Consequently, a well-designed hash function must remain effective not only under random inputs but also under structured input distributions. Robustness to structured input patterns refers to the ability of a hash function to maintain near-uniform output distribution and low collision probability when inputs possess predictable regularities. This property is essential for ensuring stable performance in practical systems.

In realistic environments, keys often follow systematic patterns rather than arbitrary distributions. Sequential integers, monotonically increasing timestamps, aligned memory addresses, formatted identifiers, and strings sharing common prefixes are typical examples. For instance, identifiers such as “student_001,” “student_002,” and “student_003,” or course codes like “CISC179-Fall2026-001,” differ only in small, structured components. Similarly, database record IDs and log sequence numbers frequently increase in arithmetic progression. When a hash function is designed with only random inputs in mind, such structured datasets may interact with weaknesses in its arithmetic or bit-level design, leading to clustering and elevated collision rates.

The vulnerability arises when input regularities align with linear or insufficiently mixed hash computations. A simple modular hash function of the form $h(k) = k \bmod m$ illustrates this problem clearly. If the keys are multiples of m , every key maps to the same bucket. For example, consider the keys 100, 200, 300, 400, and 500. These keys are not random; they follow a structured arithmetic progression—specifically, they are all multiples of 100. Since each key is divisible by 100, we obtain $100 \bmod 100 = 0$, $200 \bmod 100 = 0$, $300 \bmod 100 = 0$, and so on. The degradation occurs because the arithmetic structure of the inputs coincides exactly with the arithmetic structure imposed by the modulus operation.

This phenomenon is not unique to the modulus function; it reflects a broader property of linear hash constructions. Functions of the form $h(k) = ak + b$ preserve arithmetic relationships in the input domain. If the keys form an arithmetic progression, their hashed values will also form a predictable progression modulo the table size. Consequently, structured sequences in the input may remain structured after hashing, producing clustering effects. The issue, therefore, is not the mere existence of structure in the data, but the presence of a mathematical correlation between that structure and the transformation applied by the hash function. When the algebraic properties of the input align with those of the hash computation, distribution quality can deteriorate significantly—even if the function performs adequately under random input assumptions.

To achieve robustness, a hash function must incorporate strong mixing mechanisms that diffuse input regularities across the output space. Effective mixing ensures that even highly correlated inputs produce outputs that appear statistically independent. Multiplicative hashing with carefully selected constants, combined with bitwise shifts, XOR operations, and rotations, helps break linear relationships and spread entropy from both high-order and low-order bits. This consideration is particularly important when hash table sizes are powers of two, since Naive masking — the practice of computing a hash index by simply retaining the lower-order bits of a key through a bitwise AND operation with $m - 1$ can discard meaningful variation contained in the higher-order bits, thereby increasing the risk of systematic clustering when structural entropy resides outside the preserved portion. If structured inputs differ primarily in those higher bits, a weak hash may collapse them into the same buckets. Robust design therefore requires that all portions of the input meaningfully influence the final hash value.

Naive masking

String hashing introduces additional challenges because many real-world strings exhibit substantial structural similarity, particularly in the form of long shared prefixes. Examples include file paths, URLs, and systematically generated identifiers, which often differ only in their suffixes. If a hash function disproportionately emphasizes early characters in the string, inputs that share common prefixes may map to nearby or identical hash values, resulting in clustering.

Well-designed string hashing methods, such as polynomial rolling hashes, mitigate this issue by assigning positional weights to characters and incorporating each symbol into a cumulative computation. This approach ensures that differences in later positions meaningfully influence the final hash value. Careful selection of parameters—such as the base and modulus—further reduces correlation effects and improves distribution quality.

The central objective in string hashing is to prevent localized similarity in the input (e.g., shared prefixes or minor variations) from translating into localized similarity in the output.

From a theoretical perspective, robustness to structured input patterns is closely related to universal hashing and independence properties. A hash family that approximates uniform distribution for any distinct pair of inputs limits the probability of collision to approximately $1/m$, where m is the number of buckets. Such probabilistic guarantees reduce the likelihood that any particular structured dataset can systematically exploit weaknesses in the hash design. In adversarial settings, where inputs may be intentionally crafted to cause collisions, robustness becomes even more critical to prevent degradation from expected $O(1)$ performance toward worst-case $O(n)$ behavior.

The practical implications of insufficient robustness are substantial. When a hash function fails to distribute structured inputs evenly, performance degradation follows regardless of the specific collision-resolution strategy employed. Uneven distribution increases the frequency of collisions, raises access costs, and undermines the expected constant-time behavior commonly associated with hashing. In large-scale or distributed environments, poor hashing can result in load imbalance, where certain storage locations or processing units become disproportionately burdened. In security-sensitive contexts, predictable collision patterns may be exploited to intentionally degrade system performance, potentially leading to denial-of-service scenarios. For these reasons, modern programming languages, libraries, and database systems incorporate carefully engineered mixing techniques to ensure resilience against structured and adversarial input patterns.

It is important to distinguish robustness from the avalanche effect. The avalanche effect ensures that a small change in input—such as flipping a single bit—produces widespread changes in the output bits. Robustness to structured input patterns, by contrast, concerns the global statistical behavior of the hash function when confronted with correlated or patterned datasets. Avalanche describes local sensitivity; robustness describes stability under structured distributions. Both properties are necessary

for reliable hash performance.

In summary, while theoretical analysis often relies on uniform randomness assumptions, practical hash design must anticipate structured, correlated, and predictable inputs. Robustness to structured input patterns ensures that such regularities do not undermine uniform distribution, low collision probability, and expected constant-time performance. Through strong mixing, avoidance of linear dependence, incorporation of all input bits, and probabilistic safeguards against correlation, a well-designed hash function maintains stable and reliable behavior even under highly structured workloads.

6.2.6 Computational Efficiency of the Hash Function

Computational efficiency is a fundamental design requirement of any hash function because hashing is not an occasional operation—it is performed repeatedly and often at very high frequency in practical systems. Every insertion, lookup, deletion, indexing operation, dictionary access, symbol-table query, cache lookup, or database retrieval typically begins with computing a hash value. If the hash computation itself is slow, then even a theoretically well-designed hash table will suffer poor overall performance. Thus, before discussing distribution quality, collision resistance, or avalanche behavior, one must first ensure that the hash function is computationally efficient.

In algorithmic terms, the total time of a hash-table operation can be written as

$$T_{\text{operation}} = T_{\text{hash}}(n) + T_{\text{collision handling}} \quad (6.21)$$

where n refers to the length (or size) of the key being hashed, not the number of elements in the hash table.

For example:

- If the key is an integer, then n is effectively constant (e.g., 4 or 8 bytes).
- If the key is a string of length 20, then $n = 20$.
- If the key is a long document of 10,000 characters, then $n = 10,000$.

Even if collision handling is expected to be $O(1)$ under standard assumptions, the hashing cost $T_{\text{hash}}(n)$ is unavoidable. Therefore, the efficiency of the hash function directly influences system throughput. When millions of keys are processed per second—as in compilers, databases, distributed systems, and networking infrastructure—the constant factors inside the hash computation become critically important.

The computational efficiency of a hash function refers to the computational resources required to evaluate the hash value of an input of length n , where time is the primary concern and space plays a comparatively minor role. In algorithmic analysis, efficiency is typically described using asymptotic notation. If the input key is modeled as a sequence of bytes

$$x = (x_0, x_1, \dots, x_{n-1})$$

then any hash function $h(x)$ must examine each component at least once in order to produce a value that depends on the entire input. Consequently, every practical hash function is subject to a fundamental lower bound

$$T(n) = \Omega(n)$$

where Ω (the Greek letter Omega) denotes a lower bound in asymptotic analysis. Since no algorithm can compute a function of all input bits without first reading them, the running time cannot grow more slowly than linearly in the input length. This establishes a theoretical minimum: linear time is unavoidable.

The central objective in hash function design is therefore to achieve linear time complexity while minimizing the associated constant factor, ensuring that the running time is bounded above by a linear function of the input size. Formally, this requirement is expressed as

$$T(n) = O(n) \tag{6.22}$$

where the notation $O(n)$ denotes an asymptotic upper bound on growth rate of the running time. Specifically, it indicates that the execution time does not increase faster than linearly as n becomes large.

Equation 6.22 can be written precisely as:

$$T(n) = an + b$$

where $a > 0$ represents the multiplicative constant corresponding to the cost incurred per input element, and $b \geq 0$ denotes the additive constant accounting for fixed overhead independent of n .

To formalize this expression using Big-O notation, the equality is converted into an inequality of the form:

$$T(n) \leq c \cdot n + b$$

for some constant $c > 0$. This inequality captures the idea that the running time is bounded above by a linear function of the input size. The constant c serves as an upper bound on the number of elementary operations required per unit of input, while the additive term b reflects fixed computational costs that do not scale with n .

For sufficiently large n , the linear term dominates the additive constant, and constant multiplicative factors do not alter the order of growth. Hence, both the coefficient of n and the fixed constant term are absorbed in the asymptotic representation, yielding the classification $T(n) = O(n)$.

The computational efficiency of a hash function is fundamentally governed by the necessity of examining each component of the input. This requirement establishes a lower bound of $\Omega(n)$, while careful algorithmic design ensures a corresponding upper bound of $O(n)$. Together, these bounds yield a tight asymptotic characterization $T(n) = \Theta(n)$, indicating that linear time is both unavoidable and optimal.

Although asymptotic notation abstracts away constant factors and additive terms, practical performance remains sensitive to these implementation details. Consequently, the design of efficient hash functions focuses not only on achieving linear growth in theory but also on minimizing the constant cost per input element in practice. An effective hash function therefore balances theoretical optimality with engineering efficiency, ensuring that hash computation does not become a bottleneck in higher-level data structure operations.

This understanding of computational efficiency provides a foundation for subsequent design considerations, including distribution quality, mixing behavior, and robustness under structured inputs.

6.3 Principles of Compression Function Design

A compression function in hashing serves the critical role of transforming a potentially large intermediate hash value into a bounded integer suitable for indexing within a finite table. Unlike the primary hash computation—which may generate wide integer outputs to preserve entropy—the compression function reduces this value to a fixed range while preserving as much of its distributional quality as possible. Because this step directly influences index distribution and collision behavior, its design must balance mathematical soundness with computational efficiency. In the following section, we examine the fundamental design principles that govern the construction of effective compression functions, with particular attention to correctness, uniformity, and robustness.

6.3.1 Range Constraints

The most fundamental requirement of a compression function in array-based implementations is that its output must lie within the bounds of the underlying table. Formally, this requirement is expressed as:

$$0 \leq h(k) < m \quad (6.23)$$

where $h(k)$ denotes the array index obtained by evaluating the hash function on key k and m represents the size of the hash table. This condition guarantees that every generated index corresponds to a valid array position.

In practical implementations, this constraint is typically enforced by applying a modulus operation to a preliminary hash value:

$$h(k) = H(k) \bmod m \quad (6.24)$$

where $H(k)$ denotes the intermediate hash value generated before the application of the compression function.

The underlying design principle is one of correctness and safety. A hash function must deterministically map every possible key to a valid table index, thereby preventing out-of-bounds memory access and ensuring reliable operation of the hash table.

6.3.2 *Mixing and Diffusion*

Another requirement in compression function design is the principle of mixing and diffusion. The compression function must combine input bits in such a manner that each output bit is influenced by a large subset—ideally all—of the input bits. This property prevents structural correlations between input patterns and output patterns and ensures that the transformation behaves in a statistically decorrelated manner.

To illustrate the necessity of diffusion, consider a weak compression scheme

$$h(x) = x \bmod m$$

where $m = 2^k$. In this case, only the lowest k bits of x determine the result. Suppose $x_1 = 0x12340000$ and $x_2 = 0x56780000$.

If the lowest k bits of two inputs are identical, both values will produce the same table index, even if their higher-order bits differ substantially. In this situation, any variation in the upper portion of the input has no effect on the output. This illustrates zero diffusion between bit regions: similarity in the retained lower bits is preserved, while differences in the higher-order bits are entirely disregarded. Zero diffusion refers to a pathological condition in which a change in one portion of the input produces no effect on certain portions of the output. In such cases, influence fails to propagate across specific bit regions, and some output bits remain completely independent of some input bits. A well-designed compression function must eliminate structural independence by ensuring that variation in any portion of the input propagates throughout the entire output space via systematic mixing and diffusion. Equivalently, the function must exhibit sensitivity to all input positions: no subset of input bits should remain algebraically or structurally disconnected from any region of the output.

Now consider a slightly improved design that incorporates multiplicative mixing:

$$h(x) = (x \cdot C) \bmod 2^{32}$$

where C is a carefully chosen odd constant. Suppose, $x_1 = 1000$ and $x_2 = 1001$

Even though the inputs differ by only one unit, multiplication introduces carry propagation across bit positions. The least significant change affects higher-order bits due to the arithmetic carries produced during multiplication. When followed by a bit

rotation or further XOR combination, the resulting outputs differ across many bit positions. This illustrates diffusion: a localized input change propagates globally through arithmetic interaction.

Bit rotations provide another illustrative example. Consider a function:

$$h(x) = (x \oplus (x \gg 16)) \quad (6.25)$$

Here, the higher 16 bits are XORed into the lower 16 bits. If two inputs differ only in their upper half, the difference is transferred into the lower half. This ensures that compression into a smaller table size does not discard the variation. Rotations improve this further:

$$h(x) = \text{rotate_left}(x, r) \oplus x \quad (6.26)$$

where, the parameter r denotes the rotation distance, that is, the number of bit positions by which the word x is cyclically shifted to the left. A rotation repositions bits before combination, allowing influence to spread across positions that would otherwise remain independent.

To illustrate diffusion in a string context, consider hashing two similar keys:

```
"data_structure_2026"
"data_structure_2027"
```

If a compression function overemphasizes early characters, both strings produce nearly identical intermediate states until the final character is processed. Without cumulative mixing at each step (for example, multiplying by a base and adding the next character), similarity persists deep into the computation. In contrast, a polynomial rolling approach ensures that each character contributes with positional weight, and each step multiplies the current state before incorporating the next symbol. This repeated multiplication induces diffusion, so that a single character difference affects many bits of the final result.

$$h(s) = \sum_{i=0}^{n-1} s[i] \cdot p^i \mod M \quad (6.27)$$

The expression in Equation 6.27 defines a polynomial rolling hash for a string s . Here, s denotes the input string and n its length. The index variable i ranges from 0 to $n - 1$, so that each character of the string is processed exactly once. The term $s[i]$ represents the numeric value of the character at position i , typically its ASCII or Unicode code. The constant p is a fixed base (often a small prime) that assigns positional weight to each character through the factor p^i , ensuring that characters in different positions contribute differently to the final result. The summation symbol indicates that all

weighted character values are added together, forming a polynomial-like representation of the string. Finally, the modulus M reduces the computed sum to a fixed numeric range, preventing unbounded growth and allowing the hash value to fit within a machine word or a predefined table size.

In summary, mixing and diffusion are not abstract theoretical ideals; they are realized through specific arithmetic and bitwise operations that cause carries, wraparound effects, positional reordering, and cross-bit interaction. When properly combined, these mechanisms ensure that even minimal input variation produces widespread transformation across the output space, thereby promoting uniform distribution and structural robustness.

6.3.3 *Non-Linearity*

Since the concepts of mixing and diffusion have already been established, it is important to distinguish non-linearity as a complementary but conceptually distinct design principle in compression function construction. Mixing and diffusion ensure that small changes in the input propagate widely throughout the internal state. However, if the compression transformation consists exclusively of linear operations—such as XOR, bitwise shifts that can be modeled linearly, or other linear combinations over a binary field—the resulting function remains algebraically linear. Although the bits of the input may be rearranged or diffused across positions, the underlying algebraic relationships between inputs are not destroyed; they are merely redistributed. Such structural preservation renders the function vulnerable to linear analysis and facilitates the construction of collisions under adversarial or patterned inputs.

A well-designed compression function therefore incorporates non-linear components to disrupt algebraic regularities. Non-linearity may be introduced through modular multiplication (which is non-linear over binary vector spaces), non-linear bit transformations, or substitution-like mechanisms that break linear superposition properties. These mechanisms ensure that the output cannot be expressed as a simple linear function of the input bits.

Formally, the compression function should resist linear characterization over finite fields by avoiding transformations that can be represented as purely linear mappings. By combining prior mixing and diffusion properties with explicit non-linear operations, the design achieves both broad propagation of input influence and resistance to algebraic predictability.

6.3.4 *Bit Independence*

Bit independence is a fundamental probabilistic requirement in the design of hash and compression functions. It states that when a single input bit is altered, the resulting changes in the output bits should occur independently of one another.

This property is closely related to, but distinct from, the avalanche effect. The avalanche effect concerns the magnitude of change: ideally, toggling one input bit should cause approximately half of the output bits to flip. Equation 6.20 formalizes

the avalanche criterion as an expected Hamming distance of half the output length.

In addition to the modulo operator, several alternative techniques exist for computing a valid array index from a hash value, depending on the implementation strategy and performance considerations of the hash table.

Bit independence imposes a stronger condition. It requires that the random variables $X_1, \dots, X_w$ exhibit no statistical dependence. That is, the event that one output bit flips should not influence the probability that another output bit flips. If correlations exist—such as certain bits tending to flip together or following consistent joint patterns—the function may still preserve structural relationships, even if the overall number of flipped bits appears adequate.

Thus, avalanche measures how many bits change on average, whereas bit independence governs how those changes are distributed statistically. Together, these properties reduce structured correlations between inputs and outputs, mitigate clustering effects, and enhance resistance to analytical attacks.

6.3.5 *Parameter Selection and Constant Choice*

The selection of parameters and constants within a compression function is frequently underestimated, yet it plays a decisive role in practical performance and distributional quality. Poorly chosen constants may introduce structural bias, amplify regularities in the input, or enable trivial collision patterns. Conversely, carefully selected constants enhance statistical dispersion and improve mixing across bit positions.

Prime multipliers are commonly employed because they reduce alignment effects between input regularities and modular arithmetic, thereby mitigating periodic clustering. Odd constants are preferred in many multiplicative schemes to ensure full participation of lower-order bits and to avoid degenerate mappings in power-of-two table sizes. In more refined designs, constants derived from irrational quantities—such as scaled fractional bits of the golden ratio are used to discourage unintended arithmetic correlations.

Although general-purpose hashing does not require cryptographic-level rigor, principled constant selection strengthens robustness and provides theoretical depth to the design. Proper parameterization ensures that the compression function does not inadvertently preserve algebraic structure present in the input domain.

6.3.6 *Efficient Computability on Target Hardware*

In contrast to cryptographic compression functions, which prioritize adversarial resistance, general-purpose hash compression functions must emphasize computational efficiency. Since hashing is often performed in performance-critical paths—such as indexing, caching, and memory management—the per-operation cost directly influences overall system throughput.

Efficient implementations favor operations that map naturally to hardware primitives, including integer addition, multiplication, and bitwise shifts or XORs. Branch-

free constructions are preferred where possible, as they minimize pipeline stalls and branch misprediction penalties. Cache-friendly behavior is equally important; avoiding large lookup tables reduces memory latency and improves locality of reference.

Thus, beyond theoretical distributional properties, a well-designed compression function must achieve a balance between statistical robustness and low-level implementation efficiency. The objective is to deliver high-quality mixing at minimal computational cost, thereby ensuring scalability across diverse hardware architectures.

6.4 Hash Collisions: Inevitability and Impact

Despite the careful design of hash functions and compression mechanisms, hash collisions are an inherent and unavoidable aspect of hash-based data structures. This inevitability stems from the fundamental mismatch between the potentially vast—or even unbounded—space of input keys and the finite number of available storage locations (buckets) within a hash table. Regardless of how uniformly a hash function distributes keys, it is impossible to guarantee a unique mapping for all inputs. Consequently, collisions must be regarded as a natural outcome of the hashing process rather than a deficiency in design.

A hash collision occurs when two or more distinct keys are mapped to the same index in the hash table. Formally, for keys $k_1 \neq k_2$, a collision occurs if $h(k_1) = h(k_2)$, where $h(\cdot)$ denotes the combined effect of the hash function and the compression function. In practical terms, this implies that multiple elements must be stored within the same bucket, requiring additional mechanisms to differentiate and retrieve them correctly.

The occurrence of collisions is primarily a consequence of the compression of a large domain of hash values into a limited range of indices. Even if the hash function produces well-distributed values, the compression step inevitably maps multiple distinct hash codes to the same bucket. Moreover, real-world input distributions are often non-uniform, which can further increase the likelihood of collisions in certain regions of the table.

The impact of collisions on performance is significant. In the ideal case, where each key maps to a unique bucket, hash table operations such as insertion, deletion, and lookup can be performed in constant time, $O(1)$. However, when collisions occur, multiple elements accumulate within the same bucket, increasing the number of comparisons required to locate a specific key. As a result, the time complexity of operations becomes dependent on the number of elements associated with a given bucket. In the worst case, where many keys are mapped to a single bucket, the hash table may degrade to linear-time performance, $O(n)$.

Load factor

A central factor influencing the frequency and severity of collisions is the load factor, defined as the ratio of the number of stored elements n to the number of buckets m . The load factor provides a quantitative measure of how densely populated a hash table is. A higher load factor is more memory-efficient, as it requires fewer buckets

to store a given number of elements; however, it also necessitates accommodating more elements within a fixed number of buckets, thereby increasing the likelihood of collisions. As a result, multiple elements are more frequently associated with the same bucket, leading to increased costs for search, insertion, and deletion operations and consequently degrading overall performance. In contrast, maintaining a lower load factor promotes a more uniform distribution of elements across buckets, thereby preserving efficient access times, albeit at the expense of increased memory consumption.

To mitigate the adverse effects of an increasing load factor, many hash table implementations employ [rehashing](#), a process that increases the number of buckets and redistributes existing elements according to the hash function. Although rehashing incurs a linear cost $O(n)$, it is performed infrequently and ensures that the average-case performance of operations remains close to constant time.

[Rehashing](#)

Rehashing is typically governed by a predefined threshold on the load factor, rather than a universally fixed value. In practice, most implementations initiate rehashing when the load factor exceeds a value in the range of approximately 0.7 to 1.0. In the context of C++ standard library containers such as `std::unordered_map` and `std::unordered_set`, this behavior is formalized through the concept of a maximum load factor, which is by default set to approximately 1.0. Once the load factor surpasses this threshold, the container automatically increases the number of buckets and redistributes all elements accordingly.

[Maximum load factor](#)

Maximum Load Factor in Unordered Containers

The maximum load factor of an unordered container can be queried and modified using the `max_load_factor()` function, which returns (or sets) the threshold that governs when rehashing is performed. While it is possible to adjust this value, such modifications should be undertaken with caution, as inappropriate settings may adversely affect performance or lead to inefficient memory usage. In general, it is advisable to rely on the default value unless there is a clear understanding of the underlying trade-offs.

The following example illustrates how the maximum load factor can be inspected and modified in a C++ unordered container, as well as how rehashing may be triggered.

Code Example 6-1: Example of hashing

```
1 #include <iostream>
2 #include <unordered_map>
3 using namespace std;
4
5 int main()
6 {
7     unordered_map<int, int> m;
8 }
```

```

9      // Set a custom maximum load factor
10     m.max_load_factor(0.7);
11
12     // Insert elements
13     for (int i = 0; i < 10; i++)
14     {
15         m[i] = i * 10;
16         cout << "Size: " << m.size() // this is 'n'
17              << ", Buckets: " << m.bucket_count() // this is 'm'
18              << ", Load Factor: " << m.load_factor()
19              << endl;
20     }
21
22     return 0;
23 }

```

```

Size: 1, Buckets: 2, Load Factor: 0.5
Size: 2, Buckets: 5, Load Factor: 0.4
Size: 3, Buckets: 5, Load Factor: 0.6
Size: 4, Buckets: 11, Load Factor: 0.363636
Size: 5, Buckets: 11, Load Factor: 0.454545
Size: 6, Buckets: 11, Load Factor: 0.545455
Size: 7, Buckets: 11, Load Factor: 0.636364
Size: 8, Buckets: 23, Load Factor: 0.347826
Size: 9, Buckets: 23, Load Factor: 0.391304
Size: 10, Buckets: 23, Load Factor: 0.434783

```

In this example, the maximum load factor is set to 0.7, causing the container to maintain a lower threshold for the number of elements per bucket. As elements are inserted, the load factor gradually increases. Whenever an insertion would cause the load factor to exceed the specified threshold, the container performs rehashing by increasing the number of buckets and redistributing the elements. In this case, rehashing occurs multiple times, as indicated by successive changes in the bucket count. This behavior ensures that the load factor remains within the desired bound, thereby preserving efficient average-case performance.

Given the inevitability of collisions and their direct influence on performance, it is essential to adopt effective mechanisms for managing them. These mechanisms, known as collision resolution techniques, enable multiple elements to coexist within the hash table while maintaining efficient access. The following sections examine these techniques in detail, highlighting their design principles and performance characteristics.

6.5 Collision Resolution Strategies

Collision resolution constitutes a fundamental component of hash table design, as collisions are unavoidable when a potentially large set of keys is mapped into a finite number of buckets. An effective collision resolution strategy is essential to preserving the expected average-case efficiency of hash table operations in the presence of such conflicts. Broadly, collision resolution techniques may be classified into two principal categories: separate chaining and open addressing, each encompassing several variations and implementation strategies.

6.5.1 Separate Chaining

Separate chaining is a fundamental collision resolution technique used in hash tables, wherein each bucket is capable of storing multiple elements. Rather than restricting each index of the hash table to a single entry, separate chaining associates each bucket with an auxiliary data structure—most commonly a linked list—allowing all elements that hash to the same index to be stored together. This approach effectively transforms each bucket into a container of elements sharing the same hash value.

Let $h(k)$ be a hash function that maps a key k to an index in a table of size m . In separate chaining, all keys that satisfy

$$h(k_1) = h(k_2) = \dots = i$$

are stored in the same bucket i , typically as a sequence (or chain) of elements. Each bucket therefore maintains a collection of entries rather than a single value.

The hash table consists of an array of size m , where each entry points to a secondary data structure:

- **Linked List (most common):** Each bucket contains a linked list of key–value pairs.
- **Dynamic Array (vector):** Used when locality of reference is important.
- **Balanced Tree (e.g., red–black tree):** Employed in some implementations to improve worst-case performance.

Insertion in separate chaining involves placing a key–value pair (k, v) into the appropriate bucket determined by the hash function. Specifically, the hash index $i = h(k)$ is first computed, after which the element is inserted into the corresponding bucket B_i . The insertion is typically performed at either the head or the tail of the chain, depending on the implementation strategy. Under the assumption of uniform hashing and a controlled load factor, the average-case time complexity of insertion remains $O(1)$. However, in the worst case—when all elements hash to the same bucket—the operation degrades to $O(n)$. The degradation of insertion to $O(n)$ in the worst

case arises from the cost of handling collisions within a single bucket. When all elements hash to the same index, the corresponding bucket B_i effectively contains all n elements of the table, forming a chain of length n .

If the implementation requires checking for duplicate keys before insertion (as the case in containers such as `std::unordered_map`, where keys must be unique), the algorithm must traverse the entire chain to ensure that the key k does not already exist. This traversal takes $O(n)$ time. Only after this check can the new element be safely inserted.

Keys may be inserted at either the head or the tail of the linked list in a separate chaining scheme; both approaches are theoretically valid. However, in practice, insertion at the head is generally preferred due to its superior efficiency characteristics. In a typical implementation using a singly linked list, insertion at the head can be performed in constant time $O(1)$, as it requires only updating a small number of pointers without any need for traversal.

In contrast, insertion at the tail necessitates locating the last node in the list prior to appending the new element. If the chain contains k elements, this operation incurs a traversal cost of $O(k)$, which may degrade to $O(n)$ in the worst case when all elements are mapped to a single bucket.

It is possible to achieve constant-time tail insertion by maintaining additional structural information, such as a pointer to the tail of each chain or by employing a doubly linked list. However, such enhancements introduce additional memory overhead and increase implementation complexity.

Consequently, insertion at the head is typically favored, as it guarantees $O(1)$ time without requiring auxiliary storage. Nevertheless, when the hash function fails to distribute keys uniformly, leading to excessive collisions and the accumulation of all elements within a single bucket, the cost of operations—including insertion under certain conditions—may degrade to linear time $O(n)$, effectively resembling the behavior of a sequential data structure.

Cost of hashing

Insertion in separate chaining does include the cost of hashing, but this cost is typically considered constant time, which is why it is absorbed into the overall $O(1)$ complexity.

However, it is important to note that this assumption depends on the nature of the key. For simple keys (e.g., integers), hashing is indeed constant time. For more complex keys, such as strings, the hash computation may take $O(|k|)$, where $|k|$ is the length of the key. In such cases, the cost of insertion becomes:

$$O(|k| + 1) = O(|k|)$$

Despite this, in algorithmic analysis of hash tables, the cost of hashing is typically treated as constant to simplify the model and focus on the behavior of collision

resolution.

Search (or lookup) in separate chaining involves locating a key k within the appropriate bucket determined by the hash function. The process begins by computing the hash index $i = h(k)$, which identifies the bucket B_i where the key is expected to reside. The algorithm then traverses the chain associated with this bucket, examining each element until the desired key is found or the chain is exhausted.

The average-case time complexity of search in separate chaining is expressed as $O(1 + \alpha)$ because the operation consists of two distinct components.

First, computing the hash index $i = h(k)$ takes constant time, which contributes the $O(1)$ term. This step directly identifies the bucket in which the key is expected to reside.

Second, once the appropriate bucket B_i is located, the algorithm must traverse the chain within that bucket to find the key. Under the assumption of uniform hashing, elements are distributed evenly across all m buckets. Consequently, each bucket contains, on average, $\alpha = \frac{n}{m}$ elements, where n is the total number of stored elements. Therefore, the expected cost of traversing the chain is proportional to α , giving rise to the $O(\alpha)$ term.

Combining these two components, the total expected cost of the search operation becomes:

$$O(1) + O(\alpha) = O(1 + \alpha) \quad (6.28)$$

The Equation 6.28 highlights that the efficiency of search depends not only on constant-time hashing but also on maintaining a small load factor. When α is bounded by a constant (e.g., through rehashing), the overall complexity remains effectively $O(1)$.

Deletion in separate chaining involves removing a key from the hash table by first identifying the appropriate bucket and then eliminating the corresponding element from the associated chain. Specifically, the hash index is computed to determine the bucket in which the key resides, after which the chain within that bucket is traversed to locate the target element. Once found, the element is removed from the underlying data structure, typically a linked list. This operation is relatively straightforward, particularly when linked lists are employed, as node removal can be performed efficiently with appropriate pointer adjustments. Under the assumption of uniform hashing, the average-case time complexity of deletion is $O(1 + \alpha)$, where α denotes the load factor, reflecting the expected number of elements per bucket.

Load Factor and Performance

The efficiency of separate chaining is strongly influenced by the load factor α . Unlike open addressing, separate chaining can tolerate load factors greater than 1, since multiple elements may reside in a single bucket.

- When α is small, chains are short, and operations are close to constant time.
- As α increases, chains grow longer, leading to increased traversal time.

In practice, rehashing may be performed to maintain a desirable load factor by increasing the number of buckets and redistributing elements.

Advantages & Limitations

Separate chaining provides several advantages that underpin its widespread adoption in hash table implementations. It is both conceptually clear and straightforward to implement, facilitating its use in theoretical analysis as well as practical applications. In its most common form, separate chaining is implemented using linked lists, wherein each bucket maintains a sequence of elements that share the same hash index. This approach simplifies insertion and deletion operations, as elements can be added or removed with minimal structural overhead. Additionally, the method exhibits flexibility with respect to the load factor, as it can maintain acceptable performance even when $\alpha > 1$, given that multiple elements may coexist within a single bucket. Deletion operations are particularly efficient, since elements can be removed directly from the chain without requiring auxiliary markers or structural reorganization.

In modern implementations, alternative data structures and heuristics may be employed within buckets to enhance performance. For instance, balanced trees (e.g., red-black trees) can be used to replace linked lists when the number of elements in a bucket exceeds a predefined threshold, thereby reducing the worst-case time complexity of search, insertion, and deletion operations from $O(n)$ to $O(\log n)$. Additionally, heuristic techniques such as the move-to-front strategy may be applied in list-based buckets, whereby frequently accessed elements are repositioned at the front of the chain, improving average access time by exploiting temporal locality in access patterns.

Notwithstanding these strengths, separate chaining is subject to certain limitations. It incurs additional memory overhead due to the reliance on auxiliary data structures, such as linked lists and pointer-based representations. Furthermore, the non-contiguous storage of elements results in poor cache locality, which may adversely affect performance on modern hardware architectures. The overall efficiency of the technique is also contingent upon the quality of the hash function; inadequate key distribution can lead to disproportionately long chains, thereby degrading performance and, in extreme cases, resulting in linear-time behavior.

6.5.2 Open Addressing

Open addressing is a collision resolution technique in hash tables in which all elements are stored directly within the table array, and collisions are resolved by probing alternative positions according to a well-defined sequence. When a key is hashed to an index that is already occupied, the algorithm generates a sequence of candidate indices using a probing function $h(k, i)$, where k is the key and i denotes the probe attempt. The process continues until either an empty slot is found (for insertion) or the

target key is located (for search). Unlike separate chaining, no auxiliary structures are used; thus, all operations are confined to the table itself, making open addressing space-efficient and cache-friendly.

A central aspect of open addressing is the choice of probing strategy, which determines how alternative slots are explored after a collision.

Linear Probing

The simplest and one of the most widely studied probing strategies used in open addressing for resolving hash collisions. In linear probing, the **insertion process** begins by computing the initial hash index $h(k)$. If the corresponding slot is empty, the key is inserted directly. However, if a collision occurs (i.e., the slot is already occupied by another key), the algorithm follows the linear probing sequence

$$h(k, i) = (h(k) + i) \bmod m, \quad i = 0, 1, 2, \dots, m - 1,$$

where m is the size of the hash table and i denotes the probe number. Thus, if the slot at $h(k)$ is occupied, the algorithm checks $h(k) + 1$, then $h(k) + 2$, and so on, wrapping around to the beginning of the table when necessary due to the modulo operation.

A subtle but important aspect of insertion is the treatment of tombstones (deleted slots). During insertion, tombstone slots are considered reusable. In many implementations, the algorithm records the first tombstone encountered during probing and may choose to insert the new key into that position rather than continuing to search for a completely empty slot. This helps in reclaiming space and maintaining shorter probe sequences. However, care must be taken to ensure correctness: if the same key already exists further along the probe sequence, inserting prematurely into a tombstone without checking may lead to duplicate entries in structures that require uniqueness (e.g., sets or maps).

Tombstones

Another critical consideration arises when no empty slot is found. If the table is full, insertion cannot proceed and must trigger a rehashing operation, in which a larger table is allocated and all existing elements are reinserted according to the hash function. In practice, this situation is avoided by maintaining the load factor below a predefined threshold (commonly around 0.7), ensuring that insertions remain efficient and probe sequences remain short.

The performance of insertion in linear probing is closely tied to the load factor. At low load factors, insertions typically require only a few probes, often constant time. However, as the table becomes more populated, clustering effects increase the number of probes required to find an empty slot, potentially degrading performance. In the worst case, insertion may require scanning a large portion of the table, resulting in $O(n)$ time complexity.

In linear probing, during a **search operation**, the algorithm follows the same probe sequence used during insertion and continues examining slots sequentially. If the

current slot contains a different key (i.e., a collision has occurred), the search does not terminate; instead, it proceeds to the next slot in the sequence. This process continues until one of the following conditions is met: (i) the desired key is found, or (ii) an empty slot that has never been occupied is encountered, which implies that the key is not present in the table.

If no empty slot exists—i.e., the table is completely full—the search may traverse the entire table before concluding that the key is absent. However, in practical implementations, such a situation is avoided by maintaining the load factor below a predefined threshold and triggering rehashing before the table becomes full.

It is important to distinguish between an empty slot and a tombstone. A tombstone represents a previously occupied slot that has been logically deleted. When a tombstone is encountered during a search, the probing must continue, since the desired key may have been placed further along the probe sequence. Only a truly empty slot (one that has never held any key) guarantees that the search can safely terminate unsuccessfully.

In linear probing, **deletion** cannot be performed by simply removing an element and marking its slot as empty. This is because the correctness of search operations relies on the continuity of the probe sequence. When keys collide, they are stored in contiguous positions following their original hash index. If a slot in this sequence were cleared, subsequent searches for elements further along the chain might terminate prematurely upon encountering the empty slot, incorrectly concluding that the key is not present.

To preserve the integrity of the probe sequence, deletion is implemented using a special marker known as a tombstone (or deleted marker). When an element is removed, its slot is not marked as empty but instead marked as deleted. This indicates that the slot no longer contains a valid key but must still be treated as occupied during probing. Consequently, during search operations, encountering a tombstone does not terminate the probe sequence; the algorithm continues probing until it finds the desired key or reaches a slot that has never been occupied.

From the perspective of insertion, tombstone slots are considered reusable. A new key may be inserted into a slot marked as deleted, thereby reclaiming space. However, care must be taken in implementations that enforce key uniqueness: the probing sequence should continue beyond the tombstone to ensure that the key does not already exist later in the sequence before performing the insertion.

A practical concern with tombstones is that they accumulate over time, effectively increasing probe lengths and degrading performance, even though some slots are logically available. To address this, implementations periodically perform rehashing, in which a new table is created and only active elements are reinserted, thereby eliminating tombstones and restoring efficient operation.

Quadratic Probing

In addition to linear probing, quadratic probing is another widely used probing strategy in open addressing for resolving hash collisions. In this approach, the probe sequence is generated using a quadratic function of the probe number, typically expressed as

$$h(k, i) = (h(k) + c_1 i + c_2 i^2) \bmod m$$

where c_1 and c_2 are constants and m is the table size. Unlike linear probing, which examines consecutive slots, quadratic probing increases the step size non-linearly, thereby spreading probes more widely across the table. This reduces the effect of primary clustering, although it may still exhibit secondary clustering when multiple keys share the same initial hash value.

Double Hashing

Double hashing is an advanced probing strategy used in open addressing to resolve collisions in hash tables, designed to minimize clustering and improve the uniformity of probe sequences. In this method, two independent hash functions are employed: a primary hash function $h_1(k)$, which determines the initial index, and a secondary hash function $h_2(k)$, determines the step size (offset) used to move to the next candidate positions. The probe sequence is defined as

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m, \quad i = 0, 1, 2, \dots$$

where m is the table size. The first probe checks index $h_1(k)$, if that slot is occupied, the algorithm performs the second probe $(h_1(k) + h_2(k)) \bmod m$ and if still occupied, the third probe is $(h_1(k) + 2 \cdot h_2(k)) \bmod m$ and so on. For double hashing to be effective, the secondary hash function must satisfy certain properties: it must never evaluate to zero, and its values should be relatively prime to m , ensuring that the probe sequence can traverse the entire table without repetition. This is elaborated further in the following discussion.

Case 1: Relatively prime (desired case) Let the table size be $m = 10$, and let $h_2(k) = 3$. Since $\gcd(3, 10) = 1$, the two values are relatively prime. Now observe the probe sequence (ignoring $h_1(k)$ for simplicity and starting at 0; 0, 3, 6, 9, 2, 5, 8, 1, 4, 7. All values are computed modulo 10. In this case, every index from 0 to 9 is visited exactly once before the sequence begins to repeat. This implies that the probe sequence covers the entire table, ensuring that no slot is skipped during the probing process.

Case 1: Not relatively prime (problematic case) Now let $m = 10$ and $h_2(k) = 4$. Here, $\gcd(4, 10) = 2 \neq 1$, so they are not relatively prime. The probe sequence becomes 0, 4, 8, 2, 6, 0, 4, 8, In this case, only the indices $\{0, 2, 4, 6, 8\}$ are visited by the probe sequence, meaning that only a subset of the table is explored.

As a result, approximately half of the table—specifically the odd indices—is never examined. Consequently, even if empty slots exist at positions such as 1, 3, or 5, they will never be reached during probing.

Thus, requiring $h_2(k)$ to be relatively prime to m ensures full coverage of the table, prevents short cycles, and guarantees the correctness and efficiency of the probing process.

A key advantage of double hashing is its ability to significantly reduce both primary clustering and secondary clustering, which are common in linear and quadratic probing, respectively. Since the step size depends on the key itself, different keys that collide at the same initial index are likely to follow distinct probe sequences, resulting in a more uniform distribution of elements across the table. Under the assumption of uniform hashing, double hashing achieves expected constant-time complexity $O(1)$ for insertion, search, and deletion, even at moderately high load factors. Empirical and theoretical analyses have shown that its performance closely approximates that of ideal uniform probing, making it one of the most effective practical strategies for open addressing [3] [4].

However, the effectiveness of double hashing depends on the careful design of both hash functions. Poorly chosen functions may lead to short cycles or uneven distribution, thereby degrading performance. Additionally, as with all open addressing schemes, performance deteriorates as the load factor approaches unity, necessitating periodic rehashing to maintain efficiency. Despite these considerations, double hashing remains a widely studied and practically relevant technique due to its strong theoretical properties and its ability to approximate random probing behavior without requiring true randomness.

Collision Resolution in C++ Unordered Containers

In practice, C++ unordered containers employ a collision resolution strategy that is functionally equivalent to separate chaining, wherein elements with identical hash values are stored together and accessed collectively.

The Standard Template Library does not mandate a specific internal data structure for storing elements. The standard specifies only the required behavior and complexity guarantees, leaving the implementation details to library vendors. In practice, however, most mainstream implementations (such as libstdc++ and libc++) use node-based structures that are similar to linked lists to store elements that share the same hash value. Each element is typically allocated as a separate node, and these nodes are connected in a manner that allows efficient insertion and deletion without requiring relocation of other elements.

6.6 Hash Table Implementations

In modern C++, hashing behavior in standard containers is defined primarily through the hash policy of the C++ Standard Library, particularly in the associative containers `std::unordered_map` and `std::unordered_set`. These containers implement hash tables and rely on a hash function provided by the function object `std::hash`.

The C++ standard does not mandate a specific hashing algorithm. Instead, it specifies an interface: a hash function must map a key to a value of type `size_t` such that equal keys produce identical hash values. The exact algorithm used to compute this hash value is implementation-defined, meaning it varies across different standard library implementations. This design allows library vendors to optimize hashing strategies according to platform architecture, compiler design, and performance considerations.

In practice, several major C++ standard library implementations are widely used:

- GNU libstdc++ (used by GCC)
- LLVM libc++ (used by Clang)
- MSVC STL (used by Microsoft Visual C++)

Although these implementations differ internally, they follow broadly similar principles in their hashing mechanisms.

6.6.1 Hashing of Integer Data Type

For fundamental types such as integers, the hash function provided by `std::hash` is typically very lightweight. In many implementations, the hash value is produced either by an identity mapping or by a near-identity transformation (Stroustrup, 2013; cppreference.com). In practice, integer hashing often returns the integer value converted to `size_t`, possibly combined with inexpensive bit-mixing operations intended to reduce trivial patterns on certain architectures.

This approach is computationally efficient because integer values already vary across their binary representation, and additional heavy transformations would introduce unnecessary computational overhead.

Identity Mapping

An identity hash function returns the key itself (possibly after type conversion). No transformation is applied to the input value. Formally

$$h(k) = k$$

The following code demonstrates how to compute a hash value for an integer using the C++ standard library.

Code Example 6-2: Identity hash example

```

1 #include <iostream>
2 #include <functional>
3
4 int main()
5 {
6     std::hash<int> hasher;
7
8     int key = 42;
9     size_t h = hasher(key);
10
11     std::cout << "Hash value: " << h << std::endl;
12 }

```

Line 2: The header `<functional>` defines several function objects used by the C++ standard library, including `std::hash`, which is a hashing functor used to compute hash values for various data types.

Lines 4–9: Inside the main function, the statement `std::hash<int> hasher;` creates an instance of the hash function object for integers. This object is capable of computing a hash value for values of type `int`. The variable `key` is then declared and initialized with the value `42`, which is an arbitrary value used solely for demonstration purposes. The program computes the hash value by invoking the hashing object as if it were a function: `hasher(key)`. This call applies the integer hash function to the value stored in `key`. The resulting hash value is stored in the variable `h`, which is declared with the type `size_t`. The type `size_t` is commonly used to store hash values because it is an unsigned integer type capable of representing sizes and indices in memory.

Line 11: When the program runs, it outputs the hash value corresponding to the integer 42. In many implementations, the hash function for integers may simply return the integer value itself (possibly converted to `size_t`), although the exact behavior depends on the specific C++ library implementation. This example illustrates how the standard hashing mechanism in C++ can be used to generate hash values for integer keys.

**What is `std::hash`?**

`std::hash<T>` is a function object (functor) defined in the C++ standard library in `<functional>`. A functor is a class that overloads the function-call operator `operator()`, allowing an object of that class to be used like a function. Further explanation can be found in Appendix 8.3.1.

Near-identity hash function

A near-identity hash function is a hashing scheme in which the computed hash value closely resembles the original key. In such functions, the transformation applied to the input is minimal, and the resulting hash value preserves much of the numeri-

cal structure of the key. Formally, a near-identity hash function can be described as $h(k) \approx k$ or more generally $h(k) = f(k)$, where $f(k)$ represents a simple transformation of the key, such as truncation, masking, or a modulus operation. Unlike hash functions that incorporate substantial mixing or diffusion, near-identity hashes apply only a small reduction step to map the key into the desired range.

A common example arises when integers are used directly as hash values and subsequently reduced to fit within the bounds of a hash table. Consider a table of size $m = 10$ and a hash function defined as

$$h(k) = k \bmod 10 \tag{6.29}$$

In this case, the hash value corresponds to the last digit of the integer key. For example, $23 \bmod 10 = 3$, $45 \bmod 10 = 5$, $67 \bmod 10 = 7$, $81 \bmod 10 = 1$

The transformation applied to the key is minimal; the hash function simply extracts a portion of the original value. Consequently, the resulting index remains strongly correlated with the input key, illustrating the near-identity nature of the mapping.

Another example appears in implementations where the hash table size is a power of two. In such cases, the index may be computed using a bitwise masking operation.

The following table presents a comparison between identity mapping and near-identity hashing, highlighting their key characteristics, operational differences, and practical implications in hash table design.

TABLE 6.1 Comparison of identity mapping and near-identity hashing

Property	Identity Mapping	Near-Identity Hashing
Transformation	No transformation is applied to the key.	A minimal transformation (such as modulus or bit masking) is applied to the key.
Output Relation to Input	The output is exactly equal to the input value.	The output remains closely related to the input but is slightly modified.
Computational Cost	Minimal; the function simply returns the key.	Minimal; requires a small arithmetic or bitwise operation.
Range Enforcement	Does not guarantee that the result lies within the table bounds.	Ensures that the resulting value falls within the valid index range of the hash table.

Diffusion of Input Bits	No diffusion; the input structure is completely preserved.	Very limited diffusion; only minor portions of the key are retained.
Typical Usage	Rarely used directly in practical hash tables.	Commonly used as the reduction step in practical hash table implementations.
Example	$h(k) = k$	$h(k) = k \bmod m$ or $h(k) \& (m - 1)$

6.6.2 Hashing of String Data Type

In the preceding sections, hashing techniques for integer keys and polynomial string hashing were discussed. Polynomial hashing offers a conceptual framework for converting a sequence of characters into a numerical value while preserving the positional contribution of each character. Although this approach is useful for illustrating the principles of string hashing, practical software systems typically employ highly optimized general-purpose hash functions that are designed to achieve an effective balance between computational efficiency, quality of distribution, and robustness against adversarial inputs.

The C++ standard does not mandate a specific hashing algorithm for strings. Instead, it only requires that implementations produce a deterministic hash value with sufficiently good distribution properties. This design choice allows library implementers flexibility in selecting algorithms that best suit their performance and security requirements.

Consequently, different compilers and standard library implementations may adopt different hashing strategies depending on their design goals and performance considerations. In practice, several widely known non-cryptographic and security-oriented hash functions are commonly used for hashing strings and other byte sequences. Examples of such approaches include the following:

- Fowler–Noll–Vo (FNV)
- MurmurHash variants
- CityHash and FarmHash
- SipHash

Detailed discussions of hashing techniques can be found in standard algorithm texts such as *Introduction to Algorithms* by Cormen et al. [3] and *The Art of Computer Programming, Volume 3* by Knuth [4]. These algorithms are not discussed further in this book.

6.6.3 *Unordered Associative Containers in C++*

The C++ Standard Template Library (STL) provides several associative containers designed for efficient data storage and retrieval. These containers can be broadly classified into two categories: ordered associative containers and unordered associative containers. The ordered containers `std::map`, `std::multimap`, `std::set`, and `std::multiset` store elements in sorted order using balanced binary search trees (typically implemented as Red-Black trees), which results in logarithmic time complexity for insertion, deletion, and lookup operations. In contrast, the unordered associative containers `std::unordered_map`, `std::unordered_multimap`, `std::unordered_set` and `std::unordered_multiset` are implemented using hash tables. Because hash tables form the foundation of unordered associative containers, the remainder of this chapter focuses on hash-based containers.

These containers are defined in the `<unordered_map>` and `<unordered_set>` headers and internally rely on hash tables to organize data. Because the position of an element is determined by the hash of its key rather than its order, elements are not stored in sorted sequence.

`std::unordered_map`

This is an associative container that stores elements as key-value pairs, where each key is unique and maps to a corresponding value. The container uses a hash function to compute an index into an array of buckets, where the element is stored. Since the container enforces unique keys, duplicate keys are not allowed. If an insertion operation is attempted with a key that already exists in the container, the new element is not inserted, and the existing key-value pair remains unchanged. However, if the value is assigned using the subscript operator `operator[]`, the value associated with the existing key is updated (overwritten) rather than creating a duplicate entry.

The following example demonstrates the basic usage and syntax of `std::unordered_map`. A complete list of its member functions is provided in Appendix 8.3.

Code Example 6-3: Example of unordered map

```

1 #include <iostream>
2 #include <unordered_map>
3 #include <string>
4
5 int main()
6 {
7     std::unordered_map<std::string, int> scores;
8
9     scores.insert({"Alice", 90});
10    scores.insert({"Bob", 85});
11    scores.insert({"Cat", 88});
12
13    scores.insert({"Alice", 95});    // duplicate key not inserted

```

```

14     scores.insert({"Bob", 92});           // duplicate key not inserted
15
16     scores["Alice"] = 95;                 // updates the value associated with
    ↪ "Alice"
17
18     std::cout << "Contents of the unordered_map:\n";
19
20     for (auto it = scores.begin(); it != scores.end(); it++)
21     {
22         std::cout << it->first << " : " << it->second << std::endl;
23     }
24
25     return 0;
26 }

```

Line 2: The header `<unordered_map>` is included. This header provides the definition of `std::unordered_map`, a hash-table-based associative container that stores elements as key-value pairs.

Line 7: An object named `scores` is declared as `std::unordered_map<std::string, int>`. This means that the container stores elements in the form of key-value pairs, where each key is a string and each associated value is an integer.

Lines 9–11: The statement `scores.insert("Alice", 90);` inserts the key-value pair "Alice" and 90 into the container. Similarly, the statements inserting "Bob" with 85 and "Cat" with 88 add additional key-value pairs to the container. After these operations, the container contains three distinct entries.

Lines 13–14: The statement `scores.insert("Alice", 95);` attempts to insert another element with the key "Alice". Since `std::unordered_map` requires all keys to be unique, this insertion fails, and the existing pair associated with "Alice" remains unchanged. Similarly, the statement `scores.insert("Bob", 92);` attempts to insert a duplicate key "Bob", and this insertion also fails for the same reason.

Line 16: The statement `scores["Alice"] = 95;` accesses the element associated with the key "Alice" using the subscript operator `[]` and assigns it a new value. Because the key "Alice" already exists in the container, the old value 90 is replaced with 95. This line demonstrates an important distinction: unlike `insert()`, which does not overwrite an existing key, the subscript operator can be used to update the value associated with an existing key.

Line 20: A for loop is introduced to iterate through the elements of the container. The iterator `it` is initialized to `scores.begin()`, which points to the first element in the map. The loop continues as long as `it != scores.end()`, meaning that the iterator has not yet reached the position past the last element. The expression `it++` advances the iterator to the next element during each iteration.

Line 22: Prints the key and value of the element currently pointed to by the iterator. An iterator behaves similarly to a pointer to an element stored in the container. The arrow operator `(->)` is used to access the members of the element pointed to by the iterator. Since elements in `std::unordered_map` are stored as key-value pairs, the expression `it->first`

accesses the key, while `it->second` accesses the associated value. Consequently, each iteration of the loop prints one key–value pair stored in the container.

In summary, the program inserts several key–value pairs into an `std::unordered_map`, ignores attempts to insert duplicate keys, updates the value of an existing element, and then iterates through the container to display its contents. Since `std::unordered_map` is implemented using a hash table, the elements are not traversed in any predictable order.

`std::unordered_multimap`

This is a hash-table–based associative container provided by the C++ Standard Template Library. Similar to `std::unordered_map`, it stores elements as key–value pairs, where each key is associated with a corresponding value. However, unlike `std::unordered_map`, the container allows multiple elements to have the same key. This capability makes `std::unordered_multimap` suitable for applications in which a single key may correspond to several values.

Internally, `std::unordered_multimap` organizes its elements using a hash table. A hash function is applied to the key to determine the bucket in which the element is stored. Because element placement depends on the hash value rather than a sorted ordering, the elements in the container are not stored in any particular order. As a result, the container provides average constant-time complexity for insertion, lookup, and deletion operations, assuming that the hash function distributes keys uniformly across the available buckets.

As discussed earlier in the section on hash collisions, multiple elements may map to the same bucket when their keys produce identical hash values. In such cases, the container handles the collision using the technique of separate chaining. In this approach, each bucket maintains a collection of elements whose keys produce the same hash value. These elements are typically organized using a linked structure, allowing multiple entries to be stored within the same bucket. When operations such as search, insertion, or deletion are performed, the container first identifies the appropriate bucket using the hash function and then traverses the elements within that bucket to locate the desired key. As in a typical linked list, the elements within a bucket are connected through links or pointers and are not necessarily stored in contiguous memory locations. Consequently, the nodes that form the chain may reside in different locations in memory even though they are logically connected through the linked structure.

Since duplicate keys are permitted, operations that retrieve elements associated with a specific key may return multiple matching entries. Functions such as `equal_range()` can be used to obtain the range of elements corresponding to a given key.

Code Example 6-4: Example of unordered multimap

```

1 #include <iostream>
2 #include <unordered_map>
3 #include <string>
4 using namespace std;
5
6 int main()
7 {
8     std::unordered_multimap<std::string, int> scores;
9
10    scores.insert({"Alice", 90});
11    scores.insert({"Dave", 85});
12    scores.insert({"Bob", 85});
13    scores.insert({"John", 85});
14    scores.insert({"Alice", 95});
15    scores.insert({"Alice", 88});
16
17    auto range = scores.equal_range("Alice");
18
19    for (auto it = range.first; it != range.second; it++)
20    {
21        std::cout << it->first << " : " << it->second << std::endl;
22    }
23
24    return 0;
25 }

```

Lines 10–15: Several key–value pairs are inserted into the container using the `insert()` function. Since `std::unordered_multimap` allows duplicate keys, all three “Alice” entries are stored in the container. Internally, the container uses a hash function to determine the bucket in which a key should be stored. Because the same key produces the same hash value, elements with identical keys typically map to the same bucket and are stored together using separate chaining, which conceptually resembles a linked structure.

Line 17: Retrieves all elements whose key is “Alice”. The function `equal_range()` returns an object of type `std::pair<iterator, iterator>`. The variable `range` is not an iterator; rather, it is an object that stores a pair of iterators representing the beginning and the end of the sequence of elements with the specified key. Consequently, the variable `range` contains two iterators. The first iterator, `range.first`, points to the first element whose key is “Alice”, while the second iterator, `range.second`, points to the position immediately after the last element whose key is “Alice”. It is important to note that elements in `std::unordered_multimap` are not stored in sorted order, nor are they guaranteed to appear in the same order in which they were inserted. Since the container is implemented using a hash table, elements are organized according to the hash values of their keys. Consequently, the element pointed to by `range.second` may be the next element in the container that does not match the key “Alice”, which could be “Bob”, “John”, “Dave”, or any other element depending on the internal bucket arrangement of the hash table.

Line 19: In this line, a for loop is used to iterate through this range of elements. The iterator

variable `it` is initialized with `range.first`, meaning the loop begins at the first “Alice” entry. The loop continues as long as the iterator is not equal to `range.second`. During each iteration, the expression `it++` advances the iterator to the next element in the container.

`std::unordered_set`

This is a hash-based associative container that stores unique elements without associated values. In contrast to `std::unordered_map` and `std::unordered_multimap`, which stores elements as key-value pairs, `std::unordered_set` maintains only the keys themselves. Since the container enforces uniqueness, attempts to insert duplicate elements are ignored.

Code Example 6-5: Example of unordered set

```

1 #include <unordered_set>
2 #include <iostream>
3
4 std::unordered_set<int> numbers;
5
6 numbers.insert(10);
7 numbers.insert(20);
8 numbers.insert(30);
9
10 if (numbers.find(20) != numbers.end())
11 {
12     std::cout << "Element found\n";
13 }
```

Line 4: The statement `std::unordered_set<int> numbers;` declares an unordered set named `numbers` that stores elements of type `int`. The container is implemented as a hash table and ensures that all elements stored in it are unique.

Lines 6–8: Insert the integers 10, 20, and 30 into the set. Since `std::unordered_set` does not allow duplicate elements, each value is stored only once.

Lines 10–12: Checks whether the element 20 exists in the set. The member function `find()` searches for the specified value and returns an iterator to the element if it is found; otherwise, it returns `numbers.end()`, which represents the position past the last element in the container. If the element is found, the condition evaluates to true and prints the message “Element found” to the console.

`std::unordered_multiset`

The `std::unordered_multiset` is similar to `std::unordered_set`, but it allows duplicate elements. While an `std::unordered_set` stores only unique values, an `std::unordered_multiset` permits multiple elements with the same value to be

stored in the container. The `std::unordered_multiset` container is useful in situations where duplicate values need to be stored while maintaining efficient access through hash-based storage, and where no particular ordering of elements is required.

Code Example 6-6: Example of unordered multiset

```

1 #include <iostream>
2 #include <unordered_set>
3
4 int main()
5 {
6     std::unordered_multiset<int> values;
7
8     values.insert(5);
9     values.insert(5);
10    values.insert(5);
11
12    std::cout << values.count(5);
13 }
```

The `count()` function returns the number of elements with the specified key.

6.6.4 Time complexity of Unordered Associative Containers

The primary goal of unordered associative containers is to achieve efficient average-time performance for common operations such as insertion, deletion, and lookup. In a hash table, a hash function maps a key to an index known as a bucket within an underlying bucket array. Multiple elements may be assigned to the same bucket if their keys produce the same hash value; this event is referred to as a collision. In most implementations, collisions are handled using separate chaining, where elements that map to the same bucket are stored in a linked structure associated with that bucket.

Under typical conditions—assuming a well-designed hash function and an appropriate load factor—unordered associative containers achieve constant average-time complexity for their primary operations. Specifically, insertion (via `insert` or `emplace`), lookup (via `find`, `count`, or `contains`), and deletion (via `erase`) all operate in $O(1)$ time on average. This efficiency arises because the computation of the hash function directly identifies the relevant bucket, after which only a small, localized search within that bucket is required. Consequently, the performance of these operations remains largely independent of the total number of elements stored in the container.

Although the average-case performance is constant, the worst-case time complexity for these operations is $O(n)$, where n is the number of elements in the container. This situation arises when many elements are mapped to the same bucket due to excessive collisions or a poorly distributed hash function. In such cases, the bucket effectively behaves like a linear structure that must be traversed sequentially. Modern standard library implementations attempt to minimize this possibility by dynamically adjusting

the number of buckets through rehashing.

As discussed previously, the performance of unordered associative containers is governed by the load factor, defined as the ratio of the number of stored elements to the number of buckets. A higher load factor increases the likelihood of collisions, thereby increasing the number of elements within individual buckets and potentially degrading performance. To maintain efficiency, the container may initiate rehashing, which expands the number of buckets and redistributes all elements according to the hash function. Although rehashing has a linear cost $O(n)$, it occurs infrequently, ensuring that the overall average-time complexity of operations remains constant.

6.7 Hash Table Applications

Hash tables are among the most widely used data structures in modern computing due to their ability to support efficient average-case constant-time insertion, deletion, and lookup operations. Their versatility allows them to be applied across a broad spectrum of domains, ranging from low-level system design to high-level data analysis. In practice, hash tables are often employed whenever fast access to data based on a key is required, making them a fundamental component in many real-world software systems such as:

Symbol Tables in Compilers: Used to store and retrieve information about identifiers such as variables, functions, and classes during program compilation.

Database Indexing and Key-Value Stores: Enable fast retrieval of records based on keys, forming the foundation of many NoSQL databases and in-memory storage systems.

Caching Mechanisms: Support rapid data access in systems such as web caches and memory caches, often combined with eviction policies like Least Recently Used (LRU).

Duplicate Detection and Set Membership: Efficiently determine whether an element has already been encountered, commonly used in data processing and search algorithms.

Frequency Counting and Data Analysis: Used to count occurrences of elements, such as words in a document or events in a dataset.

Routing Tables and Networking: Facilitate fast lookup of routing information in network devices and protocols.

Despite the numerous advantages of hashing, it is important to recognize that it also has certain limitations. Hashing is not well suited for applications in which multiple records with identical key values must be efficiently managed. While certain hash-based structures may permit duplicates, they do not provide an inherently structured mechanism for organizing or querying such records in a meaningful way.

Furthermore, hashing is unsuitable for range-based queries. It does not support efficient retrieval of all records whose key values lie within a specified interval, nor does

it facilitate operations such as finding the minimum or maximum key or traversing records in sorted order. These limitations arise because hashing destroys the inherent ordering of keys, making ordered access operations inefficient or impractical.

The following example illustrates how hashing is employed to efficiently transform high-dimensional data into a fixed-size representation, which is subsequently used by logistic regression for classification tasks.

Application: Signed Feature Hashing with Logistic Regression

In many real-world machine learning applications, particularly those involving textual data, the feature space is extremely high-dimensional and sparse. Logistic regression is widely used in such settings due to its computational efficiency, scalability, and ability to produce probabilistic predictions. It serves as a fundamental model in applications such as sentiment analysis, spam detection, click-through rate prediction, document classification, and fraud detection.

To enable efficient learning in these high-dimensional environments, it is common to employ a two-stage pipeline. First, signed feature hashing is applied as a preprocessing step to map raw input data into a fixed-dimensional numerical vector, thereby reducing memory requirements while preserving useful feature information. This transformed representation is then used by logistic regression to learn model parameters and perform classification.

Before illustrating a concrete example, it is important to understand the sequence of steps involved in combining signed feature hashing with logistic regression for classification tasks.

Step 1: Input Representation (Text Data)

The process begins with raw input data, typically in the form of unstructured text (e.g., a review or sentence). This data must first be converted into a structured form suitable for numerical processing.

Step 2: Tokenization

The input text is decomposed into smaller units called tokens (e.g., words). These tokens serve as the fundamental features of the input.

$$\text{Text} \rightarrow [\text{token}_1, \text{token}_2, \dots, \text{token}_k]$$

Step 3: Signed Feature Hashing (Preprocessing)

Each token is mapped into a fixed-dimensional feature vector using two hash functions: The first hash function determines the index.

$$\text{index} = h_1(\text{token}) \bmod N \quad (6.30)$$

where, h_1 is the first hash function, and N denotes the dimensionality of the hashed feature vector, i.e., the total number of hash buckets (or indices) available.

The second hash function determines the sign.

$$\text{sign} = \begin{cases} +1 & \text{if } h_2(\text{token}) \text{ is even} \\ -1 & \text{if } h_2(\text{token}) \text{ is odd} \end{cases} \quad (6.31)$$

The feature vector $x \in \mathbb{R}^N$ is constructed by accumulating signed contributions

$$x[\text{index}] = x[\text{index}] + \text{sign}$$

where, x is a vector of length N whose elements are real numbers, $\mathbb{R}$ is the set of real numbers and $\mathbb{R}^N$ is the set of all vectors with N real-valued components

Step 4: Linear Combination (Logistic Regression Input)

The hashed feature vector x is used as input to logistic regression. The model computes a weighted sum

$$z = w \cdot x = \sum_{i=1}^N w_i x_i \quad (6.32)$$

where w denotes the model parameters, w_i is the weight associated with the i -th feature (hash bucket), and z is the result of the dot product, representing the unscaled prediction that serves as input to the sigmoid function.

The expression $z = w \cdot x$ represents the linear combination of input features, where each feature value x_i is weighted by its corresponding parameter w_i . The resulting scalar z serves as the input to the sigmoid function for probabilistic classification.

Step 5: Sigmoid Transformation

The linear score z is transformed into a probability using the sigmoid function.

$$P(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (6.33)$$

where, y represents the target variable (label) in logistic regression.

Step 6: Classification Decision

A threshold (commonly 0.5) is applied to determine the predicted class.

$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(z) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Let us now apply the theoretical concepts in practice. Consider the following textual input.

“good product but bad packaging”

The objective is to classify this review into one of two categories: positive (1) or negative (0) using signed feature hashing as a preprocessing step, followed by logistic regression.

The input text is decomposed into the following tokens.

["good", "product", "but", "bad", "packaging"]

For this example, let the dimensionality of the hashed feature space be $N = 6$. Apply the two hash functions defined in Equations 6.30 and 6.31, and assume the following results are obtained.

Token	$h_1(\text{token}) \bmod 6$	$h_2(\text{token})$ parity	Sign	Contribution
good	1	even	+1	+1
product	4	even	+1	+1
but	1	odd	-1	-1
bad	3	odd	-1	-1
packaging	4	even	+1	+1

Initialize the feature vector

$\mathbf{x} = [0, 0, 0, 0, 0, 0]$

Now accumulate the signed contributions:

“good” maps to index 1 with sign +1

$\mathbf{x} = [0, 1, 0, 0, 0, 0]$

“product” maps to index 4 with sign +1

$\mathbf{x} = [0, 1, 0, 0, 1, 0]$

“but” maps to index 1 with sign -1

$\mathbf{x} = [0, 0, 0, 0, 1, 0]$

“bad” maps to index 3 with sign -1

$\mathbf{x} = [0, 0, 0, -1, 1, 0]$

“packaging” maps to index 4 with sign +1

$\mathbf{x} = [0, 0, 0, -1, 2, 0]$

Thus, the final hashed feature vector is

$\mathbf{x} = [0, 0, 0, -1, 2, 0]$

The weights are learned values that indicate how important each feature is in making a prediction. These parameters are obtained during the training phase of logistic regression through an optimization process (e.g., minimizing a loss function). A detailed treatment of how these weights are computed is beyond the scope of this book. For the purpose of illustration, assume that logistic regression has learned the following weight vector:

$\mathbf{w} = [0.1, 0.2, -0.1, -0.7, 0.8, 0.0]$

Using the Equation 6.32, substitute the values:

$z = (0.1)(0) + (0.2)(0) + (-0.1)(0) + (-0.7)(-1) + (0.8)(2) + (0.0)(0)$

$z = 0 + 0 + 0 + 0.7 + 1.6 + 0 = 2.3$

Using the Equation 6.33, substitute $z = 2.3$ gives ≈ 0.91

Using a threshold of 0.5, $0.91 \geq 0.5$, therefore, the review is classified as positive.

A threshold of 0.5 is commonly used in logistic regression because it corresponds to selecting the class with the higher predicted probability. However, the choice of

threshold may vary depending on the application, as different tasks may require balancing false positives and false negatives differently.

Although the review contains the negative token “bad”, its effect is outweighed by the positive contributions associated with “product” and “packaging”. In addition, the tokens “good” and “but” map to the same index, but because they receive opposite signs, their contributions cancel each other. This illustrates how signed feature hashing reduces the bias that may arise from collisions.

The above example demonstrates how signed feature hashing converts textual input into a compact numerical representation before logistic regression is applied. The preprocessing stage maps features into a fixed-dimensional space and assigns signed contributions, while logistic regression uses the resulting feature vector to compute a probability and produce a final classification decision.

The complete implementation can be found in Appendix 8.3.3.

Concept Check

■ Why must a hash function be deterministic in a hash table?

A hash function must be deterministic so that the same key always maps to the same index. This ensures that a key inserted into the table can be correctly located during lookup; otherwise, retrieval would fail.

■ Why is uniform distribution important for a hash function?

Uniform distribution ensures that keys are spread evenly across all buckets, minimizing collisions and maintaining efficient average-case performance for insertion, deletion, and lookup operations.

■ Why does the assumption of Simple Uniform Hashing (SUH) not hold in practice, and why is it still useful?

SUH assumes independent, uniformly random mapping of keys, which real deterministic hash functions cannot guarantee. However, it serves as a theoretical model that enables tractable average-case analysis and guides the design of practical hash functions.

■ Why does increasing the number of buckets m not always reduce collisions in practice?

While theoretically the expected number of collisions decreases as $\frac{1}{m}$, this assumes uniform hashing. If the hash function is poorly designed or correlated with input patterns, increasing m alone does not improve distribution. Collisions may persist or even worsen due to structural biases in the hash function.

■ Can hashing techniques designed for strings also be applied to integer keys?

Although general-purpose hash functions such as FNV, MurmurHash, and SipHash can be applied to integer values by processing their byte representation, integer keys typically require only simple hashing strategies. In many implementations, identity hashing or lightweight mixing functions are sufficient to achieve an adequate distribution of integer keys.

■ Can a bucket in a hash table store more than one element, or is it limited to holding a single element?

No. A bucket in a hash table is not restricted to storing a single element. Multiple elements may be placed in the same bucket when different keys produce the same bucket index, a situation known as a hash collision. The number of elements that can be stored in a bucket is not fixed; a bucket may contain zero, one, or many elements, depending on how many keys map to that bucket.

■ **Why can separate chaining tolerate load factors greater than 1, while open addressing cannot?**

Separate chaining stores multiple elements per bucket using auxiliary structures, so buckets can grow arbitrarily. In contrast, open addressing stores elements directly in the array, and once the table becomes nearly full, probing sequences become excessively long, degrading performance.

7

Searching algorithms

Linear Search Algorithm	165
Binary Search Algorithm	167

Searching algorithms are techniques used to find a specific value inside a collection of data, such as an array or a vector. Imagine having a list of numbers or names and wanting to determine whether a particular item exists or where it is located. A searching algorithm defines the systematic way in which a computer examines that data. The simplest approach is to check each element one by one until a match is found, which works on any dataset but can become inefficient for large collections. More efficient searching methods take advantage of how the data is organized; for example, when data is sorted, a search can skip large portions of the dataset and complete much faster. In practice, searching algorithms play a critical role in enabling programs to retrieve information quickly and efficiently, making them a fundamental building block in problem solving and software development.

Scope of the Chapter

At the outset, it is important to note that the searching algorithms discussed in this chapter are primarily intended for data at rest, where the dataset remains stable during the search process. These algorithms assume that the collection does not change frequently while searches are being performed. When data is continuously inserted, deleted, or updated, maintaining the assumptions required by these algorithms can introduce additional overhead, making them less suitable in such dynamic environments. In applications involving constantly changing data, alternative data structures and search strategies are typically more appropriate. Such algorithms and data structures designed for dynamic environments will be discussed in the next chapter.

Before analyzing specific algorithms, it is important to understand how algorithm performance is measured. Three fundamental concepts are commonly used to evaluate and compare algorithms: time complexity, space complexity, and Big-O notation. Together, these concepts describe how an algorithm's execution time and memory usage grow as the size of the input increases.

Time complexity measures how the running time of an algorithm increases as the size of the input grows. Instead of focusing on actual execution time in seconds, time complexity counts the number of basic operations performed by the algorithm relative to the input size. This approach allows algorithms to be compared independently of hardware speed, programming language, or system environment.

Time complexity

Space complexity measures the amount of additional memory an algorithm requires as the input size increases. It focuses on the extra memory used by the algorithm during execution, such as temporary variables, auxiliary data structures, or recursion stacks, rather than the memory needed to store the input itself.

Space complexity

Big-O notation is a mathematical way of expressing time and space complexity. It describes the upper bound on an algorithm's growth rate as the input size becomes large. Big-O notation focuses on the dominant term and ignores constants and lower-order terms, allowing algorithms to be compared based on their scalability. For example,

Big-O notation

Quartic Growth (Power of 4): If an algorithm performs $5n^4 + 2n^2 + 100n + 50$ operations, the term with the highest power of n , namely n^4 , dominates the growth of the function. As the input size n increases, the effects of lower-order terms and constant values become negligible in comparison. Consequently, the algorithm is classified as having a time complexity of $O(n^4)$, meaning that the number of operations increases in proportion to the fourth power of the input size.

Cubic Growth (Power of 3): If an algorithm performs $7n^3 + 4n^2 + 20n + 15$, the n^3 term dominates the growth rate, and the algorithm is classified as $O(n^3)$.

Quadratic Growth (Power of 2): If an algorithm performs $3n^2 + 10n + 25$, the dominant term is n^2 , so the time complexity is $O(n^2)$.

Linear Growth (Power of 1): If an algorithm performs $6n + 40$ operations, the dominant term is n . As the input size n increases, the constant term becomes insignificant in comparison. Therefore, the algorithm is classified as $O(n)$, which indicates that the number of operations grows linearly with the input size.

Constant Growth ($O(1)$): If an algorithm performs a fixed number of operations, such as 12, regardless of the input size n , its running time remains constant as n increases. Because the execution time does not depend on the size of the input, the algorithm is classified as $O(1)$, which is known as constant-time complexity.

This chapter explores several commonly used searching algorithms, discussing how each one works, along with its advantages and limitations. The performance of these algorithms is analyzed using best-case, average-case, and worst-case scenarios, allowing readers to understand when a particular searching technique is most appropriate and how algorithm choice impacts efficiency.

7.1 Linear Search Algorithm

Linear search, also known as sequential search, is the simplest searching algorithm. It works by examining each element in a collection one by one, starting from the beginning and continuing until the target element is found or the end of the data is reached. Because linear search makes no assumptions about how the data is organized, it works on both sorted and unsorted datasets and is easy to understand and implement. However, this simplicity comes at a cost. For large collections, linear search can be slow because it may need to examine every element in the dataset. In addition, when duplicate elements are present, a standard linear search typically finds only the first occurrence of the target value; to find all occurrences or determine how many times the value appears, the algorithm must examine every element in the dataset. More efficient searching algorithms achieve better performance by using the structure of the data. For instance, when data is sorted, a search can skip many elements and complete much faster.

The best-case scenario for linear search occurs only when the problem is defined as finding any single occurrence of the target element and the target appears at the first

position in the dataset. In this situation, the algorithm performs just one comparison, resulting in a time complexity of $O(1)$.

However, when the dataset contains duplicate elements and the search task requires identifying all occurrences, counting duplicates, or ensuring completeness, linear search cannot terminate early. In such cases, the algorithm must examine every element in the dataset, and a constant-time best case does not exist; even under the most favorable conditions, the time complexity remains $O(n)$.

i Case assumptions Best-case, average-case, and worst-case analyses are always based on specific assumptions. When discussing these cases, be clear about what the algorithm is allowed to do and what the problem requires (such as early termination or handling duplicates). Always ensure that your assumptions match the scenario you are describing.

The following program uses a linear search to scan a vector and collect the indices of all elements that match a given target value. It demonstrates how passing a vector by reference allows the function to store results directly in the caller's vector while efficiently handling duplicate elements.

Code Example 7-1: Linear search algorithm with duplicate member search

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void linearSearch(const vector<int>& v, vector<int>& d, int key) {
6      for (int i = 0; i < v.size(); i++) {
7          if (v[i] == key)
8              d.push_back(i);
9      }
10 }
11
12 int main() {
13     vector<int> data = {3, 8, 15, 23, 15};
14     vector<int> data_index;
15     linearSearch(data, data_index, 15);
16     for (int x : data_index) {
17         cout << "Index: " << x << endl;
18     }
19 }

```

Lines 1-3: Include the `<iostream>` and `<vector>` header files, which are required for input/output operations using `cout` and for using the `std::vector` container, respectively.

Line 5: Defines the `linearSearch` function, which performs the linear search operation. The first parameter, `const vector<int>& v`, represents the input dataset and is passed by

constant reference to prevent modification and avoid unnecessary copying. The second parameter, `vector<int>& d`, is passed by reference and acts as an alias to the vector supplied by the calling function, allowing the function to store results directly in that vector. The third parameter, `key`, specifies the value to be searched.

Lines 6-10: Implement a for loop that iterates through the vector `v` from the first element to the last. Each element is compared with the target value, and if a match is found, the current index is appended to vector `d`, enabling the function to record all positions where the target value appears.

Line 12: Marks the beginning of the `main` function, which is the entry point of the program.

Line 13: Initializes a vector named `data` with a set of integer values, including duplicates.

Line 14: Declares an empty vector named `data_index`, which will be used to store the indices of all matching elements.

Line 15: Calls the `linearSearch` function, passing `data` as the input vector, `data_index` as the output vector, and 15 as the target value. Because `data_index` is passed by reference, it is modified directly inside the function.

Lines 16-18: Use a range-based for loop to iterate through the indices stored in `data_index` and print each index to the console.

When working with unsorted data, linear search is often the most practical choice because it can be applied directly without any preprocessing. Although faster searching algorithms such as binary search exist, they require the data to be sorted first. Sorting a large dataset itself incurs additional computational cost, which may outweigh the benefits of faster searching if the search operation is performed only once or infrequently. Therefore, linear search is often preferred for unsorted data when the dataset is small or when the overhead of sorting is not justified. For large datasets with frequent search operations, investing time in sorting the data or using more advanced data structures can lead to better overall performance. These approaches and their performance trade-offs will be discussed in detail in the upcoming sections.

7.2 Binary Search Algorithm

The main motivation behind binary search is to search efficiently by reducing the number of comparisons needed to find an element. Instead of examining every item one by one, binary search takes advantage of sorted data to eliminate half of the remaining elements at each step. For example, if we want to find the number 144 in the sorted list [101, 115, 123, 130, 144, 150, 162, 170], a linear search may require checking many elements before reaching the target. Binary search, however, first compares the middle element (130) with 144, immediately discards the left half, and then finds 144 in the next comparison. This divide-and-conquer approach reduces the time complexity from $O(n)$ to $O(\log n)$, making binary search especially powerful for large datasets where performance and scalability matter.

In some textbooks, the time complexity of binary search is explicitly written as $O(\log_2 n)$ to emphasize that the algorithm repeatedly halves the search space at each step. However, in Big-O notation, the base of the logarithm is typically

omitted because logarithms with different bases differ only by a constant factor. Since Big-O analysis focuses on the asymptotic growth rate of an algorithm and ignores constant multipliers, $O(\log_2 n)$ and $O(\log n)$ are considered equivalent. For example, if an array contains 1,024 elements, binary search requires at most 10 comparisons because $\log_2 1024 = 10$. Using a different base, such as base 10, gives $\log_{10} 1024 \approx 3$, which differs only by a constant factor. Despite this numerical difference, both expressions describe the same logarithmic growth behavior, which is why the standard convention in algorithm analysis is to write $O(\log n)$.

Although binary search significantly reduces the number of comparisons relative to linear search, this improvement is achieved only under specific conditions. Binary search is applicable exclusively to data that is sorted according to a well-defined ordering; without this property, the algorithm cannot guarantee correct results. Furthermore, the data must be stored in a structure that supports efficient random access, such as an array or vector, allowing constant-time access to the middle element. A consistent comparison operation is also required to establish the ordering between elements. When these preconditions are satisfied, binary search operates by repeatedly dividing the search interval in half, thereby achieving logarithmic time complexity.

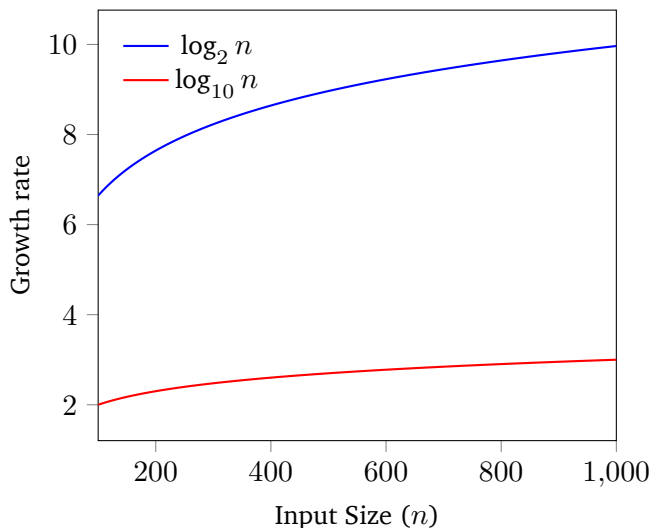


FIGURE 7.1 This figure compares the growth behavior of the logarithmic functions $\log_2 n$ and $\log_{10} n$ as the input size n increases. The x-axis represents the input size, while the y-axis represents the growth rate in terms of the number of operations. Both curves increase slowly, illustrating the efficiency of logarithmic growth, with $\log_2 n$ growing faster than $\log_{10} n$ by a constant factor. The similar shapes of the two curves emphasize that logarithmic functions differ only by a constant multiplier, which is why logarithmic time complexity is expressed as $O(\log n)$ without specifying the base.

Binary search works by systematically eliminating half of the remaining possibilities

at each step. Instead of scanning elements one by one, the algorithm compares the target value with the element located at the middle of the search range.

When applying binary search, the method for identifying the middle element depends on whether the number of elements in the search range is odd or even. If the range contains an odd number of elements, there is a single, well-defined middle element, which is selected directly. In contrast, when the number of elements is even, no unique middle element exists; instead, two central candidates are present. In this case, binary search resolves the ambiguity by consistently selecting one of these candidates—most commonly the lower middle—using integer division when computing the midpoint. Importantly, the choice of lower or upper middle does not affect the correctness or efficiency of the algorithm, provided the selection rule is applied consistently. In both cases, the selected middle element allows binary search to partition the data into two smaller subranges, ensuring that the search space is reduced at each step and preserving the algorithm’s logarithmic time complexity.

If the target is equal to the middle element, the search is complete. If the target is smaller, the algorithm ignores the entire right half of the data; if the target is larger, it ignores the entire left half. Because the data is sorted, these discarded halves are guaranteed not to contain the target.

This process repeats on the remaining half of the data, continually narrowing the search range. With each comparison, the search space is reduced by a factor of two. As a result, even for large datasets, binary search locates an element using very few comparisons, which explains its logarithmic efficiency.

Example (Even Number of Elements)

Consider the following sorted array with 8 elements: Suppose we want to search for 18.

Index:	0	1	2	3	4	5	6	7
Data:	3	7	10	14	18	21	25	30

Step 1: Choose the Middle Element

```
- Low index = 0
- High index = 7
- Middle index = (0 + 7) // 2 = 3 -> this is an integer division
- Middle element = 14
```

Since $18 > 14$, all elements to the left of index 3 (including 14) are discarded.

Step 2: Search the Right Half

Index:	4	5	6	7
Data:	18	21	25	30

```
- Low index = 4
```

```
- High index = 7
- Middle index = (4 + 7) // 2 = 5 -> this is an integer division
- Middle element = 21
```

Since $18 < 21$, all elements to the right of index 5 (including 21) are discarded.

Step 3: Final step

```
Index: 4
Data: 18
```

The target value 18 is found.

i Binary search computes the middle index using integer division because it operates on discrete index positions in an array or similar data structure, where indices must be integers and any fractional result is discarded. When the number of elements is odd, this calculation yields a unique middle index; when the number is even, it consistently selects one of the two central indices. Binary search does not require a unique middle element for correctness—what matters is that the chosen index allows the algorithm to eliminate part of the search space at each step. This deterministic selection ensures steady progress and preserves the correctness and logarithmic time complexity of the algorithm.

Binary search exhibits different performance characteristics depending on when the target element is found within the search process. In the best case, the target element is located at the middle position of the search range during the very first comparison. In this situation, binary search completes immediately, resulting in a time complexity of $O(1)$.

In the average case, the target element is found after several iterations, as the algorithm repeatedly halves the search space. On average, binary search requires a logarithmic number of comparisons relative to the size of the dataset, giving an average-case time complexity of $O(\log n)$.

In the worst case, the target element is either located at the deepest level of the search process or is not present in the data at all. Even in this scenario, binary search continues halving the search space until it is exhausted, leading to a worst-case time complexity of $O(\log n)$. This consistent logarithmic behavior across average and worst cases highlights the efficiency of binary search for large, sorted datasets.

Keep this in mind that binary search requires the data to be sorted before it can be applied, which introduces an additional preprocessing step. If the data is initially unsorted, it must first be arranged according to a well-defined ordering, and this sorting process incurs an extra computational cost. However, once the data is sorted, binary search can be applied efficiently and repeatedly, achieving logarithmic time complexity. In situations involving multiple search operations on the same dataset,

the one-time cost of sorting is often offset by the significant performance gains of optimized binary searches.

The following pseudocode presents a language-independent description of the binary search algorithm. It outlines the fundamental logic of the algorithm without relying on the syntax of any specific programming language, allowing the focus to remain on the underlying problem-solving approach.

Code Example 7-2: Pseudocode of binary search algorithm

```

BinarySearch(A, n, key) //A:Array, n:array length, key:element to find
    low ← 0
    high ← n - 1

    while low ≤ high do
        mid ← (low + high) / 2      // integer division

        if A[mid] = key then
            return mid              // key found
        else if A[mid] < key then
            low ← mid + 1           // search right half
        else
            high ← mid - 1          // search left half
        end if
    end while

    return -1                      // key not found

```

7.2.1 *Handling Duplicate Elements in Binary Search*

A standard binary search can only tell you whether a value exists and may return any one of its occurrences when duplicates are present. It cannot directly return all indices of repeated elements. When the data is sorted, duplicates naturally cluster together. The following example illustrates the binary search process in the presence of duplicate elements.

Example (Sorted array with duplicate members)

Index:	0	1	2	3	4	5	6
Array:	3	5	7	7	7	9	12
Key =	7						

Part A: Find the First Occurrence (Leftmost 7)

```

Step 1
- low = 0, high = 6
- mid = (0+6)/2 = 3
- A[mid] = A[3] = 7 (match found)

```

But we keep going left to find the first one.

- record ans = 3
- move left: $\text{high} = \text{mid} - 1 = 2$

Step 2

- low = 0, high = 2
- $\text{mid} = (0+2)/2 = 1$
- $A[\text{mid}] = A[1] = 5$

Since $7 > 5$, go right: $\text{low} = \text{mid} + 1 = 2$

Step 3

- low = 2, high = 2
- mid = 2
- $A[\text{mid}] = A[2] = 7$ (match)
- record ans = 2
- move left: $\text{high} = \text{mid} - 1 = 1$

Stop (low > high)

First occurrence = 2

Part B: Find the Last Occurrence (Rightmost 7)

Step 1

- low = 0, high = 6
- mid = 3
- $A[\text{mid}] = 7$ (match)

But we keep going right to find the last one.

- record ans = 3
- move right: $\text{low} = \text{mid} + 1 = 4$

Step 2

- low = 4, high = 6
- $\text{mid} = (4+6)/2 = 5$
- $A[\text{mid}] = A[5] = 9$

Since $7 < 9$, go left: $\text{high} = \text{mid} - 1 = 4$

Step 3

- low = 4, high = 4
- mid = 4
- $A[\text{mid}] = 7$ (match)
- record ans = 4
- move right: $\text{low} = \text{mid} + 1 = 5$

Stop (low > high)

Last occurrence = 4

Final Result: All Indices

```

Now we have:
- first = 2
- last = 4
So all occurrences are indices:
2, 3, 4

```

We have already discussed the Big-O notation of binary search in the general case. The following examines how the presence of duplicate elements influences that analysis and under what conditions the overall time complexity changes.

Let n denote the total number of elements in a sorted array, and let k denote the number of occurrences of a given key. When binary search is used solely to determine the presence of a key, the algorithm repeatedly halves the search space, resulting in a time complexity of

$$T(n) = O(\log n) \quad (7.1)$$

regardless of whether duplicate elements are present.

When duplicate elements exist and the objective is to locate a specific boundary—such as the first or last occurrence—the algorithm can be modified to continue searching after a match is found. Each such boundary search still requires $O(\log n)$ time, as the search space is reduced by a factor of two at each step.

However, if the objective is to identify all occurrences of the key, the analysis changes. After determining the first and last occurrence indices in $O(\log n)$ time, all duplicate elements within this range must be examined or reported. This additional step incurs a cost proportional to the number of duplicates, K . Therefore, the total time complexity becomes

$$T(n, k) = O(\log n + k) \quad (7.2)$$

If the number of duplicates is bounded by a constant, that is, $k = O(1)$, the complexity simplifies to $O(\log n)$.

In contrast, if the number of duplicates grows with the input size, the linear term dominates. In the worst case, where all elements are identical ($k = n$), the complexity reduces to

$$T(n, k) = O(\log n + n) = O(n) \quad (7.3)$$

This case-by-case analysis demonstrates that while binary search itself maintains logarithmic efficiency, retrieving all duplicate elements introduces an additional cost that depends directly on the number of duplicates.

The Table 7.1 summarizes how the time complexity of binary search varies when duplicate elements are present in the dataset and different search objectives are considered.

TABLE 7.1 *Time Complexity of Binary Search in the Presence of Duplicate Elements*

Scenario	Objective	Time Complexity
No duplicates	Find key in sorted array	$O(\log n)$
Duplicates present	Find any one occurrence	$O(\log n)$
Duplicates present	Find first occurrence	$O(\log n)$
Duplicates present	Find last occurrence	$O(\log n)$
Duplicates present	Find all occurrences	$O(\log n + k)$
Worst case ($k = n$)	All elements identical	$O(n)$

Table 7.2 provides a comparative overview of linear search and binary search, highlighting their performance characteristics under different scenarios.

TABLE 7.2 *Time Complexity Comparison of Linear Search and Binary Search*

Search Algorithm	Best Case	Average Case	Worst Case
Linear Search	$O(1)$ ¹	$O(n)$	$O(n)$
Binary Search	$O(1)$ ^{1,2}	$O(\log n)$	$O(\log n)$

Binary search is actively used today in many real-world systems whenever data is ordered and queried repeatedly. Some common practical applications include:

Standard library functions: Many programming languages provide built-in binary search utilities (for example, searching within sorted arrays or vectors). These are heavily used in performance-critical applications where fast lookups are required.

Searching time-ordered data: System logs, transaction histories, and monitoring data are often stored in sorted order by timestamp. Binary search is used to quickly locate the first or last record within a given time range.

Debugging and fault isolation: Developers use binary-search principles to isolate problems efficiently. A well-known example is narrowing down which change introduced a bug by repeatedly halving the range of possible causes.

Databases and indexing systems: Although databases rely on advanced index structures, the core idea of binary search—rapidly narrowing a sorted search space—underpins many index lookup operations.

¹The best-case time complexity of $O(1)$ assumes that the target element is uniquely identifiable and encountered immediately (for linear search) or at the first midpoint comparison (for binary search).

²Binary search requires the data to be sorted in advance. If the data is initially unsorted, an additional preprocessing step is required, typically with a time complexity of $O(n \log n)$. This sorting cost is often amortized when multiple search operations are performed on the same dataset.

The algorithms discussed so far rely on comparison-based techniques. In the next chapter, we explore hashing, a fundamentally different approach to searching that achieves constant-time average performance.



Appendix A

Vectors	178
Linked Lists	182
Hash Tables	183

8.1 Vectors

Case 1: Vector declared on the stack with initialized members

The following code is examined using GNU Debugger (gdb) on a Linux platform with the GCC compiler and the libstdc++ standard library.

A `std::vector` object typically maintains three internal pointers. In common implementations such as libstdc++, a vector declared on the stack stores the pointers `_M_start`, `_M_finish`, and `_M_end_of_storage`. The size and capacity of the vector are computed by taking differences between these pointers.

$$\begin{aligned} \text{size} &= \text{_M_finish} - \text{_M_start} \\ \text{capacity} &= \text{_M_end_of_storage} - \text{_M_start} \end{aligned}$$

where,

`_M_start` points to the beginning of the contiguous memory block that stores the vector's elements.

`_M_finish` points one position past the last constructed element, marking the logical end of the vector.

`_M_end_of_storage` points one position past the end of the allocated memory block, marking the vector's total capacity.

Code Example 8-1: vector.cpp

```
1 void f() {
2     std::vector<int> v = {1, 2, 3, 4}; // declares and initializes the vector
   ↪   with four integers
3 }
```

How to read the debugger

Lines beginning with (gdb) indicate commands entered into the debugger, while

↪ the indented lines show the corresponding output.

-> User-given comment

-> The following versions are used to perform the debugging

GNU gdb version 12.1

Ubuntu version 22.04.3 LTS (Jammy Jellyfish)

g++ version 11.4.0

-> The following compiler switches are used to build the code.

g++ -std=c++17 -O0 -g vector.cpp -o vector

-> -O0 = Preserve source-level behaviour

```

-> -g = Make program debuggable

gdb ./vector
  Reading symbols from ./vector...
(gdb) break main
  Breakpoint 1, main () at vector.cpp:4
(gdb) next
    std::vector<int> v = {1, 2, 3, 4};
(gdb) next
    return 0;
(gdb) print v
    std::vector of length 4, capacity 4 = {1, 2, 3, 4}
print &v
    (std::vector<int, std::allocator<int> > *) 0x7fffffff470  -> Address of
    ↪ v in stack
(gdb) print v._M_impl._M_finish - v._M_impl._M_start
    4 --> Size of vector
(gdb) print v._M_impl._M_end_of_storage - v._M_impl._M_start
    4 --> Size of storage
(gdb) x/1dw v._M_impl._M_start
    0x5555556aeb0: 1  --> First element at aeb0 heap address
Note: x/1dw --> x -> examine memory; /1 -> one unit of memory; d -> format
    ↪ (signed decimal); w -> unit size (word - 4 bytes on most systems)

(gdb) x/1dw v._M_impl._M_start+1
    0x5555556aeb4: 2
(gdb) x/1dw v._M_impl._M_start+2
    0x5555556aeb8: 3
(gdb) x/1dw v._M_impl._M_start+3
    0x5555556aebc: 4
(gdb) x/1dw v._M_impl._M_start+4
    0x5555556aec0: 0  -> Finish pointer address

```

C++ Standard Library Implementations

libstdc++ is the GNU Compiler Collection's implementation of the C++ Standard Library. When a C++ program is compiled with GCC, libstdc++ provides the concrete implementations of standard facilities such as containers (std::vector, std::array, std::map), algorithms (std::sort, std::find), iterators, memory allocators, and utilities like strings, streams, and exceptions. While the C++ standard specifies what these components must do and the guarantees they provide, it does not define how they are implemented internally. libstdc++ supplies those implementation details, which is why internal names such as `_M_start`, `_M_finish`, and `_M_end_of_storage` appear when debugging on Linux systems using GCC.

Besides libstdc++, there are several other widely used C++ Standard Library

implementations. `libc++` is the implementation developed by the LLVM project and is the default standard library on macOS and commonly used with Clang. MSVC STL is Microsoft's implementation of the C++ Standard Library, used with the MSVC compiler on Windows. Historically, STLport was another implementation aimed at portability, though it is largely obsolete today. All of these libraries conform to the same C++ standard and provide the same public interfaces, but they differ in internal structure, naming, performance characteristics, and debugging visibility.

Case 2: Vector declared on the stack with initialized members and reallocated memory space

```

1 void f() {
2     std::vector<int> v = {1, 2, 3, 4}; // declares and initializes the vector
    ↪ with four integers
3     v.push_back(5);
4     v.push_back(6);
5     return 0;
6 }

```

```

gdb ./vector
    Reading symbols from ./vector...
(gdb) break main
(gdb) run
    int main() {
(gdb) next
    std::vector<int> v = {1, 2, 3, 4};
(gdb) next
    v.push_back(5);
(gdb) print v
    std::vector of length 4, capacity 4 = {1, 2, 3, 4}
(gdb) x/1dw v._M_impl._M_start
0x55555556ceb0: 1
(gdb) x/1dw v._M_impl._M_start+1
0x55555556ceb4: 2
(gdb) x/1dw v._M_impl._M_start+2
0x55555556ceb8: 3
(gdb) x/1dw v._M_impl._M_start+3
0x55555556cebc: 4
(gdb) x/1dw v._M_impl._M_start+4
0x55555556cec0: 0
(gdb) x/1dw v._M_impl._M_start+5
0x55555556cec4: 0
(gdb) x/1dw v._M_impl._M_start+6
0x55555556cec8: 61761
(gdb) next
7         v.push_back(6);

```

```

(gdb) print v
$2 = std::vector of length 5, capacity 8 = {1, 2, 3, 4, 5}
(gdb) x/1dw v._M_impl._M_start
0x55555556ced0: 1
(gdb) x/1dw v._M_impl._M_start+1
0x55555556ced4: 2
(gdb) x/1dw v._M_impl._M_start+2
0x55555556ced8: 3
(gdb) x/1dw v._M_impl._M_start+3
0x55555556cedc: 4
(gdb) x/1dw v._M_impl._M_start+4
0x55555556cee0: 5
(gdb) x/1dw v._M_impl._M_start+5
0x55555556cee4: 0
(gdb) x/1dw v._M_impl._M_start+6
0x55555556cee8: 0
(gdb) x/1dw v._M_impl._M_start+7
0x55555556ceec: 0
(gdb) x/1dw v._M_impl._M_start+8
0x55555556cef0: 0
(gdb) x/1dw 0x55555556ced0
0x55555556ced0: 1
(gdb) x/1dw 0x55555556ceb4
0x55555556ceb4: 5
(gdb) x/1dw 0x55555556ceb0
0x55555556ceb0: 1431655788
(gdb) print &v
$3 = (std::vector<int, std::allocator<int> > *) 0x7fffffff470
(gdb) print v._M_impl._M_start
$4 = (std::_Vector_base<int, std::allocator<int> >::pointer) 0x55555556ced0
(gdb) print v._M_impl._M_finish
$5 = (std::_Vector_base<int, std::allocator<int> >::pointer) 0x55555556cee4
(gdb) print v._M_impl._M_end_of_storage
$6 = (std::_Vector_base<int, std::allocator<int> >::pointer) 0x55555556cef0
(gdb) print v
$7 = std::vector of length 5, capacity 8 = {1, 2, 3, 4, 5}
(gdb) next
8      return 0;
(gdb) print v
$8 = std::vector of length 6, capacity 8 = {1, 2, 3, 4, 5, 6}
(gdb) print v._M_impl._M_finish
$9 = (std::_Vector_base<int, std::allocator<int> >::pointer) 0x55555556cee8
(gdb) print v._M_impl._M_end_of_storage
$10 = (std::_Vector_base<int, std::allocator<int> >::pointer) 0x55555556cef0
(gdb) print v._M_impl._M_finish - v._M_impl._M_start
$11 = 6
(gdb)

```

8.2 Linked Lists

8.2.1 Member functions of linked list containers

TABLE 8.1 Common member functions of `std::list` and `std::forward_list` with description and examples

Function	Description	Example
<code>push_front()</code>	Inserts an element at the beginning (both containers).	<code>lst.push_front(10);</code>
<code>push_back()</code>	Appends an element at the end (<code>std::list</code>).	<code>lst.push_back(20);</code>
<code>pop_front()</code>	Removes the first element (both containers).	<code>lst.pop_front();</code>
<code>pop_back()</code>	Removes the last element (<code>std::list</code>).	<code>lst.pop_back();</code>
<code>insert()</code>	Inserts at a given position (<code>std::list</code>).	<code>lst.insert(it, 15);</code>
<code>insert_after()</code>	Inserts after a given iterator (<code>std::forward_list</code>).	<code>fl.insert_after(it, 15);</code>
<code>erase()</code>	Removes element at a given iterator (<code>std::list</code>).	<code>lst.erase(it);</code>
<code>erase_after()</code>	Removes element after a given iterator (<code>std::forward_list</code>).	<code>fl.erase_after(it);</code>
<code>remove()</code>	Removes all elements matching a value (both containers).	<code>lst.remove(10);</code>
<code>splice()</code>	Transfers elements between lists without copying (<code>std::list</code>).	<code>lst1.splice(it, lst2);</code>
<code>sort()</code>	Sorts elements (<code>std::list</code>).	<code>lst.sort();</code>
<code>reverse()</code>	Reverses elements (both containers).	<code>lst.reverse();</code>
<code>begin(), end()</code>	Iterators for traversal (<code>std::list</code>).	<code>for(auto it = lst.begin(); it != lst.end(); ++it)</code>

<code>before_begin()</code>	Iterator before first element	<code>auto it =</code>
	<code>(std::forward_list).</code>	<code>fl.before_begin();</code>

8.3 Hash Tables

8.3.1 What is `std::hash`?

`std::hash<T>` is a function object (functor) defined in the C++ standard library in `<functional>`. A functor is a class that overloads the function-call operator `operator()`, allowing an object of that class to be used like a function. Take a look at the simple example code.

```
class Square
{
public:
    int operator()(int x) const
    {
        return x * x;
    }
};
```

The class `Square` represents an object that can compute the square of a number. Inside the class, the function `operator()` is defined. In C++, `operator()` is called the function-call operator. When this operator is overloaded, objects of the class can be called like a function. After defining this class, an object of type `Square` can be created and used like a function:

```
Square sq;
int result = sq(5);
```

Here, `sq(5)` calls the overloaded `operator()` function, which returns 25.

In `std::hash`, the hashing object is implemented as a class template.

```
template<>
struct hash<int>
{
    size_t operator()(int key) const
    {
        return static_cast<size_t>(key);
    }
};
```

Here, `hash<int>` is a class and `operator()` allows objects of the class to be called like a function. The call `hasher(key)` invokes `hasher.operator()(key)`. The `template<>` indicates that this definition is an explicit specialization of a template. In the C++ standard library, `std::hash` is defined as a class template that can operate on many different types. Conceptually, it is declared as

```
template <class T>
struct hash;
```

This means the template can be instantiated for different types such as `hash<int>`, `hash<std::string>`, and `hash<double>`. However, different types often require different hashing strategies. Therefore, the library provides specialized implementations for particular types. The `template<>` indicates that the following structure is a specialization for a specific type, in this case `int`.

8.3.2 Member Functions of Associative Containers

The ordered associative containers in the C++ Standard Library — `std::map`, `std::multimap`, `std::set`, and `std::multiset` provide a rich set of member functions that support operations such as element insertion, removal, searching, iteration, and container management. While these containers differ in whether they store key–value pairs or keys only, and whether duplicate keys are permitted, they share a large portion of their interface. The following table provides a consolidated reference of the commonly used member functions, including a brief description and example usage. Symbols associated with each method indicate the containers for which the operation is available.

TABLE 8.3 Member functions of ordered associative containers

Method	Description	Example Usage
<code>operator[]</code> †	Accesses or inserts an element associated with a key.	<code>m["Alice"] = 90;</code>
<code>at()</code> †	Returns the value associated with a key. Throws an exception if the key does not exist.	<code>m.at("Alice");</code>
<code>insert()</code>	Inserts a new element into the container.	<code>c.insert({"Bob", 85});</code>
<code>insert_or_assign()</code> †	Inserts or replaces the value associated with an existing key.	<code>m.insert_or_assign("Alice", 95);</code>
<code>emplace()</code>	Constructs and inserts an element directly into the container.	<code>c.emplace("Alice", 90);</code>

Continued on next page

Method	Description	Example Usage
<code>emplace_hint()</code>	Inserts an element using a position hint.	<code>c.emplace_hint(c.begin(), "Bob", 85);</code>
<code>try_emplace()</code> †	Inserts an element only if the key does not already exist.	<code>m.try_emplace("John", 88);</code>
<code>erase()</code>	Removes elements from the container.	<code>c.erase("Bob");</code>
<code>clear()</code>	Removes all elements from the container.	<code>c.clear();</code>
<code>swap()</code>	Exchanges the contents of two containers.	<code>c1.swap(c2);</code>
<code>extract()</code>	Extracts a node from the container without destroying it (C++17).	<code>auto n = c.extract("Alice");</code>
<code>merge()</code>	Transfers elements from another container (C++17).	<code>c1.merge(c2);</code>
<code>find()</code>	Searches for an element with the specified key.	<code>auto it = c.find("Alice");</code>
<code>count()</code>	Returns the number of elements with the specified key.	<code>c.count("Alice");</code>
<code>contains()</code>	Checks if the container contains the specified key (C++20).	<code>c.contains("Alice");</code>
<code>lower_bound()</code>	Returns iterator to first element not less than the key.	<code>c.lower_bound("Alice");</code>
<code>upper_bound()</code>	Returns iterator to first element greater than the key.	<code>c.upper_bound("Alice");</code>
<code>equal_range()</code>	Returns range of elements matching the key.	<code>auto r = c.equal_range("Alice");</code>
<code>begin()</code>	Returns iterator to first element.	<code>c.begin();</code>
<code>end()</code>	Returns iterator to one past the last element.	<code>c.end();</code>
<code>cbegin()</code>	Returns constant iterator to first element.	<code>c.cbegin();</code>
<code>cend()</code>	Returns constant iterator to one past the last element.	<code>c.cend();</code>
<code>rbegin()</code>	Returns reverse iterator to last element.	<code>c.rbegin();</code>
<code>rend()</code>	Returns reverse iterator before first element.	<code>c.rend();</code>

Continued on next page

Method	Description	Example Usage
<code>crbegin()</code>	Returns constant reverse iterator to last element.	<code>c.crbegin();</code>
<code>crend()</code>	Returns constant reverse iterator before first element.	<code>c.crend();</code>
<code>size()</code>	Returns number of elements in the container.	<code>c.size();</code>
<code>empty()</code>	Checks whether the container is empty.	<code>c.empty();</code>
<code>max_size()</code>	Returns maximum number of elements possible.	<code>c.max_size();</code>
<code>key_comp()</code>	Returns comparison function used to order keys.	<code>c.key_comp();</code>
<code>value_comp()</code>	Returns comparison function used to compare values.	<code>c.value_comp();</code>
<code>get_allocator()</code>	Returns allocator associated with the container.	<code>c.get_allocator();</code>

Symbols:

† Available only in `std::map`. All other functions apply to unordered `map`, `multimap`, `set`, and `multiset`.

8.3.3 Code: Signed Feature Hashing with Logistic Regression

This appendix provides a complete, compilable C++ implementation of feature hashing combined with a logistic regression model. The example demonstrates how raw text can be transformed into a fixed-size numerical feature vector using hashing, followed by probability estimation using the sigmoid function.

Overview

The program performs the following steps:

1. Tokenizes input text into individual words
2. Maps each word to a feature index using a hash function
3. Assigns a signed contribution to reduce bias from collisions
4. Accumulates values into a fixed-size feature vector
5. Computes a linear combination using a weight vector
6. Applies the sigmoid function to obtain a probability

Complete Program

Code Example 8-2: Identity hash example

```

1  #include <iostream>
2  #include <sstream>
3  #include <vector>
4  #include <cmath>
5  #include <functional>
6  using namespace std;
7
8  int main() {
9      // Input text
10     string text = "good product but bad packaging";
11     stringstream ss(text);
12     string word;
13
14     // Predefined weight vector (model parameters)
15     vector<float> weights = {0.1, 0.2, -0.1, -0.7, 0.8, 0.5};
16
17     // Feature vector initialized to zero
18     vector<float> features(weights.size(), 0.0);
19
20     // Hash function
21     hash<string> hasher;
22
23     float z = 0.0;
24
25     // Step 1-3: Tokenization and feature hashing
26     while (ss >> word) {
27         // Map token to index
28         size_t index = hasher(word) % weights.size();
29
30         // Assign sign (+1 or -1)
31         int sign = (index % 2 == 0) ? 1 : -1;
32
33         // Accumulate feature value
34         features[index] += sign;
35     }
36
37     // Step 4: Linear combination (dot product)
38     for (size_t i = 0; i < features.size(); i++) {
39         z += weights[i] * features[i];
40     }
41
42     // Step 5: Sigmoid function
43     float prob = 1.0 / (1.0 + exp(-z));
44
45     // Output probability
46     cout << "Predicted Probability: " << prob << endl;
47
48     return 0;

```

```
49 }
```

Discussion

This implementation illustrates several important properties of feature hashing:

- The dimensionality of the feature space is fixed and independent of vocabulary size.
- Hash collisions are expected and handled through accumulation.
- The use of a signed hash function helps mitigate systematic bias introduced by collisions.
- The resulting feature vector is suitable for linear models such as logistic regression.

Connection to Chapter Content

This example complements the discussion in Chapter 6 by providing a concrete implementation of hash-based feature construction and its integration with a probabilistic classification model.

Compilation

To compile and run the program:

```
g++ -std=c++17 feature_hashing.cpp -o feature_hashing
./feature_hashing
```

Expected Output

The program outputs a probability value in the range between 0 to 1, representing the likelihood of the input text belonging to a particular class.

References

- [1] *Oxford english dictionary*. Oxford University Press, 2024.
- [2] *Java platform, standard edition 8 API specification*. Oracle Corporation, 2014. Available: <https://docs.oracle.com/javase/8/docs/api/java/lang/String.html>
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*, 3rd ed. MIT Press, 2009.
- [4] D. E. Knuth, *The art of computer programming, volume 3: Sorting and searching*, 2nd ed. Reading, MA: Addison-Wesley, 1998.